
From Factors to Portfolios: A Survey of AI-Driven Quantitative Investing

Zuohao Liu[†]
Peking University

liuzuohao26@stu.pku.edu.cn

Xinxin Xu[†]
Peking University

xuxinxin@stu.pku.edu.cn

Chengxin Hu
Peking University

huchengxin26@stu.pku.edu.cn

Bowen Deng
Peking University

devin@stu.pku.edu.cn

Vincent Fu^{*}
Peking University

vvincenttttt@gmail.com

[†]Equal contribution. ^{*}Corresponding author.

Abstract

The rapid expansion of multimodal financial data and the inherent non-stationarity of markets have exposed critical limitations of traditional linear quantitative frameworks. In response, the integration of Deep Reinforcement Learning and Large Language Models has catalyzed a paradigm shift from manual feature engineering to autonomous financial intelligence. To systematically synthesize these cross-disciplinary advancements, this paper reconceptualizes modern quantitative investment not as a collection of isolated predictive tasks, but as an **Four-Step Multi-Factor Investing Framework** organized along four computationally interconnected stages: *Factor Mining*, *Factor Combination*, *Risk Control*, and *Portfolio Management*. Through this unified framework, we trace the evolutionary trajectory of each stage, showing how technical innovations propagate through the pipeline: factor mining has progressed from stochastic symbolic search to LLM-driven semantic reasoning; factor combination has shifted from simple linear weighting to synergistic aggregation that resolves multicollinearity; risk control has evolved from static exposure limits to the active prediction of forward-looking tail-risk indicators; and portfolio management has advanced from static convex solvers to DRL orchestrated dynamic allocation under real-world market frictions. We further provide a structured taxonomy of multimodal datasets and benchmark simulation environments, and present a systematic comparative summary of state-of-the-art performance across diverse market regimes. This survey establishes a structural blueprint for evaluating cross-modal AI models in finance, identifies critical bottlenecks including semantic hallucinations and Out-of-Distribution fragility, and outlines open research directions toward fully autonomous multi-agent trading systems.

1 Introduction

Quantitative investment has evolved through several distinct paradigms over the past few decades. Early statistical arbitrage strategies exploited short-term price discrepancies across related instruments, while momentum-based and trend-following approaches captured persistent directional movements in commodity and foreign exchange markets. Commodity trading advisor strategies systematized these directional bets

across futures markets, and factor-based equity investing progressively matured into the modern **Multi-Factor Framework** that dominates institutional asset management today. Among these paradigms, **Multi-Factor Models** provide a particularly general methodology for capturing risk premia across large cross-sections of assets, decomposing expected returns into exposures to a small set of common factors and unifying previously fragmented strategies under a single risk-attribution framework.

A typical multi-factor model pipeline comprises four interconnected stages: factor mining, factor combination, risk control, and portfolio management. In traditional quantitative firms, each stage relies heavily on specialized human researchers. Factor mining requires quantitative researchers to manually craft predictive signals from market microstructure, fundamental data, or alternative datasets, often iterating through hundreds of candidate formulas before reaching deployable alphas. Factor combination involves hand-tuning weighting schemes across these candidates to maximize out-of-sample performance. Risk control demands dedicated risk managers who monitor exposure to market-wide shocks, tail events, and unintended style drift. Portfolio management requires portfolio managers to translate the resulting signals into executable allocation decisions under operational and regulatory constraints. This human-driven paradigm demands substantial investments in talent, computational infrastructure, and iterative backtesting, while still suffering from limited scalability, cognitive biases, and slow adaptation to regime shifts, motivating the search for systematic automation alternatives at every stage.

The rapid advancement of artificial intelligence over the past decade has fundamentally reshaped this landscape and opened systematic alternatives to the manual pipeline. Early automation efforts were anchored in symbolic evolutionary search, where Genetic Programming (GP) and its variants evolved mathematical factor expressions through mutation and crossover operations, providing the first scalable alternative to hand-crafted factor discovery. Subsequently, deep learning and Deep Reinforcement Learning (DRL) introduced powerful non-linear modeling capabilities, enabling end-to-end optimization of trading and allocation policies directly from market observations. More recently, the emergence of Large Language Models (LLMs) has opened new possibilities: foundation models have demonstrated strong capabilities across scientific domains, from solving Fields-Medal-level mathematical problems to predicting three-dimensional protein structures with high accuracy and automating complex scientific reasoning. These capabilities are increasingly being transferred to the financial domain, where LLMs serve as semantic signal extractors from news and disclosures, autonomous research agents that orchestrate multi-step discovery workflows, and reasoning backbones that can be embedded throughout the multi-factor pipeline. Rather than replacing traditional quantitative methods wholesale, these AI techniques are increasingly used as functional components embedded within different stages of the pipeline, complementing and gradually substituting the manual tasks previously performed by human researchers.

To organize the rapidly expanding literature on AI-driven automation of the multi-factor pipeline, this paper adopts a structural abstraction of modern quantitative investment as a multi-stage information processing system. We model the system as a **Four-Step Multi-Factor Investing Framework** with a layered dependency structure and implicit feedback, in which information is progressively transformed from raw observations into actionable portfolio decisions, while allowing interactions and regime-dependent coupling across components. Within this abstraction, we decompose the investment pipeline into the same four stages introduced above, namely **Factor Mining**, **Factor Combination**, **Risk Control**, and **Portfolio Management**, and systematically review how GP, DRL, LLM-based, and other AI methods are reshaping each stage. Due to the propagation of estimation errors and modeling bias across stages, understanding inter-stage interactions is critical for robust system design.

1.1 Notation and Problem Setup

Modern quantitative investment can be viewed as a **Four-Step Multi-Factor Investing Framework** that progressively transforms heterogeneous market information into executable portfolio decisions. Rather than treating financial prediction as a collection of isolated machine learning tasks, we adopt a structural abstraction in which information flows through a sequence of dependent stages, with each stage refining the output of the previous one. This layered formulation reflects the fact that information flows in financial systems are inherently non-stationary and may exhibit feedback effects across stages. Figure 1 summarizes this architecture, depicting the four stages, Factor Mining, Factor Combination, Risk Control, and Portfolio

Management, together with the implicit feedback channel that routes realized allocation outcomes back into upstream modules.

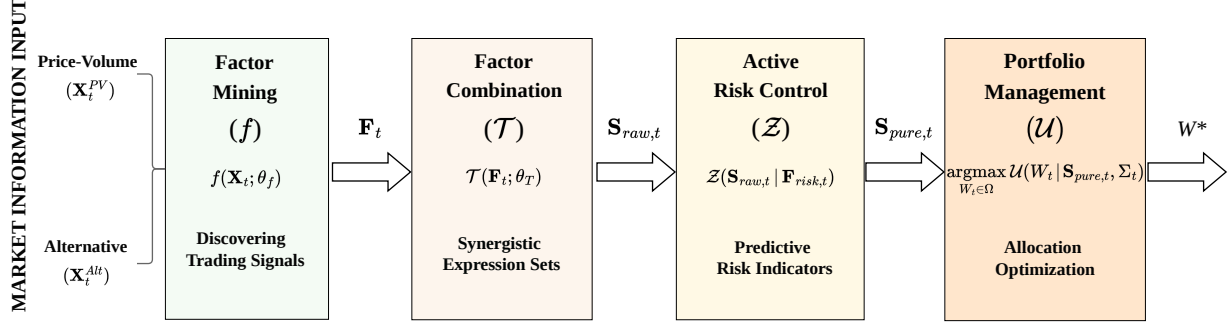


Figure 1: **The Four-Step Multi-Factor Investing Framework.** This unified four-stage abstraction illustrates the transformation of the market tensor X_t . Information is processed through Factor Mining (f), Factor Combination ($\mathcal{T}$), Risk Control ($\mathcal{Z}$), and Portfolio Management ($\mathcal{U}$), ultimately producing portfolio weights W_t^* . Unlike strict DAG formulations, this framework allows implicit feedback and regime-dependent interactions across stages.

In what follows, we first introduce the input data representation, then walk through each of the four stages in turn. For each stage, we provide an intuitive description of what the stage does in plain language, followed by a single formal equation that captures its input–output relationship. The four stage-level equations are finally assembled into an overall pipeline formulation.

Input. At each cross-sectional time step t , the system observes the market information $X_t = [X_t^{PV}, X_t^{Alt}]$, where X_t^{PV} represents price-volume data such as prices, trading volumes, and technical indicators, and X_t^{Alt} represents alternative data including financial news, earnings-call transcripts, macroeconomic and fundamental indicators, and regulatory filings.

Task 1: Factor Mining. The first stage is concerned with extracting predictive signals from raw market data. Concretely, the goal is to identify features, known as *factors*, that are informative about the future cross-sectional ranking or directional movement of assets, or to directly produce tradable signals without going through an explicit factor representation. Researchers have approached this stage through a wide range of methods: hand-crafted formula design rooted in financial intuition, genetic programming that evolves mathematical expressions through mutation and crossover, reinforcement learning policies that frame factor construction as a sequential search, and more recently large language models that translate natural-language market hypotheses into executable factor code or directional predictions. Regardless of the method, the essential output of this stage is a set of predictive representations that summarize actionable information from the high-dimensional and noisy input.

Formally, the factor mining module applies a set of extraction operators $f(\cdot; \theta_f)$ to the market information X_t , producing a collection of predictive factor representations F_t :

$$F_t = f(X_t; \theta_f). \quad (1)$$

Task 2: Factor Combination. The factors produced in the previous stage are typically noisy, highly correlated with each other, and individually weak in predictive power. Directly trading on a single factor often leads to unstable performance and concentration in specific risk exposures. The second stage therefore aggregates multiple factors into a unified signal that preserves their complementary predictive content while suppressing redundancy and noise. Methods at this stage range from classical linear and statistical aggregation, to tree-based and neural-network-based supervised learning that capture non-linear factor interactions, and further to reinforcement learning and LLM-enhanced approaches that dynamically adjust factor weights in response to changing market conditions. The output is a single composite signal that serves as the working predictive signal for the downstream stages.

Formally, the combination operator $\mathcal{T}(\cdot; \theta_T)$ aggregates the factor representations F_t into a unified raw signal $S_{raw,t}$:

$$S_{raw,t} = \mathcal{T}(F_t; \theta_T). \quad (2)$$

Task 3: Risk Control. Even after combination, the signal may still carry unwanted exposures to systematic risk sources, such as market-wide movements, sector or style tilts, and may behave poorly under extreme market conditions such as volatility spikes or liquidity crises. The third stage adjusts the combined signal to remove these unintended exposures and to incorporate explicit risk considerations. Typical operations include cross-sectional orthogonalization that strips out common factor structure, regime-aware volatility scaling that modulates exposure based on the prevailing market state, and tail-risk truncation that bounds potential losses using volatility, Value-at-Risk (VaR), or Conditional VaR estimates. Recent LLM-based approaches further extract risk-relevant signals from textual sources such as news and filings, allowing semantic risk perception to enter the adjustment process. The result is a purified, risk-adjusted signal that is more robust to distributional shifts.

Formally, the risk control operator $\mathcal{Z}(\cdot)$ adjusts the raw signal $S_{raw,t}$ using exogenous risk information $F_{risk,t}$, producing a purified signal $S_{pure,t}$:

$$S_{pure,t} = \mathcal{Z}(S_{raw,t} \mid F_{risk,t}). \quad (3)$$

Task 4: Portfolio Management. The final stage decides how the risk-adjusted signal is turned into actual money, that is, how much of each asset to buy or sell. This "how much" question operates at two levels. At the *macro* level, a manager distributes a given amount of capital across broad asset classes: given, say, one hundred million in funds, how much goes to equities, how much to commodities, how much to cryptocurrencies, and how much to prediction markets. At the *micro* level, a manager constructs a hedged, long-short book within a single market: after the signal ranks the constituents (e.g., the 1,000 stocks of the CSI 1000), the manager buys the top quantile expected to appreciate (e.g., the top 25%) and shorts the bottom quantile expected to depreciate (e.g., the bottom 25%), and must still decide both position sizing and breadth, such as whether a high-conviction name receives 1% or 10% of the budget, whether a short receives 10% or 20%, and whether to trade only the two extreme quantiles (top/bottom 25%), a wider window (top/bottom 30%), or even the entire ranking. Methods for carrying out this allocation include classical methods that solve a static convex program, constrained optimizers with modern solver technology that scale the formulation to large universes under realistic constraints, RL methods that learn allocation policies from sequential interaction with the market under trading frictions, and more recent LLM-based agentic systems that decompose the process into specialized roles such as analysts, risk managers, and traders, coordinated by an LLM-based meta-agent that injects semantic reasoning about macro regimes and asset-specific events. The output of this stage is the final portfolio weight vector that will be deployed in the market.

Formally, given the market structure information summarized by the asset covariance matrix $\Sigma_t \in \mathbb{R}^{N \times N}$ estimated from the observed market data, and a set of trading constraints Ω (position limits, turnover bounds, transaction cost models), the portfolio management module determines the optimal allocation vector W_t^* by solving a constrained utility optimization problem:

$$W_t^* = \arg \max_{W_t \in \Omega} \mathcal{U}(W_t \mid S_{pure,t}, \Sigma_t). \quad (4)$$

Overall Pipeline. Together, Equations (1)–(4) constitute the overall quantitative investment pipeline, in which heterogeneous market information X_t is progressively transformed into portfolio allocations W_t^* through a sequence of nested mappings:

$$\begin{cases} F_t = f(X_t; \theta_f) \\ S_{raw,t} = \mathcal{T}(F_t; \theta_T) \\ S_{pure,t} = \mathcal{Z}(S_{raw,t} \mid F_{risk,t}) \\ W_t^* = \arg \max_{W_t \in \Omega} \mathcal{U}(W_t \mid S_{pure,t}, \Sigma_t) \end{cases} \quad (5)$$

The above formulation characterizes a sequential decision abstraction of modern quantitative investment systems. Within this framework, recent advances in genetic programming, deep reinforcement learning, and LLM-based methods can be interpreted as functional components that influence multiple stages of the pipeline, including signal generation, risk adjustment, and allocation optimization. The remainder of this survey is organized along these four stages, with each technical section devoted to a systematic review of AI-driven methods at the corresponding stage.

1.2 Survey Methodology

This literature review was conducted through a systematic search process across major academic databases including arXiv, Google Scholar, Semantic Scholar, and SSRN. The search query combined keywords related to quantitative finance, reinforcement learning, and large language models, including variants of quantitative investment, reinforcement learning trading, LLM finance, factor mining, portfolio optimization and risk control.

Inclusion criteria. Papers were included if they (1) proposed or evaluated AI-based methods for quantitative investment tasks, (2) were published in peer-reviewed venues or as preprints between 2017 and 2026, and (3) provided empirical evaluation on standard financial datasets. To ensure reproducibility, papers without clearly specified evaluation protocols were excluded. The screening process involved title and abstract review followed by full-text assessment. After deduplication across databases, approximately 280 papers were initially identified, with 120 included in the final review based on methodological rigor and empirical contribution.

Classification framework. The included papers were categorized along our four-stage pipeline (Factor Mining, Factor Combination, Risk Control, Portfolio Management) and further grouped by methodological paradigm (symbolic search, deep learning, reinforcement learning, LLM-based approaches). For each category, we synthesized core methodological innovations and reported key empirical results as published in the original works, including Information Coefficient, Sharpe Ratio, and cumulative return metrics where available.

1.3 Contributions

The main contributions of this survey are summarized as follows:

- **A structural abstraction of quantitative investment systems:** We propose a unified multi-stage abstraction of quantitative investment systems, organizing the pipeline into Factor Mining, Factor Combination, Risk Control, and Portfolio Management. Unlike strictly modular formulations, this framework emphasizes inter-stage dependencies and implicit feedback effects across the system.
- **A cross-stage taxonomy of AI-driven methodologies:** We provide a systematic review of AI-based methods in quantitative finance, including symbolic search, deep learning, reinforcement learning, and LLM-based approaches. We highlight their roles as functional components that may operate across multiple stages rather than being confined to a single module.
- **A unified perspective on risk-aware portfolio construction:** We jointly analyze risk control and portfolio optimization as coupled decision processes under uncertainty. The survey covers classical optimization frameworks as well as recent learning-based approaches, including reinforcement learning and LLM-assisted decision systems under realistic market constraints.
- **Discussion of limitations and open challenges:** We identify key unresolved challenges, including robustness under distribution shifts, reliability of LLM-generated signals, multimodal information integration, and the gap between offline evaluation and real-world deployment in financial markets.

The remainder of this paper is organized as follows. Section 2 establishes the foundational vocabulary, surveying representative benchmark methods organized into four methodological paradigms: machine learning, genetic programming, reinforcement learning, and large language models (with the LLM family further

decomposed into general-purpose LLMs, financial LLMs, downstream applications, and agentic systems), which serve both as conceptual predecessors and as baselines for later comparison. Section 3 reviews Factor Mining under a dichotomy of discovery targets, formulaic alpha generation versus direct signal generation, tracing the evolution from genetic programming and reinforcement learning to LLM-based and generative-network paradigms. Section 4 covers Factor Combination along four paradigms of increasing expressiveness, from classical linear weighting and supervised nonlinear aggregation to reinforcement-learning-based dynamic routing and LLM-enhanced closed-loop optimization. Section 5 examines Risk Control through two paradigms, statistical methods (cross-sectional orthogonalization, regime-aware volatility scaling, and tail-risk truncation) and LLM-based methods that extract risk perception from textual data. Section 6 analyzes Portfolio Management along a two-level taxonomy—single-period versus multi-period formulations, and within single-period construction, macro-level allocation across asset classes versus micro-level position sizing within a market—spanning classical optimization, constrained optimizers, deep reinforcement learning, and LLM-based agentic systems. Section 7 surveys datasets along two orthogonal axes (asset class and data modality), simulation environments, and evaluation metrics and frameworks. Section 8 discusses current limitations and open challenges. Section 9 concludes.

2 Preliminaries: Foundational Concepts and Classical Methods

This section provides the foundational knowledge required to understand the methodologies surveyed in this paper. Modern AI-driven quantitative investment does not exist in isolation. It builds upon a rich heritage of classical financial theory, statistical learning, symbolic search, reinforcement learning, and more recently, large language model paradigms. Readers unfamiliar with these building blocks may find the technical depth of subsequent sections challenging. We therefore establish a common vocabulary by surveying the representative benchmark methods that recur throughout the literature, organized by methodological paradigm: Machine Learning, Genetic Programming, Reinforcement Learning, and Large Language Models. The LLM paradigm is further decomposed into general-purpose LLMs, financial LLMs, downstream LLM applications, and agentic LLM systems, reflecting the rapid specialization of this family. These methods serve dual roles: they are both the conceptual predecessors that modern AI approaches seek to improve upon, and the standard baselines against which new methods are empirically compared in Section 7.

2.1 Machine Learning.

The machine learning era introduced powerful non-linear modeling capabilities to quantitative finance. Tree-based ensemble methods, particularly XGBoost (Chen and Guestrin, 2016) and LightGBM (Ke et al., 2017), have become the de facto standard for factor combination tasks due to their ability to capture hierarchical feature interactions, handle missing data gracefully, and provide interpretable feature importance rankings. Deep sequence models, including LSTM (Hochreiter and Schmidhuber, 1997) and Transformer (Vaswani et al., 2017) architectures, enable the modeling of temporal dynamics in financial time series, capturing long-range dependencies that traditional autoregressive models miss. Understanding these architectures is essential for the technical sections that follow, as they form the backbone of modern factor combination (Section 4) and portfolio management (Section 6) methodologies.

2.2 Genetic Programming.

Automated factor discovery has a rich history rooted in symbolic search. Genetic Programming and its software implementations (e.g., gplearn) evolve mathematical expressions through mutation and crossover operations, exploring the vast space of possible factor formulas. The Alpha101 library (Kakushadze, 2016), developed by WorldQuant, represents a curated collection of 101 handcrafted quantitative factors that have been widely validated across global equity markets. These symbolic approaches serve as critical baselines for evaluating newer RL-based and LLM-based factor mining methods discussed in Section 3.

2.3 Reinforcement Learning.

Reinforcement Learning methods frame trading and portfolio management as sequential decision-making problems under uncertainty. The standard policy-gradient and actor-critic algorithms, including Proximal Policy Optimization (PPO) (Schulman et al., 2017), Advantage Actor-Critic (A2C) (Mnih et al., 2016), and Deep Deterministic Policy Gradient (DDPG) (Lillicrap et al., 2015), form the algorithmic toolkit for modern RL-based trading agents. These methods are extensively discussed in Sections 3, 4, and 6, making familiarity with their theoretical foundations a prerequisite for understanding the technical discussion.

2.4 Large Language Models.

The most recent technological era introduces Large Language Models to quantitative finance. We organize LLM-related preliminaries around three categories: **General LLMs** used as off-the-shelf reasoning backbones, **Financial LLMs** specialized through domain-specific pre-training and fine-tuning, and **LLM Agents** that orchestrate tool-augmented autonomous research workflows. We also discuss training-data bias and predictive reliability as shared concerns across these categories. Frameworks like Evolution of Heuristics (EoH) (Liu et al., 2024a) and Reflective Evolution (ReEvo) (Ye et al., 2024) represent state-of-the-art LLM-guided evolutionary search that combines the symbolic reasoning capabilities of LLMs with traditional evolutionary algorithms; these baselines are essential for evaluating the growing class of LLM-based methods surveyed throughout this paper.

General LLMs. General LLMs, such as ChatGPT, GPT-4, Claude, DeepSeek, and the LLaMA family, are increasingly used in quantitative finance as zero-shot or few-shot reasoning backbones without domain-specific fine-tuning. Their appeal lies in requiring no additional task-specific financial fine-tuning: a general-purpose model can be prompted directly to classify the sentiment of a news headline, summarize an earnings call transcript, or judge whether a firm-specific event is bullish or bearish for the stock. This does not imply that financial documents or historical market outcomes were absent from its pre-training corpus. Mumtaz and Mumtaz (2023) demonstrated ChatGPT’s ability to predict daily tech stock trends purely from the semantic tone of Twitter data, while Chen et al. (2025b) confirmed that foundation models like ChatGPT and DeepSeek-R1 parse complex market contexts in Wall Street Journal headlines far better than traditional dictionaries or standard BERT architectures. These models serve both as direct zero-shot baselines and as reasoning components inside the agentic systems surveyed in the 2.4 below. These models can nevertheless exhibit limited numerical sensitivity and hallucinations. Financial LLMs seek to improve domain-specific capabilities through further pre-training and instruction tuning, but such adaptation alone does not establish predictive reliability or rule out training-data contamination.

Financial LLMs. As foundational infrastructures, FinLLMs act as pure textual feature extractors on alternative data. Starting from early encoder-only sentiment scorers like Araci (2019) (FinBERT), initial generative approaches like Wu et al. (2023) (BloombergGPT) combined proprietary financial texts with general corpora. Yang et al. (2023a) (FinGPT) decentralized this process using low-rank adaptations, directly mapping text to market impact. Building on this accessibility, Zhang et al. (2023) (Instruct-FinGPT) applied instruction tuning to improve numerical sensitivity and contextual understanding, achieving high computational efficiency. The open-source ecosystem expanded rapidly with Xie et al. (2023) (PIXIU), which introduced the FIT instruction dataset and the LLaMA-based FinMA model, alongside the FLARE evaluation benchmark. To capture specific financial reasoning, Yang et al. (2023b) (InvestLM) instruction-tuned LLaMA-65B on highly curated datasets (including CFA exams and SEC filings) using expanded RoPE context windows, achieving GPT-3.5-level performance. At the same time, Chen et al. (2023a) (DiscFinLLM) introduced a Chinese-focused Multi-Expert Fine-Tuning (MEFF) approach, using modular LoRA experts for distinct tasks like consulting, document processing, calculating, and retrieval. Pushing multi-modal capabilities further, Bhatia et al. (2024) (FinTral) used a 20-billion token FinSet and distilled Direct Preference Optimization (dDPO) on Mistral-7B, natively integrating vision encoders and external Python math tools. Finalizing this pre-training trajectory, the Huang et al. (2024) (Open-FinLLMs) suite released models trained on a 52-billion token corpus that uniquely combines text, tabular, and time-series data, consistently outperforming much larger proprietary models.

To run these massive architectures efficiently within quantitative research pipelines, researchers have heavily adopted High-Performance Computing (HPC) and parameter-efficient techniques. Liu et al. (2024b) optimized large-scale training using activation recomputation, pipeline parallelism, and 4-bit quantized LoRA, significantly lowering VRAM requirements. To demonstrate the effectiveness of these compressed models, Luo and Gong (2024) showed that a quantized Llama2-7B, equipped with either Supervised Fine-Tuning (SFT) or a dedicated classification head, outperforms FinBERT in capturing nuanced market syntax. Taking this further, Konstantinidis et al. (2024) (FinLlama) used an 8-bit quantized LoRA framework to not only classify sentiment but accurately quantify its intensity, combining computational efficiency with actionable investment metrics. Moving beyond standard SFT, which often suffers from data memorization, Iacovides et al. (2025) (FinDPO) applied Direct Preference Optimization to align LLMs directly with human financial preferences. By introducing a novel logit-to-score conversion mechanism, FinDPO mapped discrete sentiment preferences into continuous probabilities, allowing causal LLMs to be seamlessly integrated into high-frequency, daily long-short portfolio rebalancing.

To standardize evaluation globally and prevent data leakage, a critical vulnerability explicitly addressed by Guo et al. (2023) (FinLMEval), which curated proprietary datasets (such as QA from earnings calls and ESG reports) to prove models were not merely memorizing public benchmarks, Xie et al. (2024) (FinBen) established a multi-dimensional benchmark evaluating strategic decision-making. Most recently, Qian et al. (2025) (Fin-o1) introduced the FinCoT dataset, targeting exact financial reasoning, while Chen et al. (2025d) (StockBench) provided a contamination-free simulation environment for testing state-of-the-art models in continuous, multi-day trading loops. Beyond dataset-level evaluation, methodological work on reasoning alignment has sought to make the chain-of-thought process itself more reliable in financial settings: Singhal and Singhal (2025) (AD-FCoT) introduced Analogical-Driven Financial Chain-of-Thought, grounding the model’s reasoning against historical causal templates rather than relying entirely on internal statistical correlations, thereby reducing factual errors in extraction-heavy tasks such as fundamental signal generation.

LLM Bias Problems. LLM outputs reflect both the information contained in their training corpora and the statistical preferences induced by their training objectives. For quantitative finance, four related but distinct concerns are training-distribution dependence, memorization, look-ahead bias, and investment bias. These concerns should not be conflated with social bias or treated as evidence that all LLM predictions are unreliable.

Training-distribution dependence. McCoy et al. (2023) show that GPT-3.5 and GPT-4 perform better on more probable tasks, inputs, and outputs across controlled evaluations, even when the correct answer is deterministic. This demonstrates that apparent reasoning ability can depend on familiarity with the statistical structure of language. Although these experiments do not directly test financial forecasting, they motivate evaluating financial applications across unfamiliar market conditions rather than assuming that success on familiar examples transfers to new regimes.

Memorization and leakage. Carlini et al. (2021) recover verbatim training examples, including personally identifiable information, from GPT-2 through model queries, establishing that language models can retain extractable training content. Training-data extraction is distinct from evaluation contamination: the former concerns disclosure of memorized content, whereas the latter arises when evaluation examples or their answers have already influenced model training. Neither phenomenon alone establishes that every prediction is memorization-driven.

Look-ahead bias in financial forecasting. When evaluating a forecast at historical time t , information learned from reports published after t can enter through model parameters even if the prompt itself contains only contemporaneous information. Gao et al. (2025) construct a Lookahead Propensity measure from firm-and-date recall queries and, under their econometric assumptions, diagnose memorization-related contamination in stock-return and capital-expenditure forecasts. Their findings illustrate why historical predictive accuracy need not measure forecasting ability available at the original decision time. More broadly, Kong et al. (2026) argue that financial LLM evaluation must explicitly address temporal leakage, unsupported narratives, and uncertainty calibration. Accordingly, evaluation should document model versions and training cutoffs, enforce point-in-time availability in retrieval and tools, and include prospective or demonstrably unseen observations together with tests across market regimes. Domain adaptation and fluent explanations alone do not establish

these properties. Consistent with this concern, Alonso et al. (2024) survey how look-ahead information can leak into financial LLM outputs and which applications are most exposed, Liang (2024) document look-ahead contamination in LLM-generated financial forecasts, and BRADFORD (2026) show that look-ahead bias also degrades numerical-reasoning evaluations beyond finance-specific prompts.

Investment bias in financial decisions. Beyond leakage of information, LLMs exhibit directional preferences of their own when rendering investment judgments. Lee et al. (2025a) measure such implicit investment bias by eliciting arguments for and against stock debates, finding model-specific preferences—for example, toward technology stocks, large-capitalization firms, and contrarian or reverse strategies—that escalate into confirmation bias as evidence accumulates. Park et al. (2026) show that this bias can be steered at inference time: an intervention on a single neuron acts as a continuous "investment-bias dial" that adjusts buy-versus-sell propensity without modifying prompts or parameters and, in long-context settings, is more stable than system-prompt instruction; the intervention also shapes which evidence the model retrieves and the resulting portfolios. Related human biases persist in LLMs as well: Nguyen (2024) document anchoring effects in LLM judgments and evaluate mitigation strategies, and Hantel (2026) show that even small anchoring biases can be amplified when a market is populated by language-model agents, eventually collapsing prices. Hu and Zhao (2026) provide a comprehensive benchmark of LLM decision-making under human biases, including anchoring, framing, and overconfidence, in the finance domain. Mitigation itself requires care: Wang and Gu (2026) document gender and racial bias in LLM-generated investment advice and Tan and Ding (2026) survey the rapidly expanding literature on responsible financial LLMs. Financial pipelines should therefore audit the bias regimes relevant to their deployment setting rather than assume neutrality.

LLM Agent. Beyond model backbones, recent quantitative research systems share a common layer of agentic infrastructure. Rather than treating LLMs as isolated reasoners, the field has progressively moved toward agent architectures that combine reasoning, tool use, and persistent memory. To organize this rapidly expanding landscape, we adopt a six-dimension taxonomy that reflects how agentic systems are actually engineered in practice: **(1) Model Context Protocol (MCP)** for standardized tool-data interoperability, **(2) Skill-based architectures** for reusable workflow modularization, **(3) Harnesses** that provide the agent runtime environment, **(4) Single- and Multi-agent System** topologies for orchestrating collaboration, **(5) Continuous Learning** mechanisms that accumulate experience across research cycles, and **(6) Autonomous Research Systems** that assemble the preceding components into end-to-end, self-directed scientific investigation pipelines. These components are not standalone factor mining methods but shared prerequisites that the agentic systems surveyed in later sections rely upon.

(1) MCP. A persistent structural challenge in LLM-based quantitative research is system integration: it is inherently difficult to connect a generative model directly to proprietary financial databases, real-time exchange feeds, and localized backtesting engines. The Model Context Protocol, introduced as an open standard for connecting LLMs to external data sources and tools, addresses this by providing a uniform client-server interface. With MCP, an agent no longer needs custom integration code for every new dataset; instead, it dynamically discovers and connects to MCP servers that expose quantitative utilities through a shared schema. The official MCP servers repository¹ maintains a growing collection of reference implementations spanning file systems, databases, developer tools, and productivity services, forming the baseline ecosystem that domain-specific servers build upon. In the financial domain, the open-source **LLMQuant Data** MCP server exemplifies this paradigm, exposing high-frequency limit order book data, alternative sentiment databases, macroeconomic calendars, and **qlib**-style factor libraries (Yang et al., 2020b) through a single endpoint. Beyond data access, MCP servers also wrap computational resources such as backtesting engines, optimization solvers, and code execution sandboxes, allowing the agent to treat remote execution as a first-class tool call. This standardization is a prerequisite for the cross-platform agent behaviors discussed in the Harness and Multi-agent dimensions below, since a skill or agent that targets the MCP interface can be reused across any compliant runtime.

(2) Skill-based Architectures. Alongside protocol standardization, the field is moving toward Skill-based architectures that decompose the research lifecycle into reusable, composable units. A Skill is a self-contained module that combines a prompt template, a specific tool-set (such as an optimization solver or a backtesting

¹<https://github.com/modelcontextprotocol/servers>

engine), and clear success-evaluation logic; skills are stored in a shared registry that the agent queries on demand rather than relying on long monolithic prompts. The open-source **LLMQuant Skills** project² provides a representative instantiation of this paradigm for quantitative finance, organizing its content into eighteen domain-specific categories spanning equities, ETFs, options, futures, foreign exchange, and cryptocurrency markets, together with cross-cutting categories for factor mining, factor combination, risk control, portfolio management, and trading execution. Beyond dedicated skill registries, the broader open-source quantitative ecosystem provides a rich pool of reusable components that serve as the substrate for skill-based architectures: the **awesome-quant** curated list³ catalogues hundreds of quantitative libraries, backtesting engines, and data tools across multiple languages; Microsoft’s **Qlib** platform⁴ provides a modular AI-oriented quant pipeline with standardized data, model, and backtest interfaces; and **FinRL**⁵ offers a unified reinforcement-learning-based trading framework with pre-built environments and agents. These mature projects, while not originally designed as LLM skills, provide the algorithmic and infrastructural building blocks that skill-based systems wrap and compose. Each Skill follows a standardized folder layout consisting of a **SKILL.md** manifest, a **workflows/** directory of executable procedures, optional **scripts/** for tool invocations, and **assets/** for reference data. This modular structure allows a research agent to chain, for example, a factor profiling workflow with an orthogonalization workflow and a regime-aware backtesting workflow without re-engineering the underlying prompts. By externalizing research procedures into a discoverable skill library, Skill-based architectures also enable community contribution: practitioners can author, version, and share domain-specific skills through plugin marketplaces compatible with major agent runtimes, turning isolated in-house pipelines into reusable public infrastructure.

(3) Harnesses. The MCP and Skill layers described above are exercised through a Harness, the runtime environment that hosts the LLM, mediates tool calls, and manages agent state over long horizons. Frontier agentic coding systems such as Claude Code, Codex, Cursor, Antigravity, OpenClaw, DeepSeek Harness⁶, and Open Interpreter⁷ have emerged as the de facto harnesses for sustained research workflows, each exposing persistent task primitives that allow users to specify an objective once and let the agent decompose, execute, and iterate over multiple days. Two such primitives are particularly relevant for quantitative research: goal-driven execution commands (in the style of `/goal`) that anchor a long-running investigation to a single high-level specification, and scheduled looping commands (in the style of `/loop`) that periodically re-execute a workflow against fresh market data. By maintaining its own task state, the harness decouples the researcher’s high-level intent from the mechanics of tool invocation, allowing the same MCP server and skill library to be exercised across different harnesses without modification. In effect, the harness is the integration point at which the protocol layer (MCP), the workflow layer (Skills), and the orchestration layer (Single/Multi-agent, discussed next) are bound together into a runnable system.

(4) Single- and Multi-agent Systems. On top of a chosen harness, agentic quantitative systems are organized along a spectrum from single-agent to multi-agent topologies. Single-agent configurations deploy one LLM instance that iteratively reasons, calls tools, and updates its context; this design minimizes inter-agent communication overhead and is well suited to tightly coupled tasks such as factor expression search or single-asset trading. Early autonomous single-agent systems such as AutoGPT⁸ pioneered the goal-driven, self-directed execution paradigm, in which an agent decomposes a high-level objective into subtasks and iterates until completion. More recently, general-purpose agent engineering frameworks such as LangChain⁹ have become the de facto foundation for building both single- and multi-agent systems, providing standardized abstractions for chains, tools, memory, and agent orchestration. Multi-agent configurations partition the pipeline across specialized roles—for example, a Data Agent that curates market observations, a Researcher Agent that proposes candidate factors, a Critic Agent that stress-tests assumptions, and an Executor Agent that places trades—coordinated through structured message passing or shared memory. Several mature

²<https://github.com/LLMQuant/skills>

³<https://github.com/wilsonfreitas/awesome-quant>

⁴<https://github.com/microsoft/qlib>

⁵<https://github.com/AI4Finance-Foundation/FinRL>

⁶<https://github.com/deepseek-ai/deepseek-harness>

⁷<https://github.com/KillianLucas/open-interpreter>

⁸<https://github.com/Significant-Gravitas/AutoGPT>

⁹<https://github.com/langchain-ai/langchain>

open-source frameworks have standardized this multi-agent orchestration pattern: LangGraph¹⁰ extends LangChain with stateful graph-based execution for long-running multi-agent workflows; Microsoft AutoGen¹¹ provides a multi-agent programming framework with conversational coordination and code-executing agents; and CrewAI¹² specializes in role-playing multi-agent teams where each agent assumes a defined persona and responsibility. The trade-off is explicit: multi-agent designs increase modular specialization and parallel exploration at the cost of coordination overhead and potential role inconsistency. This organizational choice is orthogonal to the MCP, Skill, and Harness layers: any agent, regardless of its role, can call any MCP server or invoke any Skill, which is what makes the layered architecture composable in practice.

(5) Continuous Learning. The final dimension concerns how an agentic system accumulates experience across research cycles. A persistent limitation of one-shot agent runs is that lessons learned from a failed factor candidate or a regime-shift event are not carried forward to subsequent investigations. Memory-augmented architectures address this by maintaining an external knowledge store that the agent reads from and writes to during each cycle. Representative systems such as Yu et al. (2026a) and Liu et al. (2026) maintain structured memory of past factor attempts, failed strategy hypotheses, and regime-conditioned performance statistics, allowing the agent to retrieve analogous prior experience when encountering a similar market context. Beyond passive retrieval, recent frameworks also incorporate reflective mechanisms that periodically distill running experience into higher-level research heuristics, effectively closing the loop between online execution and offline knowledge consolidation. This dimension is particularly relevant for the unattended, multi-day research loops introduced by modern harnesses: without continuous learning, a long-running agent repeatedly rediscovers the same dead ends; with it, the system progressively improves its effective sample efficiency. Continuous learning thus connects the four preceding dimensions: MCP supplies the raw observations, Skills encode the operational procedures, Harnesses provide the persistence substrate, and Multi-agent topologies determine which agent population the memory is shared across, into a self-improving research organism.

(6) Autonomous Research Systems. Building on the protocol, skill, harness, agent, and memory layers above, autonomous research systems aim to take end-to-end ownership of the scientific investigation loop: formulating hypotheses, designing experiments, executing analyses, interpreting results, and iterating on findings without step-by-step human direction. This rapidly maturing area extends the agentic paradigm from tool-augmented assistance to fully self-directed research, with representative systems spanning several complementary approaches.

Programmatic optimization frameworks such as DSPy¹³ treat prompt design and pipeline configuration as an optimizable search problem rather than a manual craft, introducing compilers and telemetry-based optimizers that automatically tune LLM pipelines against target metrics. By formalizing research workflows as programmable graphs with learnable parameters, DSPy provides the algorithmic foundation for self-improving research systems that can optimize their own reasoning pipelines over time.

Software engineering agents bring autonomous code-writing capability to research systems. OpenDevin¹⁴ provides an open-source platform for AI-driven software development, where agents autonomously navigate codebases, write and debug code, and execute complex engineering tasks within a sandboxed environment. Similarly, SWE-agent¹⁵ demonstrates that LLM agents can autonomously resolve real-world GitHub issues by combining code editing, testing, and repository navigation. For quantitative research, these general-purpose code-generation agents provide the execution substrate that turns factor hypotheses and portfolio strategies into runnable backtest pipelines.

Evolutionary AI research systems take a different approach, using evolutionary algorithms to autonomously discover novel model architectures, data representations, and research directions. The evolutionary model merging work from Sakana AI¹⁶ exemplifies this paradigm, using evolutionary optimization to combine and

¹⁰<https://github.com/langchain-ai/langgraph>

¹¹<https://github.com/microsoft/autogen>

¹²<https://github.com/crewAIInc/crewAI>

¹³<https://github.com/stanfordnlp/dspy>

¹⁴<https://github.com/OpenDevin/OpenDevin>

¹⁵<https://github.com/princeton-nlp/SWE-agent>

¹⁶<https://github.com/SakanaAI/evolutionary-model-merge>

mutate existing foundation models without retraining, discovering new capabilities that no single model possesses alone. This line of work points toward a future in which research progress is driven not by human-designed experiments alone, but by population-based search over the space of model configurations and training regimes.

Together, these directions show how the lower-level agentic components (MCP, Skills, Harnesses, multi-agent orchestration, and continuous learning) compose into systems that can conduct increasingly autonomous research. In quantitative finance, this trajectory maps directly onto the multi-factor pipeline, where autonomous research agents can take on end-to-end roles spanning factor discovery, combination, risk control, and portfolio construction.

Continuous learning thus connects the four preceding dimensions: MCP supplies the raw observations, Skills encode the operational procedures, Harnesses provide the persistence substrate, and Multi-agent topologies determine which agent population the memory is shared across, into a self-improving research organism.

3 Task 1: Factor Mining

As the first stage of our four-step multi-factor investing framework, Factor Mining is where raw market information is first transformed into predictive signals. Using the consistent notation established in Section 1.1, its objective is to construct the mapping $f(\cdot; \theta_f)$ from observable market states X_t to future return realizations Y_{t+1} , producing the factor matrix F_t that feeds into the Factor Combination stage. Section 3.1 formalizes this task and previews the dichotomy that organizes the remainder of the chapter.

3.1 Task Definition and Mathematical Formulation

Task definition. *Factor mining* is the process of turning raw market observations into compact predictive signals, called *factors*, that can guide trading decisions. A factor is, in plain terms, a numerical feature computed from price, volume, fundamental, or textual data whose cross-sectional ranking at time t carries information about the future return of each asset at time $t + 1$. For example, the ratio of today’s close to its 20-day moving average is a classic price-volume factor, while the polarity of an earnings-call transcript is a textual factor. The goal of factor mining is to discover as many such predictive features as possible, ideally ones that are mutually uncorrelated so that they can later be combined into a robust composite signal. The methods used to mine factors have evolved along a clear trajectory: early work manually hand-crafted formula expressions from domain knowledge; GP then automated this by evolving expression trees through mutation and crossover; RL reformulated expression construction as a sequential decision problem with explicit reward signals; and more recently, LLMs and generative neural networks such as GFlowNets have been used to inject semantic reasoning and structural diversity into the search process. Regardless of the method, the output is the same, namely a factor matrix F_t whose columns are the mined factors.

Formally, at each cross-sectional time step t , the factor mining module observes a heterogeneous information set $X_t = [X_t^{PV}, X_t^{Alt}]$ consisting of:

- **Price-Volume Data** ($X_t^{PV} \in \mathbb{R}^{N \times M}$): Standard numerical market inputs, including historical price-volume series (OHLCV) and technical indicators.
- **Alternative Data** (X_t^{Alt}): All non-price-volume inputs, including textual data (financial reports, macroeconomic announcements, earnings call transcripts, news articles) as well as fundamental ratios, knowledge graphs, and on-chain data.

The task is to learn a mapping $f(\cdot; \theta_f)$ from X_t to a factor matrix $F_t \in \mathbb{R}^{N \times K}$ whose columns serve as predictors of future returns Y_{t+1} , where K is the number of mined factors. This mapping is shared by all four methodological families surveyed below; they differ only in how θ_f is parameterized and optimized.

Mathematical formulation. The factor mining problem is typically formulated as an empirical risk minimization task:

$$\arg \min_{\theta_f} \mathbb{E} [\|f(X_t; \theta_f) - Y_{t+1}\|^2] \quad (6)$$

To reduce overfitting and encourage structural sparsity or generalization, regularized objectives are often used:

$$f^* = \arg \min_{f \in \mathcal{F}} \mathcal{L}(f(X_t), Y_{t+1}) + \lambda \mathcal{R}(f) \quad (7)$$

where $\mathcal{R}(f)$ is a regularization functional and λ controls the trade-off between fit quality and model complexity.

Classification of methods. Beyond the shared objective above, surveyed methods differ most fundamentally in the form of their output: some emit explicit, interpretable output in the form of either symbolic factor expressions or executable factor code (formulaic factor/code generation), while others emit direct trading signals without symbolic forms (signal generation). We therefore organize the remainder of this chapter along this dichotomy: Section 3.3 reviews signal generation by the nature of the emitted signal. Figure 2 visualizes this taxonomy chronologically: rows correspond to method paradigms and columns to technological eras, so that the migration from symbolic search and early reinforcement learning toward LLM-based agents can be traced directly along the timeline.

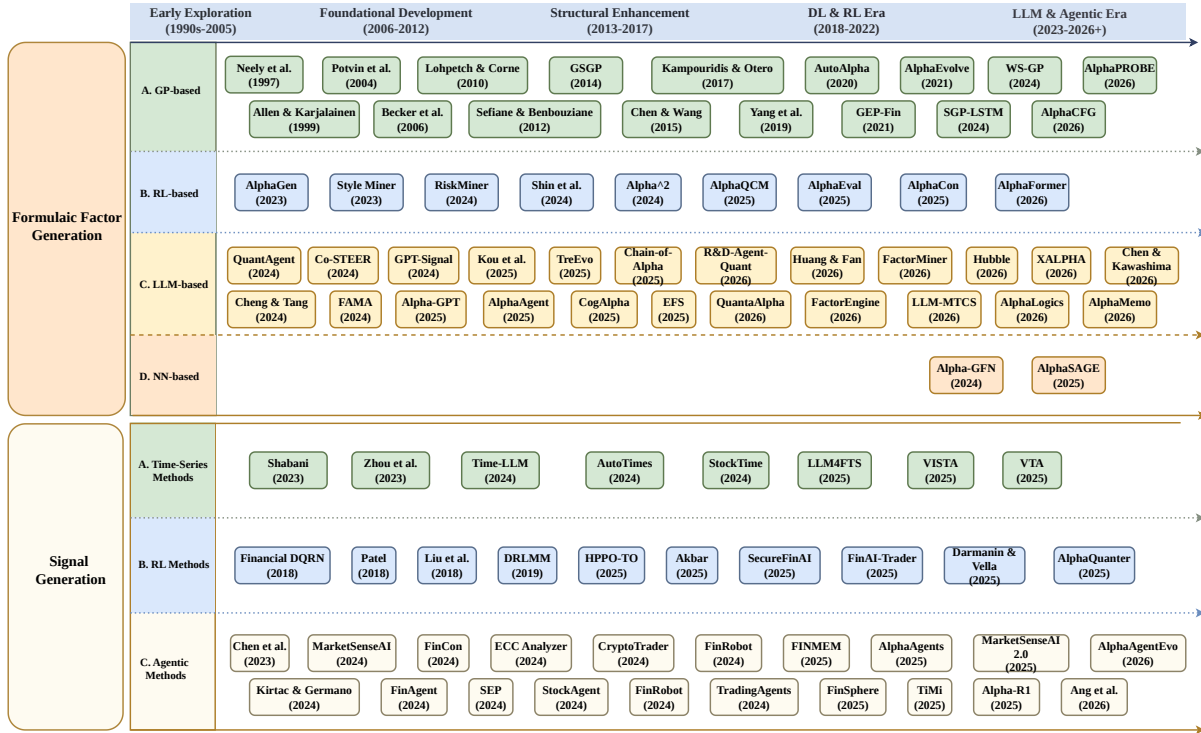


Figure 2: **Methods in Quantitative Finance: Factor Mining.** This branch illustrates how different paradigms, including Genetic Programming, Reinforcement Learning, and Large Language Models, are used to explore the non-convex space of predictive factor construction.

3.2 Formulaic Factor/Code Generation

Methods in this category output explicit, inspectable predictive rules, taking the form of either symbolic factor expressions or executable code (Python snippets, DSL programs, etc.). Automated discovery methods can be organized along an evolutionary trajectory of increasing search sophistication; rather than replacing earlier techniques, each new paradigm expands the accessible hypothesis space while building on prior methods. From an evolutionary-computation standpoint, Yu et al. (2026b) unify these paradigms within a single framework, characterizing automated formulaic alpha discovery as a noisy, dynamic, multi-objective symbolic search problem and proposing a six-component analysis covering representation, variation, fitness evaluation, selection, memory, and adaptation.

Genetic Programming. Early approaches formulate factor discovery as symbolic regression over a pre-defined operator set. Candidate expressions are generated through mutation and crossover. While these methods provide interpretable analytical forms, the search process is largely heuristic and computationally inefficient in high-noise environments.

Reinforcement Learning. To improve search efficiency, factor construction is reformulated as a sequential decision problem. A policy model is trained to select operations or transformations based on historical rewards, such as Information Coefficient (IC) or predictive accuracy. This reduces random exploration but still depends heavily on the design of the reward function and state representation.

LLM-based Methods. More recent approaches incorporate large language models to assist in feature design and transformation. These models are typically used to propose candidate factor expressions or transformation rules conditioned on textual descriptions or domain knowledge. In this setting, alternative data (X_t^{Alt}) can be incorporated more systematically, complementing traditional price-volume feature engineering.

Neural Network-based Methods. Most recently, generative flow networks reframe expression search as sampling from a reward-proportional distribution, explicitly optimizing structural diversity rather than a single optimum, thereby mitigating the mode-collapse tendency of reward-maximizing RL.

Overall, these paradigms differ primarily in their search strategy and information utilization mechanism. In the following subsections, we review representative methods, their empirical performance, and their respective limitations.

3.2.1 GP-based Factor Mining

GP provides a symbolic framework for alpha discovery by representing candidate models as expression trees. In this setting, the parameterization θ_f corresponds to a population of tree-structured functions, typically represented as Abstract Syntax Tree (AST), enabling explicit mathematical interpretability of generated factors. As shown in Figure 3, this search proceeds as a constrained evolutionary loop: feature inputs supply the raw material of expressions, grammar rules and functional constraints delimit the syntactically valid space, and a population of Abstract Syntax Trees (ASTs) is recursively refined through crossover and mutation, evaluation and refinement, and selection under an explicit search strategy, with surviving candidates exported to an external alpha pool. The four evaluation axes at the bottom of the figure, namely factor diversity, inefficiencies, syntactic validity, and numerical quality, guard against degenerate expressions dominating the population.

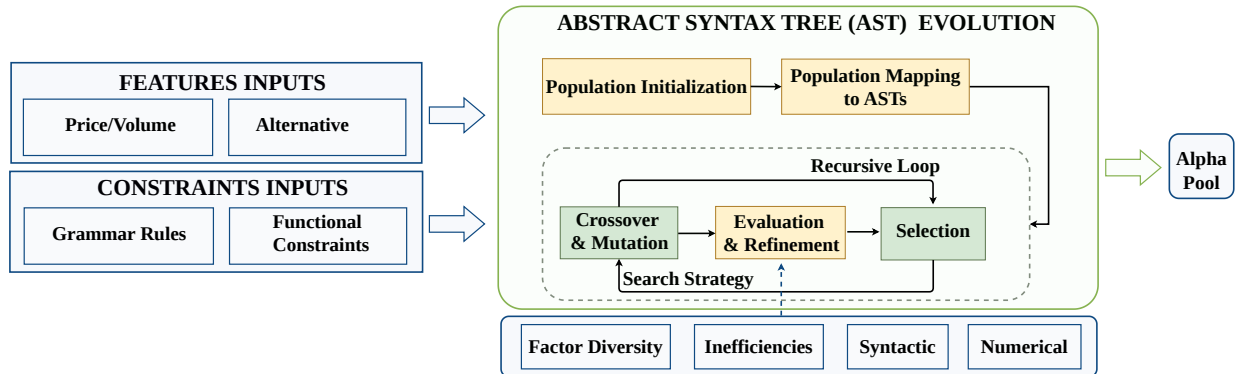


Figure 3: **Architecture of GP-based Factor Mining.** This framework illustrates the evolutionary symbolic search process over an expression tree space. Candidate solutions are represented as ASTs, and structural constraints such as grammar rules are imposed to ensure syntactic validity of generated factor expressions.

Foundational Framework. The GP-based factor mining framework can be decomposed into four canonical steps that any concrete instantiation must implement. **(1) Representation:** candidate factors are encoded as ASTs over a predefined operator set (arithmetic, rolling statistics, cross-sectional ranks, etc.) and a terminal set (price/volume features, constants). **(2) Initialization:** an initial population is generated either randomly or seeded from known factor templates, with grammar rules imposed to ensure syntactic validity. **(3) Evolutionary Search:** the population is iteratively refined through biologically inspired operators, namely selection (favoring high-fitness candidates), crossover (swapping subtrees between parents), and mutation (random perturbation of nodes or subtrees), with fitness evaluated against a historical objective such as Information Coefficient or risk-adjusted return. **(4) Selection and Export:** surviving candidates after a fixed number of generations are exported to an external alpha pool, subject to diversity and redundancy checks. This four-step template, canonically instantiated by open-source implementations such as `gplearn`¹⁷, is the shared scaffold that all subsequent GP-based methods refine, whether by enriching the representation (e.g., grammar constraints, graph structures), by improving the search operators (e.g., semantic-aware crossover, warm-start initialization), or by replacing the fitness evaluation (e.g., RL-guided or LLM-guided selection). The open-source `gplearn` library provides the canonical software implementation of this template and is the most commonly used baseline in subsequent work.

The following sections organize GP-based factor mining methods along four lines of improvement. First, we review common baseline models that establish the empirical behavior of the canonical framework. Second, we examine work on syntactic efficiency, which improves search stability and reduces code bloat by restructuring the representation space. Third, we cover guided search, which introduces structured exploration mechanisms to move GP from blind random search toward directed, goal-oriented exploration. Finally, we survey other extensions including matrix-valued operators, hybrid architectures, and adaptive retraining mechanisms.

Common Baselines. A large body of empirical work has used the canonical GP framework above as a baseline against which to measure proposed improvements. Allen and Karjalainen (1999) applied GP to S&P 500 data and showed that, while GP can identify weak predictive patterns, its performance is sensitive to transaction costs and tends to be comparable to simple benchmark strategies in highly efficient markets. Subsequent studies highlighted that performance depends strongly on asset class and market structure: Neely et al. (1997) found that GP achieves stronger out-of-sample performance in foreign exchange markets, where structural inefficiencies and macroeconomic interventions are more pronounced; in equity markets, Potvin et al. (2004) showed that incorporating short-selling constraints improves robustness under bearish and volatile regimes; and Lohpetch and Corne (2010) demonstrated that reducing data frequency and enforcing strict validation protocols can improve generalization performance. From a portfolio construction perspective, Becker et al. (2006) showed that alternative fitness specifications can capture nonlinear interactions that are not well represented in linear factor models. These works collectively establish the empirical behavior of the canonical GP baseline across asset classes and fitness specifications, and are routinely used as comparison points by the syntactic-efficiency, guided-search, and hybrid methods reviewed below.

Syntactic Efficiency. This line of work addresses the structural inefficiency of standard GP search, where candidate trees bloat uncontrollably and small syntactic changes produce huge fitness swings, making the search unstable and hard to direct. In plain terms, it fixes the problem that GP trees grow out of control and similar-looking formulas can produce wildly different results. Despite its interpretability advantages, standard GP suffers from well-known structural issues, including code bloat and unstable search dynamics. To address syntactic inefficiency, Chen et al. (2021) proposed Gene Expression Programming (GEP), which separates genotype and phenotype representations to enforce syntactic validity during evolution.

From a different perspective, McDermott et al. (2014) (GSGP) reformulated the search process in semantic space, smoothing the fitness landscape by operating on functional outputs rather than symbolic structure. This reduces sensitivity to local syntactic perturbations and improves search stability.

To further constrain the hypothesis space, grammar-constrained approaches such as Yang et al. (2026) (AlphaCFG) introduce explicit context-free rules to restrict factor construction. These constraints reduce invalid expressions and improve interpretability, but also reduce exploration flexibility. In addition, tree-search-based

¹⁷<https://github.com/trevorstephens/gplearn>

implementations often rely on Monte Carlo Tree Search (MCTS) combined with structured encoders to manage combinatorial growth. However, such approaches still face redundancy issues due to equivalent symbolic representations. Event-driven reformulations, such as directional-change sampling Kampouridis and Otero (2017), further illustrate that alternative market representations can significantly alter the structure of the search space.

Guided Search. This line of work moves GP from blind random search toward directed, goal-oriented exploration: instead of letting mutation and crossover wander freely, it injects structure into where and how the search proceeds, so that the algorithm spends less time exploring dead ends and more time on promising regions of the formula space. To improve search efficiency, later work introduced structured exploration mechanisms. Zhang et al. (2020) proposed a hierarchical decomposition strategy, where simple base factors are first constructed and then recursively combined into more complex expressions. This reduces the effective depth of the search space and improves stability.

Warm-start strategies (WS-GP) were introduced by Ren et al. (2024b), where the initial population is seeded with empirically validated factors rather than random expressions. This improves convergence speed but may introduce prior bias toward known factor structures.

More recently, graph-based representations have been used to model factor evolution. Guo et al. (2026b) (AlphaPROBE) represent the factor pool as a directed graph, where nodes correspond to candidate factors and edges represent derivation relationships. This formulation allows reuse of intermediate structures and supports more efficient exploration of the search space by avoiding redundant recomputation.

At the operator level, crossover mechanisms have also been refined. Sefiane and Benbouziane (2012) show that arithmetic crossover provides smoother interpolation in continuous parameter spaces compared to discrete splicing operators, improving stability in evolutionary optimization.

Others. This last group collects GP extensions that do not fit neatly into syntactic efficiency or guided search, covering richer data types, hybrid architectures with downstream models, and mechanisms for adapting to non-stationary markets. To better handle complex data structures, GP has been extended to support matrix and vector-valued operations. Cui et al. (2021) introduced specialized operators for cross-sectional ranking and temporal aggregation, enabling more expressive factor representations.

In hybrid architectures, GP is often used as a feature construction layer prior to downstream statistical or neural models. Li et al. (2024a) demonstrated that symbolic feature generation can improve robustness in neural forecasting models, particularly under regime shifts.

Finally, to address non-stationarity, several studies integrate evolutionary search with adaptive retraining mechanisms. Chen et al. (2010) and Chen and Hirasawa (2011) incorporate sliding-window updates to maintain relevance under distributional drift. In graph-based extensions such as Genetic Network Programming, Chen and Wang (2015) introduced structured state transitions for dynamic allocation under risk constraints. Co-evolutionary frameworks such as Yang et al. (2019) further extend this idea by combining multiple evolving sub-populations under regime-switching environments.

Overall, GP-based methods provide a flexible symbolic search framework for factor construction. However, their reliance on stochastic exploration and local fitness evaluation limits scalability in high-dimensional and highly non-stationary financial environments. This motivates more structured and policy-driven search paradigms discussed in subsequent sections.

3.2.2 RL-based Factor Mining

Classical GP suffers from undirected random mutation with no explicit optimization objective. RL addresses this limitation by formulating factor mining as a structured, objective-driven Markov Decision Process (MDP). The mapping function $f(\cdot)$ is replaced by an agent’s policy network $\pi(\mathbf{a}_t | \mathbf{S}_t; \theta_f)$, and the agent learns to search for factors that maximize long-term, risk-adjusted rewards. The MDP is defined by:

- **State S_t** : market features aggregating price/volume, microstructure, and macro/news/fundamental channels;
- **Action a_t** : formula-construction steps that assemble operator tokens into executable expressions (distinct from RL signal agents in Section 3.3, whose actions are trading signals or weights directly);
- **Reward r_t** : factor-level metrics such as Information Coefficient, marginal synergy to an existing factor pool, or risk-adjusted return;
- **Policy $\pi(a_t | S_t; \theta_f)$** : maps states to formula-construction actions, trained to maximize $\mathbb{E}[\sum_t \gamma^t r_t]$.

As illustrated in Figure 4, the agent observes a state S_t aggregating price/volume, microstructure, and macro/news/fundamental channels, and acts through a three-fold action space: discrete trading primitives (buy, sell, hold), continuous portfolio weights $[w_1, w_2, \dots, w_N]^T$, and formula actions that assemble operator tokens into executable expressions. An execution module then applies order execution, inventory updates, and transaction costs, and rewards evaluated on PNL, risk metrics, and other objectives flow back to update the policy.

Within this output category, RL research focuses on learning policies that construct explicit factor expressions, progressing from reward shaping and variance reduction to syntactic priors and LLM-guided semantic search. RL agents trained to emit trading signals or portfolio weights directly, as well as RL-trained LLM agents that optimize multi-agent reasoning loops, are instead classified under Signal Generation (Section 3.3).

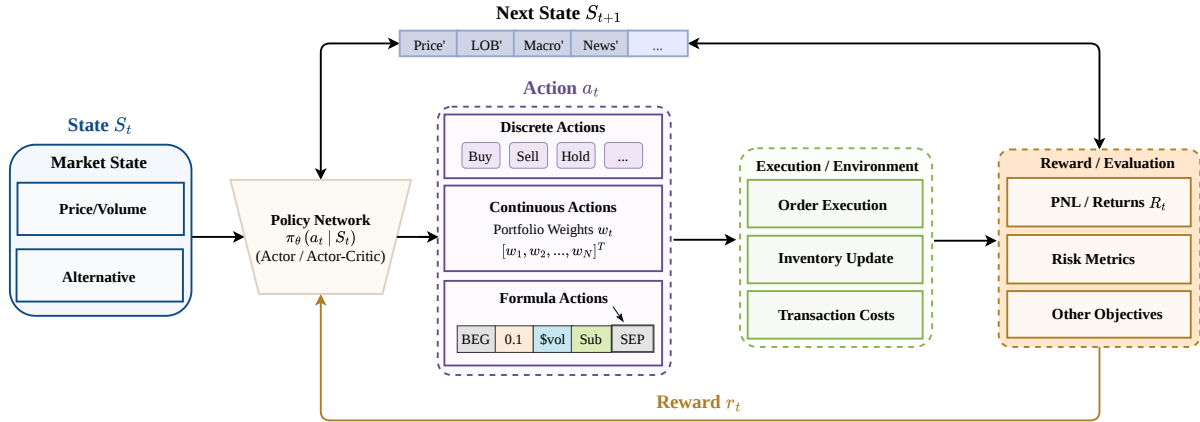


Figure 4: **Architecture of RL-based Factor Mining.** This setup frames factor mining as an MDP. The agent uses a policy network π to navigate the formula space through goal-directed exploration, updating its strategy based on delayed financial rewards to construct interpretable mathematical expressions.

Foundational Frameworks. The RL agent functions as an automated factor designer, selecting mathematical operators to construct explicit functional expressions. The foundational advance in this direction was Yu et al. (2023) (AlphaGen), which reframed factor construction as an MDP whose reward measured the marginal synergy a new factor contributes to an existing portfolio, rather than evaluating factors based solely on isolated IC scores. AlphaGen established the standard RL formulation: state as market features, action as formula-construction tokens, and reward as factor-level IC or synergy. Subsequent work extends this template along three directions. First, reward design refines what the agent is optimized for so that the learned factors are more stable and more useful downstream. Second, search efficiency reduces the computational cost of building factors from scratch. Third, self-evolving agents close the loop between RL exploration and LLM reasoning so that the system can improve its own discovery strategy over time.

Reward Design. This line of work improves what the RL agent is actually rewarded for: instead of a simple IC score, it designs rewards that encourage stable, diversified, or risk-aware factors, so that the agent does not chase noisy high-IC formulas that fall apart out of sample. Building on AlphaGen’s formulation, a substantial line of work refines the reward function to address high variance, distributional uncertainty, and tail-risk awareness. Shin et al. (2024) expanded the expressiveness of the framework by incorporating highly non-linear operators, including cross-sectional rankings, time delays, and conditional functions, with a warm-start initialization strategy seeding the agent’s pool with empirically validated factors. To address the high variance of standard architectures, Zhao et al. (2025a) demonstrated that in deterministic formula generation settings, critic networks introduce unnecessary estimation bias; switching to a pure policy gradient with a greedy baseline and strict variance regularization produced substantially more stable factors. From a different methodological perspective, Li et al. (2023a) proposed a constrained RL framework combining a Variational Autoencoder and Actor-Critic architecture to explicitly balance a factor’s explanatory power against its temporal stability. To characterize complete outcome distributions rather than expected returns alone, Zhu and Zhu (2025) employed Distributional RL, pairing DQN with Implicit Quantile Networks to explicitly model uncertainty. Conversely, Ren et al. (2024a) argued that focusing exclusively on average performance neglects tail events with high economic significance, developing a risk-seeking Monte Carlo Tree Search (MCTS) that updates its policy using only the top τ -quantile trajectory rewards. To inject domain priors and accelerate convergence, Zhao et al. (2025b) provided the agent with dense intermediate rewards based on prefix structures matching human-designed factors, while Jiang et al. (2026b) constructed self-supervised rewards from the model’s token-level uncertainty, directing search gradients toward the most difficult reasoning nodes.

Search Efficiency. This line of work makes RL factor mining faster and cheaper: instead of generating factors token by token from a cold start, it introduces code-generation framing, cached intermediate results, and LLM-guided navigation to cut redundant computation. A complementary direction reduces the computational cost of searching from scratch. Xu et al. (2024) framed factor generation as code generation, introducing a dimension check that exponentially reduces the search space. Improving syntactic efficiency further, Yang et al. (2026) represented formulas via Abstract Syntax Representations (ASR), combining Tree-LSTM encodings with grammar-aware MCTS to avoid exploration of grammatically redundant paths. To eliminate the need for full retraining across datasets, Huang et al. proposed an end-to-end Transformer architecture that conditions on both current market state and the existing factor pool, learning cross-dataset priors to generate complementary factors without complete retraining. For large asset universes, Xu et al. (2025) proposed a two-level Hierarchical RL framework in which a top-level option policy assigns attention weights to specific stocks while a low-level policy constructs intraday factors. Ding et al. (2025) developed a method for mapping candidate signals to orthogonal evaluation spaces, enabling holistic and parallel assessment of factor mining systems. A complementary direction replaces purely token-level RL exploration with LLM-guided semantic search: Sun et al. (AlphaCon) framed factor generation as an In-Context Adaptation problem with a Proposer-Refiner architecture, allowing the agent to navigate the formulaic search space with genuine semantic understanding and adapt rapidly to recent market data.

Self-Evolving Agents. This line of work upgrades the RL agent from a passive formula builder to a self-improving system: it combines RL’s reward-driven optimization with LLMs’ semantic reasoning, so the agent can read news, reflect on its own mistakes, and change how it searches without human intervention. The most recent major development is self-evolving agentic RL, which addresses traditional RL’s inability to natively process alternative (textual) semantic data by optimizing the analyze-reflect-plan reasoning loops of LLMs. To navigate the formulaic search space with genuine semantic understanding, Sun et al. framed factor generation as an In-Context Adaptation problem, employing a Proposer-Refiner architecture to adapt rapidly to recent market data. Jiang et al. (2025) constructed a Historical Memory database to convert quantitative data into interpretable market context, applying Group Relative Policy Optimization (GRPO) so that the model reasons through these semantic states to dynamically activate or deactivate factors. This integration of RL’s reward-driven optimization and LLM’s open-ended reasoning culminates in Tang et al. (2026), which moves beyond local formula adjustments to long-term evolutionary planning, using GRPO to optimize a multi-agent loop and calculating the advantage of entire evolutionary trajectories rather than individual states. A complementary instance of parameter-level RL fine-tuning is Zhang et al.

(2026) (QuantEvolver), which replaces prompt-level generation-evaluation-feedback loops with reinforcement fine-tuning: a Miner LLM converts executable Factor-DSL backtests into GRPO policy updates, rewarded by a Diversity-Complementarity objective, so that optimization experience is internalized in model parameters rather than accumulated in the prompt.

3.2.3 LLM-based Factor Mining

In modern quantitative systems, the traditional factor mining operator $f(\cdot; \theta_f)$ encounters massive dimensionality problems when processing alternative data $\mathbf{X}_t^{Alt}$. LLMs address this by functioning as high-capacity nonlinear encoders that directly process semantic data. For textual data, this extraction is mathematically defined as a deep semantic mapping:

$$\mathbf{H}_t = \text{LLM}(\mathbf{X}_t^{Alt}; \theta_{LLM}) \quad (8)$$

where $\mathbf{H}_t$ is a semantic embedding that can either be decoded into an explicit factor expression (Category I, this section) or feed directly into a signal predictor (Category II, Section 3.3). As shown in Figure 5, the framework branches into two pipelines: closed-loop formulaic generation, where a discovery cycle $C_k : H_k \rightarrow F_k \rightarrow M_k \rightarrow D_k$ is supported by an experiential memory library and a known factor library, with the LLM acting either as a self-evolving reasoner or as a heuristic guide alongside traditional algorithms; and direct signal generation, where regression stacks, agentic decision networks, and fine-tuned FinLLMs emit predictions without explicit symbolic forms (Section 3.3).

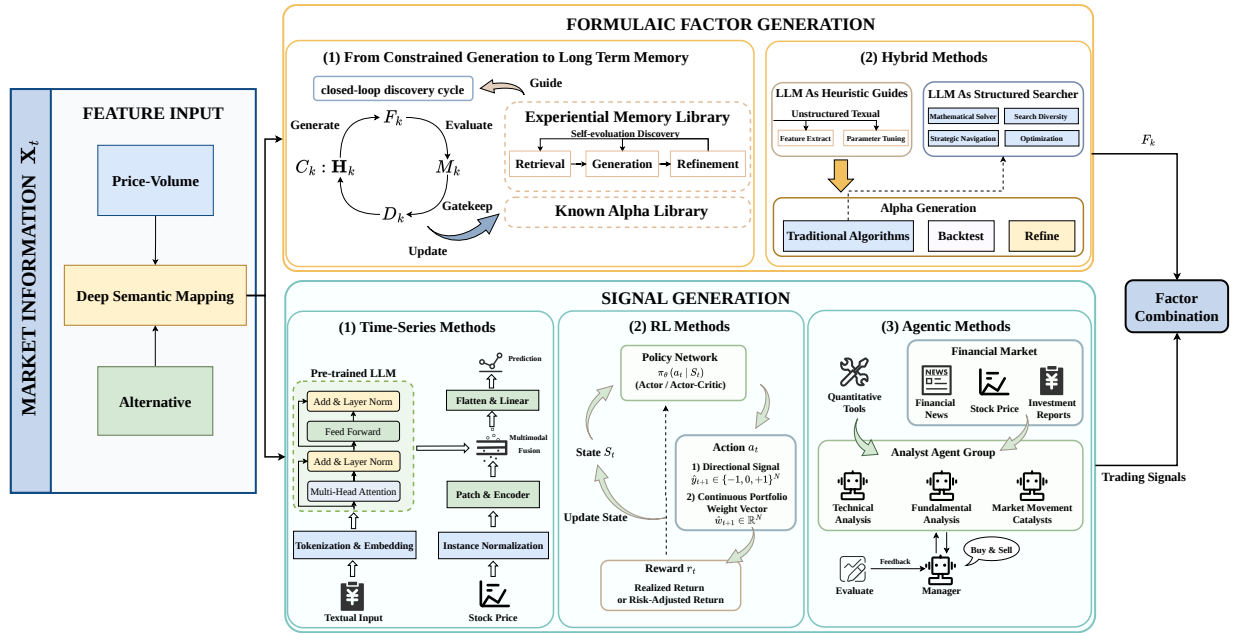


Figure 5: **Architecture of LLM-based Factor Mining.** This framework illustrates the shift toward semantic search spaces. It highlights how LLMs act as deep semantic encoders to process alternative data ($\mathbf{X}_t^{Alt}$), driving closed-loop formulaic generation through Agentic RAG and structured memory.

Within formulaic and code-based generation, LLM-based methods span a spectrum of output forms, from compact mathematical expressions to full Python or DSL implementations, and fall into two strands: pure LLM-driven pipelines that iteratively refine candidates through prompting, feedback, and memory; and hybrid pipelines that couple LLMs with traditional search algorithms. We review them in turn.

Foundational Frameworks. The LLM-based factor mining framework is the closed-loop hypothesis-to-code pipeline, in which an LLM iteratively proposes candidate factor expressions as executable code, evaluates them against historical data, and refines its proposals based on structured feedback. The framework comprises three components: (1) **Prompt-driven Generation**, where the LLM is conditioned on a

hypothesis (a natural-language description of an expected predictive pattern) and emits a Python or DSL formula implementing it; **(2) Memory-augmented Refinement**, where an external knowledge base records successful templates, failed operator combinations, and factor-correlation statistics, allowing the LLM to retrieve relevant context before the next proposal; and **(3) Iterative Evaluation**, where each candidate is backtested and its performance is fed back to the LLM as evidence for the next refinement cycle. The earliest instantiation of this template used static prompting: Cheng and Tang (2024) employed simple prompting strategies to generate factors based on internal LLM knowledge, establishing the basic hypothesis-to-code pipeline, but static prompting frequently produces invalid code and lacks a systematic optimization path. This motivated the autonomous feedback loop instantiated in its clearest form by Wang et al. (2024b) (QuantAgent), whose dual-loop architecture pairs an inner loop that generates and tests factors with an outer loop that updates the knowledge base from test outcomes, establishing the canonical reference against which subsequent LLM-based factor mining methods are compared.

The following sections organize LLM-based factor mining methods along three lines of extension. First, we review constrained generation, which introduces structural bounds on what the LLM can produce so that generated code is valid, semantically sensible, and non-redundant. Second, we cover reasoning and memory, which structures the LLM’s thought process and accumulates discovery experience across cycles. Third, we survey hybrid methods that combine LLM semantic search with GP or RL for better mathematical search efficiency.

Constrained Generation. This line of work adds guardrails to raw LLM factor generation so the LLM does not produce invalid code, hallucinated factors, or formulas that are just restatements of existing ones. In plain terms, it tells the LLM what it is allowed to write and checks its output against rules. It tackles the operational and semantic risks of raw LLM code generation by introducing structural constraints that bound what the LLM is allowed to produce. The foundational extension in this direction is Yang et al. (2024b) (Co-STEER), which generalized QuantAgent’s single-agent dual-loop into a multi-agent collaborative framework in which specialized agents continuously update a shared internal knowledge base, providing the multi-agent substrate that subsequent constraint-based methods build upon. Allowing LLMs to write raw Python code poses operational risks such as infinite loops and frequently fails to produce diverse architectures; Shi et al. (2026a) (Hubble) addressed this by constraining the LLM with a strict Domain-Specific Language (DSL) and a multi-stage validation sandbox, restricting outputs to pre-approved financial operators to ensure syntactic correctness and operational safety. Wang et al. (2024d) (GPT-Signal) used a two-step prompting strategy to push the LLM toward high-order, non-linear feature interactions beyond basic linear combinations. To convert abstract financial concepts into executable expressions, Weng et al. (2026) (AlphaLogics) introduced a Logic-to-Financial-Constraints (L2FC) agent that translates behavioral finance concepts into mathematical constraints. Tang et al. (2025) (AlphaAgent) deployed a three-fold constraint framework combining semantic alignment via natural-language hypotheses, AST similarity to prevent redundancy, and AST complexity limits to mitigate overfitting, demonstrating strong performance in top-50 long and bottom-5 short daily execution across CSI 500 and S&P 500. Rather than constraining syntax, Yi et al. (2026) (AlphaSchema) constrains exploration semantics: it constructs a structured space of trading semantics in which each point is a schema plan (Event, Context, Qualities, Direction, Output) specifying a factor’s intended meaning before implementation, then decouples exploration from implementation by learning a surrogate reward model over the semantic space that balances global exploration, exploitation, and local mutation. Moving from constraints to verification, Kim et al. (2026) (FaVOR) restructure factor mining around hypothesis-level evidence: a broad economic hypothesis is decomposed into independent observable conditions, each candidate factor is validated against whether it actually reflects its intended condition using distributional statistics on OHLCV data, and only validated factors are integrated into a composite signal, filtering out return-generating but economically meaningless pseudo-factors.

Reasoning and Memory. The most recent strand structures the LLM’s reasoning process and accumulates experience across discovery cycles. Building directly on the closed-loop template, two works established the canonical memory-augmented refinements: Wang et al. (2026) (FactorMiner) developed the Ralph loop (Retrieval, Generation, Evaluation, Refinement), maintaining structured experiential memory to record functional templates and failed operator combinations to address factor correlation, where newly discovered

factors heavily overlap with existing ones; and Li et al. (2024d) (FAMA) further improved evolutionary consistency via Cross-Sample Selection (CSS) and a Chain-of-Experience (CoE) that leverages past mining trajectories. Subsequent work then generalized these memory mechanisms along several directions. Ren et al. (2025) (TreEvo) modeled reasoning as evolutionary trees with semantic-level operators, while Cao (2025) (Chain-of-Alpha) constructed parallel optimization chains with linear complexity. To maintain auditability, Liu et al. (2025) (CogAlpha) implemented a 7-level hierarchical validation framework, and Han et al. (2026) (QuantaAlpha) modeled trajectory-level self-evolution using mutation and crossover mechanisms. Huang and Fan (2026) formalized this self-evolving process into a closed-loop discovery cycle:

$$C_k : H_k \xrightarrow{\text{Generate}} F_k \xrightarrow{\text{Evaluate}} M_k \xrightarrow{\text{Gatekeep}} D_k \quad (9)$$

ensuring the system continuously accumulates empirical evidence. As Kou et al. (2024) and Luo et al. (2025a) (EFS) showed, these generated factors are increasingly integrated with downstream synergistic combination modules. Li et al. (2026b) (R&D-Agent-Quant) fully automated this pipeline with a joint, end-to-end iterative optimization of both factors and predictive models, controlled by a contextual multi-armed bandit scheduler that switches between factor mining and model architecture refinement. To address coarse-grained attribution and redundant rediscovery, Yu et al. (2026a) (AlphaMemo) introduced Structured Search-Process Memory (SSPM), which tracks specific formula modifications and learns residual improvements rather than absolute performance, with an Asymmetric Process Veto to block reliably failing edits. Extending this memory-centric idea to the entire research lifecycle, Liu et al. (2026) (XALPHA) applies a Report-to-Memory Absorption procedure that distills external research into 48 Research Archetypes, with a Macro Brain assembling hypotheses, a Micro Brain implementing them as factor code with tri-alignment checks, and a Cross Brain consolidating outcomes into generation-level, cycle-level, and archetype-level cues; on CSI 300 ten-day open-to-open return prediction, XALPHA raises portfolio IR to 1.5368 versus 0.9516 for AlphaAgent, with 60 elite factors across six themes exhibiting average pairwise absolute correlation of only 0.142. Autonomous search must also budget and audit its own evaluation: Ye et al. (2026) (AgonAlpha) searches over frozen research artifacts (hypotheses, executable expressions, platform evidence, rationales) rather than formulas alone, coupling a fresh-context adversarial reviewer with re-execution and veto authority to a pending-aware parallel budget allocation and reporting a complete public evidence trail; independent deployments on WorldQuant BRAIN produced high-fitness alphas across multiple users and model backends. Extending this memory-centric view from individual factors to complete research trajectories, Zhu et al. (2026) (AlphaSeek) make the full hypothesis-to-backtest lineage the unit of evolution: research directions are mined autonomously from multi-source financial information, trajectories are mutated or crossed over while validated sub-steps are frozen, and factor-library-level portfolio backtests feed the next search round, improving out-of-sample portfolio performance on CSI 300 while retaining factor transferability to CSI 500.

Hybrid Methods. This line of work combines LLM semantic search with GP or RL’s structured mathematical search: the LLM supplies high-level ideas and domain understanding, while the symbolic or RL engine handles the precise numerical search, getting the best of both worlds. Despite improved reasoning capabilities, pure LLM architectures still face challenges regarding mathematical search efficiency. Recent research demonstrates the effectiveness of neuro-symbolic combinations (Chen and Kawashima, 2026), where LLMs act as structural guides for traditional algorithms. Wang et al. (2025) (Alpha-GPT) integrated human-derived market insights with precise mathematical expressions. To reduce factor homogeneity, Shi et al. (2026b) (LLM-MTCS) structured the mining process as a directed tree search, employing Frequent Subtree Avoidance (FSA) to direct exploration toward novel structural spaces. Guo et al. (2026b) formulated mining as a Directed Acyclic Graph (DAG) process, balancing exploration and exploitation across CSI 300 and CSI 500 datasets. Lin et al. (2026) (FactorEngine) achieved high scalability by delegating macro-level logical revisions to LLMs while assigning micro-level parameter tuning to Bayesian optimization.

3.2.4 Neural Network-based Factor Mining

While RL effectively drives policy-guided exploration, standard RL frameworks optimize strictly for maximum expected return. This often causes mode collapse, producing a highly correlated, homogeneous pool of factors. To solve this trade-off between formulaic diversity and search efficiency, researchers have turned

to Generative Flow Networks (GFlowNets) as an alternative discovery framework. GFlowNets reframe expression search as sampling from a reward-proportional distribution: given a factor expression s with reward $R(s)$, the network learns a forward policy $P_F(s'|s)$ such that the marginal probability of sampling s satisfies $P(s) \propto R(s)$. Unlike RL’s reward-maximizing objective $\max_{\theta} \mathbb{E}[R(s)]$, GFlowNet’s proportional-sampling objective $\min_{\theta} \|P_F(s) - R(s)/Z\|$ explicitly encourages structural diversity, mitigating the mode-collapse tendency of standard RL.

Foundational Frameworks. Shen (2024) introduced generative flow architectures to quantitative finance with **Alpha-GFN**. Instead of treating the sequential generation of mathematical operators as a standard Markov Decision Process, they framed it as a flow network. Because Alpha-GFN samples candidate formulas with probabilities proportional to their rewards, it naturally pushes the system to find diverse, multimodal factor structures. This directly prevents the narrow convergence issues common in PPO setups. The following section reviews methodological advances that build on this generative flow network framework to address reward sparsity, structural underrepresentation, and exploration diversity in automated factor mining.

Methodological Advances. This line of work improves the generative flow network approach to factor mining by tackling the three main bottlenecks that limit performance: sparse rewards that give the model too little feedback, structural underrepresentation that misses useful factor shapes, and lack of diversity that leads to correlated outputs. Building on this setup, Chen et al. (2025a) (AlphaSAGE) systematically tackled three major bottlenecks in automated factor mining: reward sparsity, structural underrepresentation, and exploration diversity. Traditional RL relies on delayed Information Coefficient (IC) scores, which are only calculated after the entire formula is finished. This creates severe cold start problems. AlphaSAGE fixed this by swapping out the isolated feedback for a multi-dimensional dense reward function ($R_{IC} + R_{SA} + R_{NOV}$). By adding predictive power (R_{IC}), structure-action alignment (R_{SA}), and structural novelty (R_{NOV}) to policy entropy regularization, the model constantly steers the agent through the search space. Older sequential models usually flattened factor expressions into 1D Reverse Polish Notation (RPN) sequences and fed them into LSTMs. This flat format completely misses commutative equivalencies (like $X_{open} + X_{close}$ versus $X_{close} + X_{open}$). AlphaSAGE changed the format by parsing factors into Abstract Syntax Trees (ASTs) and running them through Relational Graph Convolutional Networks (RGCN). This explicitly captures the actual semantic topology and the distinct mathematical edges of the formula. To make sure this push for diversity does not trigger infinite loops of redundant math, AlphaSAGE added a dynamic early stopping mechanism. Tied to the current AST length, it cuts off invalid formula branches before they burn through computational resources. A complementary advance improves how structural decisions are taught: Su (2026) (AlphaG-OPD) turn terminal factor evaluations into local action guidance via structural on-policy distillation, exposing grammar-valid sibling actions at partial abstract-syntax-tree states and admitting a distillation target only when matched sibling comparisons reach sufficient winner agreement and a positive lower confidence bound, with consolidated replay preventing redundant factor evaluations; the method delivers strong cross-market performance on CSI 300, CSI 500, CSI 1000, and S&P 500.

Researchers have thoroughly validated these generative network approaches using standard historical back-testing. For instance, AlphaSAGE evaluations used the standard 6-dimensional market features (Open, Close, High, Low, VWAP, Volume). The output factors showed strong out-of-sample performance across different global markets, powering daily-rebalanced strategies with a 20-day holding period. The models generated solid returns in both long-only Chinese equity portfolios (CSI 300 and CSI 500, trading the top 20% of signals) and long-short U.S. setups (S&P 500, symmetrically trading the top and bottom 10% of signals).

3.3 Signal Generation

Methods in this category bypass explicit factor expressions and emit trading signals directly: directional return predictions, sentiment distributions, or executable trading decisions. The model itself functions as the factor, and its decision logic remains implicit in network weights or natural-language reasoning traces. Formally, signal generation replaces the factor matrix F_t with a direct mapping $g(\cdot; \theta_g) : X_t \rightarrow \hat{y}_{t+1}$, where $\hat{y}_{t+1}$ can be a regression-style return forecast $\hat{r}_{t+1} \in \mathbb{R}^N$, a directional signal $\hat{y}_{t+1} \in \{-1, 0, +1\}^N$, or a

portfolio weight vector $\hat{w}_{t+1} \in \mathbb{R}^N$. We organize these methods by the nature of the emitted signal into three families:

- **Time-Series Methods** (Section 3.3.1): regression-style return forecasts from sequential models;
- **RL Methods** (Section 3.3.2): RL-trained execution policies that directly output trading signals or portfolio weights;
- **Agentic Methods** (Section 3.3.3): LLM-driven multi-agent frameworks that synthesize multimodal signals into trading decisions.

3.3.1 Time-Series Methods

Time-series methods formulate signal generation as a sequence forecasting problem. Given a window of historical price-volume data $X_{t-W:t}^{PV} \in \mathbb{R}^{W \times N \times M}$ over window size W , the model learns a mapping $g(\cdot; \theta_g)$ that directly predicts future returns $\hat{r}_{t+1} = g(X_{t-W:t}^{PV}; \theta_g) \in \mathbb{R}^N$ without emitting an explicit factor expression. The objective is:

$$\arg \min_{\theta_g} \mathbb{E} [\mathcal{L}(g(X_{t-W:t}^{PV}; \theta_g), Y_{t+1})] \quad (10)$$

Unlike formulaic generation (Section 3.2), the model g itself functions as the factor, and its decision logic remains implicit in network weights. The historical foundation of this template is the recurrent neural network family, where LSTMs and GRUs incrementally update a hidden state over the input window and project it to a return forecast, with subsequent work extending this pattern via temporal convolutions and Transformer encoders. The LLM era reshaped this template by replacing the learned-from-scratch encoder with a frozen pre-trained language model backbone, repurposing the semantic priors learned from text pre-training for numeric forecasting.

Foundational Framework. The time-series signal generation framework treats signal generation as a sequence forecasting problem in which a window of historical price-volume data is mapped to a future return prediction through a learned encoder. The classical foundation of this framework is the recurrent neural network template, where a hidden state is updated incrementally over the input window and projected to a return forecast through a linear head, capturing temporal dependencies without explicit feature engineering. This recurrent template was extended along several directions, including temporal convolutions, attention-augmented encoders, and Transformer architectures, with Shabani et al. (2023) exemplifying the pre-LLM line by augmenting recurrent architectures with auxiliary weight matrices for temporal feature adaptation. The foundation model era reshaped this template by replacing the learned-from-scratch encoder with a frozen pre-trained language model, giving rise to the LLM-backbone reprogramming template that dominates recent work. The LLM-backbone template comprises three components: **(1) Numeric-to-Token Projection**, where raw price-volume sequences are mapped into the LLM’s token embedding space through a learned projection layer; **(2) Frozen LLM Backbone**, where the pre-trained transformer parameters are kept fixed (or only lightly adapted via parameter-efficient methods), preserving the semantic priors learned from text pre-training; and **(3) Forecast Head**, where the LLM’s hidden states are projected back to a numeric return forecast. This template is appealing because it inherits the temporal reasoning capabilities of text pre-trained models without requiring domain-specific financial training data. Zhou et al. (2023) (FPT) instantiated this template in its clearest form by freezing a GPT-style backbone and learning a linear numeric-to-token projection, establishing the canonical reference against which subsequent LLM-based time-series signal methods are compared. The following section reviews recent advances that refine this reprogramming recipe along instruction extraction, modal fusion, and reasoning alignment.

Recent Advances. This line of work builds on the FPT foundation and improves it in three ways: it extracts better instructions from the data, it fuses multiple data modalities beyond just prices, and it aligns the model’s verbal reasoning with actual numerical accuracy. Subsequent work refines FPT’s reprogramming recipe along the axes of instruction extraction, modal fusion, and reasoning alignment. Jin et al. (2024) (Time-LLM) extended FPT’s projection recipe with prompt-based instruction extraction, establishing the

standard prompt-prefix formulation that subsequent methods adopt. Liu et al. (2024c) (AutoTimes) improved automatic extraction of temporal instructions from the LLM, while Wang et al. (2024c) (StockTime) and Jia et al. (2025) (LLM4FTS) built unified forecasting ensembles directly from price sequences. Moving past text-only inputs, Khezresmaeilzadeh et al. (2025) (VISTA) introduced a multimodal method combining numeric sequences with visual K-line charts. To align verbal reasoning with numerical accuracy, Koa et al. (2025) (VTA) optimized reasoning trajectories using an inverse MSE reward tied to market behavior.

3.3.2 RL Methods

RL-based signal agents formulate factor mining as a MDP where the action space is the trading signal or portfolio weight directly, rather than a formula-construction step. The MDP is defined by:

- **State $\mathbf{S}_t$:** market features at time t (price/volume, microstructure, sentiment);
- **Action $\mathbf{a}_t \in \mathcal{A}$:** either a directional signal $\hat{y}_{t+1} \in \{-1, 0, +1\}^N$ or a continuous portfolio weight vector $\hat{w}_{t+1} \in \mathbb{R}^N$ (contrast with RL-based formulaic generation in Section 3.2.2, where the action is a formula-construction step);
- **Reward r_t :** realized return or risk-adjusted return of the executed action;
- **Policy $\pi(\mathbf{a}_t | \mathbf{S}_t; \theta_\pi)$:** trained to maximize $\mathbb{E}[\sum_t \gamma^t r_t]$.

The agent learns θ_π end-to-end against portfolio-level rewards, bypassing the intermediate factor representation entirely.

Foundational Frameworks. Early RL signal agents established the basic state-action-reward formulation under non-stationary market conditions. Huang (2018) addressed market non-stationarity in foreign exchange markets by substantially reducing replay memory to 480 steps to ensure that the model focuses exclusively on recent market regimes, and by employing action augmentation. For continuous equity portfolios, Liu et al. (2018) established the standard Actor-Critic formulation that directly maps market states to asset allocation weights. These two works define the foundational template of replay-memory management for non-stationarity and Actor-Critic for continuous portfolios that subsequent RL signal methods adopt and extend.

The following sections organize RL-based signal generation along two directions of extension. First, we review market microstructure work, which injects fine-grained order-book and execution details into the agent so it can operate at higher frequencies. Second, we cover hybrid RL-LLM methods, which combine RL’s execution strength with LLM’s cognitive capabilities for richer observation spaces.

Market Microstructure. This line of work makes RL signal agents aware of the detailed mechanics of how markets actually operate, such as limit order books, bid-ask spreads, and execution costs, so the agent can make smarter decisions at high frequencies. As high-frequency trading became increasingly competitive, models required deeper micro-structural awareness. For cryptocurrency market making, Sadighian (2019) developed the DRLMM framework to learn liquidity provision end-to-end, processing a comprehensive 143-dimensional Environmental State Space (ESS) encompassing 15-level limit order book data, Order Flow Imbalances (OFI), and Trade Flow Imbalances (TFI) across 100 lags, with an Agent State Space (ASS) tracking inventory ratios, fill rates, and profit and loss, executing 17 distinct quotation-related actions including an emergency flattening function for extreme price swings. Since individual agents struggle to generalize across diverse market conditions, Patel (2018) divided responsibilities between a Macro-agent for directional trades and a Micro-agent for tactical limit order book execution. He et al. (2025) extended this division of labor to equity markets with specialized agents (Macro, Fundamental, Technical, News) coordinated by an RL Portfolio Agent screener. To enable end-to-end operation without black-box limitations, Deng and Wang (2025) developed a tool-orchestrated agentic RL framework enabling a single agent to dynamically learn coherent strategies directly from market feedback.

Hybrid RL-LLM. This line of work pairs RL’s execution engine with LLMs’ reading and reasoning abilities: the LLM processes news and text to give the RL agent richer market context, while the RL agent handles the actual trading decisions, combining the strengths of both. The current frontier combines RL execution with the cognitive capabilities of LLMs. Akbar (2025) encoded limit order book data, Alpha101 factors, and LLM sentiment into a group of RL agents (DQN, PPO, SAC) with weighted ensemble voting. Grzesiak and Aggarwal (2025) (SecureFinAI) integrated LLM sentiment with high-frequency data via a Feature Aggregation Transformer and Ensemble DQN voting to enhance robustness during crypto market volatility. Ahmed et al. (2025) (FinAI-Trader) employed PPO variants to rapidly adapt to shifting sentiment, establishing the hybrid RL-LLM execution framework. To address the myopia of pure RL trading policies and the unreliability of direct LLM execution with minimal computational overhead, Darmanin and Vella (2025) proposed a modular neuro-symbolic setup operating on a dual timeline (monthly strategy and daily execution), running a multi-stage prompt pipeline ($P_0 \rightarrow P_4$) with expert gating, extracting high-level insights from GPT-4o Mini condensed into a single entropy-weighted directional signal $\tau = \text{dir}(\pi^g) \cdot \text{str}(\pi^g)$ appended to the RL agent’s observation space, preserving the algorithmic stability of Double DQN while incorporating explicit economic causality.

3.3.3 Agentic Methods

Agentic methods model signal generation as a multi-agent decision process in which multiple specialized LLM-based agents collaborate to produce a trading decision. Formally, an agentic framework consists of:

- a set of agents $\{A_1, \dots, A_K\}$, each with a specialized role (analyst, risk manager, trader, etc.);
- a communication graph $\mathcal{G}$ defining information flow between agents;
- a meta-agent or scheduler that routes tasks and aggregates evidence;
- a memory module $\mathcal{M}$ storing historical context and past decisions.

The system maps multimodal inputs $X_t = [X_t^{PV}, X_t^{Alt}]$ to a final trading decision $\hat{y}_{t+1}$ through the multi-agent interaction $\hat{y}_{t+1} = \text{Meta}(\{A_k(X_t, \mathcal{M})\}_{k=1}^K; \theta_{\text{agent}})$. Unlike single-model approaches, the decision logic is distributed across agents and explicit in natural-language reasoning traces.

Foundational Framework. The agentic signal generation framework is the role-specialized multi-agent template, in which a set of LLM-based agents, each assigned a distinct role such as analyst, risk manager, or trader, collaborate through a communication graph coordinated by a meta-agent, with a shared memory module storing historical context. The framework comprises four components: **(1) Role-Specialized Agents**, where each agent is prompted to behave as a specific financial persona (fundamental analyst, technical analyst, news sentiment reader, etc.); **(2) Communication Graph**, defining which agents exchange messages and in what order; **(3) Meta-Agent or Scheduler**, which routes tasks, aggregates evidence, and produces the final trading decision; and **(4) Memory Module**, storing historical market context and past decisions to enable temporal consistency. Li et al. (2023b) (TradingGPT) instantiated this template in its clearest form by assigning distinct risk personas to multiple agents and using collaborative reasoning to produce trading decisions, establishing the canonical reference against which subsequent agentic methods are compared.

The following sections organize agentic signal generation along three lines of extension. First, we review multi-agent architectures, which enrich how agents interact and share information. Second, we cover specialization and hierarchy, which split the work across specialized sub-agents to handle heterogeneous data. Third, we survey autonomous self-evolution, where the agentic framework can modify and improve itself over time.

Multi-Agent Architectures. This line of work builds on the basic multi-agent template and adds more sophisticated ways for agents to work together, such as multimodal memory, deeper specialization, and better coordination mechanisms, so that the group as a whole makes better trading decisions. A first line of extension enriches the agent interaction structure, building directly on TradingGPT’s role-specialized template by introducing multimodal memory and deeper agent specialization. Yu et al. (2025b) (FINMEM) and

Zhang et al. (2024b) (FinAgent) introduced multimodal memory-augmented agent networks that synthesize global market views from heterogeneous inputs, refining the memory module and modality coverage of the original TradingGPT template. Fatouros et al. (2024) (MarketSenseAI) and its successor Fatouros et al. (2025) (MarketSenseAI 2.0) combined structured pricing with unstructured news to output causal insights. Chen et al. (2023b) used LLMs as zero-shot financial engineers to dynamically infer company relationships from daily headlines, building dynamic connectivity graphs for Graph Neural Networks (GNNs) instead of relying on static industry knowledge bases, uncovering leading-lagging effects robust in forecasting downside risks. Kirtac and Germano (2024) systematically tested LLMs (like OPT and FinBERT) against traditional lexicons, linking extracted sentiments directly to cumulative excess returns via simulated long/short portfolios with realistic transaction costs. Moving beyond binary polarity, Wang and Wei (2025) proposed Event-Aware Sentiment Factors, using Gemini-2.5-Pro for zero-shot multi-label classification to assign over 70 fine-grained event tags (such as retail hype or rumors) to tweets, and showed that specific narrative tags act as independent alpha sources with distinct return profiles. To extract deep fundamental signals rather than short-term sentiment, Gupta (2023) (GPT-InvestAR) deployed a RAG-based framework using ChromaDB to answer 27 strategic questions from over 24,000 corporate 10-K filings, converting qualitative LLM confidence scores into quantitative factors that consistently outperform the S&P 500.

Specialization and Hierarchy. This line of work organizes agents into a hierarchy where different agents handle different types of data or different levels of decision-making: lower-level agents specialize in specific tasks like reading earnings calls, while higher-level managers coordinate them and make the final call. A second line of extension manages the computational load of single agents processing heterogeneous data through hierarchical and specialized designs. Yu et al. (2024) (FinCon) and Koa et al. (2024b) (SEP) used hierarchical Manager-Analyst architectures. For detailed fundamental analysis, Cao et al. (2024) (ECC Analyzer) and Han et al. (2025) (FinSphere) focused on deep causal extraction specifically from earnings calls. Testing across asset classes, Li et al. (2024c) (CryptoTrade) and Zhang et al. (2024a) (StockAgent) validated agentic adaptability in different market microstructures.

Autonomous Self-Evolution. This line of work gives the agent system the ability to change and improve itself: instead of relying on humans to update its prompts or add new agents, the system can reflect on its own performance and modify its own structure, roles, and strategies autonomously. The most recent strand makes the agentic framework itself self-evolving. Xiao et al. (2024) (TradingAgents) and Zhao et al. (2025c) (AlphaAgents) explicitly modeled real-world organizational structures with bounded debates to stop information distortion over long contexts. Yang et al. (2024a) (FinRobot) created a transparent, four-layer architecture driven by Financial Chain-of-Thought (CoT), forcing agents to separate reasoning into explicitly observable steps and using a scheduler to dynamically route tasks to specialized fine-tuned models (like ChatGLM for Chinese markets, FinGPT for specific analyses), validated across indices like the DOW 30 and SSE 50. To address the high latency of continuous reasoning in fast-moving markets, Song et al. (2025) (TiMi) introduced a decoupled **Trade in Minutes** architecture, moving semantic strategy formulation and feedback reflection to an offline LLM layer while deploying specialized, parameter-driven trading bots online, combining strategic depth with millisecond-level mechanical execution. A complementary line drives self-evolution through RL-based optimization of LLM reasoning: Jiang et al. (2025) constructed a Historical Memory database converting quantitative data into interpretable market context, applying Group Relative Policy Optimization (GRPO) so that the LLM reasons through these semantic states to dynamically activate or deactivate factors, and Tang et al. (2026) extended this RL-LLM integration to long-term evolutionary planning, using GRPO to optimize a multi-agent loop and calculating the advantage of entire evolutionary trajectories rather than individual states. Combining all hierarchical, memory, and reflection mechanisms, Ang et al. (2026) establishes an autonomous asset management system governed by an Investment Policy Statement, organizing over 50 specialized agents, including Macro, Asset Class, and Adversarial Diversifier agents, to conduct structural deliberation via a Borda count voting protocol, with a Meta-agent capable of global self-evolution that autonomously modifies the natural language instructions or underlying Python code of specific sub-agents when discrepancies between realized market returns and historical forecasts are detected. At the level of the research process itself, Guo et al. (2026a) (AQuA) run two independent LLM-driven research systems, one for symbolic factor discovery and one for trainable model development, each closing its own research loop by retaining validated evidence in a sealed sandbox and using it to guide

subsequent proposals; the factor system reaches a combined information coefficient of about 0.190 on a crypto universe, while the model system converts a per-stock information coefficient of +0.0843 on US equities into a threshold long-short strategy with a held-out Sharpe ratio up to +2.50, positive in every year from 2021 to 2025. Similarly closing the loop at the strategy level, Mao et al. (2026) (EVOQUANT) frame strategy optimization as self-evolving, verifier-guided refinement: an LLM diagnoses the performance bottlenecks of an existing strategy, generates semantically controlled candidate edits, selects the best candidate through a multi-stage verification pipeline, and distills optimization experience into reusable knowledge, raising the average test Sharpe ratio from -0.298 to 0.538 across A-share and cryptocurrency strategies.

4 Task 2: Factor Combination

As the second stage of our Four-Step Multi-Factor Investing Framework, Factor Combination implements the aggregation operator $\mathcal{T}(\cdot; \theta_T)$ introduced in Section 1.1, transforming the raw factor matrix F_t into a unified signal $S_{raw,t}$. The signals emerging from Factor Mining are typically high-dimensional, heterogeneous, and highly correlated. Factor Combination addresses three fundamental challenges: multicollinearity among factors, amplification of estimation errors during aggregation, and the modeling of nonlinear cross-factor interactions. Section 4.1 formalizes this task and previews the four paradigms that organize the remainder of the chapter. The output of this stage then feeds directly into the Risk Control stage for orthogonalization and adjustment.

4.1 Task Definition and Mathematical Formulation

Task definition. *Factor combination* is the process of merging multiple mined factors into a single composite signal that is more reliable than any individual factor. The need for this stage arises from three properties of mined factors: they are typically correlated with each other (multicollinearity), they each carry estimation noise that can be amplified when naively aggregated, and they exhibit nonlinear interactions (e.g., a momentum factor may only be predictive when volatility is low). In plain terms, factor combination takes a matrix of K candidate factors and produces one vector of composite scores, one per asset, that a downstream portfolio optimizer can directly consume. The methods used to perform this aggregation have evolved along a clear trajectory: classical methods apply fixed or statistically estimated linear weights (equal weighting, IC weighting, risk parity); supervised methods learn nonlinear aggregations from labeled future returns (tree ensembles, recurrent networks, autoencoders); RL methods treat dynamic re-weighting as a sequential decision problem under market feedback; and LLM-enhanced methods close the loop between factor generation and combination by letting LLM-driven pipelines participate in the weighting decision.

Formally, building on the factor matrix $F_t \in \mathbb{R}^{N \times K}$ produced by the upstream Factor Mining stage (Section 3), the Factor Combination module learns an aggregation operator $\mathcal{T}(\cdot; \theta_T)$ that maps K heterogeneous predictive signals into a single composite signal $S_{raw,t} \in \mathbb{R}^N$:

$$S_{raw,t} = \mathcal{T}(F_t; \theta_T) \quad (11)$$

where θ_T denotes the internal parameters or structural setup of the operator. The composite signal $S_{raw,t}$ then feeds into the Risk Control stage (Section 5) for orthogonalization and adjustment.

Mathematical formulation. The standard objective is to maximize the out-of-sample Information Ratio or minimize the prediction error of the composite signal, while keeping the system dynamically adaptable to shifting market regimes. As shown in Figure 6, the combination operator $\mathcal{T}(\cdot; \theta_T)$ is structured as three parallel computational layers, namely a Linear Combination Layer, a Nonlinear Transformation Layer, and an LLM Enhanced Layer, that together implement the four paradigms of increasing expressiveness: classical linear weighting, supervised nonlinear aggregation, reinforcement-learning-based dynamic routing, and LLM-enhanced closed-loop optimization.

Classification of methods. Based on how $\mathcal{T}(\cdot)$ aggregates information, we group combination methods into four paradigms of increasing expressiveness. **Classical methods** (Section 4.2) restrict $\mathcal{T}$ to linear statistics. **Supervised methods** (Section 4.3) parameterize $\mathcal{T}$ as a regression model learned from labeled returns. **RL methods** (Section 4.4) frame dynamic weighting as a sequential decision problem under market feedback.

LLM-enhanced methods (Section 4.5) close the loop between generation and combination, letting LLM-driven pipelines participate in the weighting decision. Figure 7 visualizes this taxonomy: rows correspond to method paradigms and columns to technological eras, so that the migration from classical statistics toward supervised learning, RL, and LLM-based methods can be traced directly along the timeline.

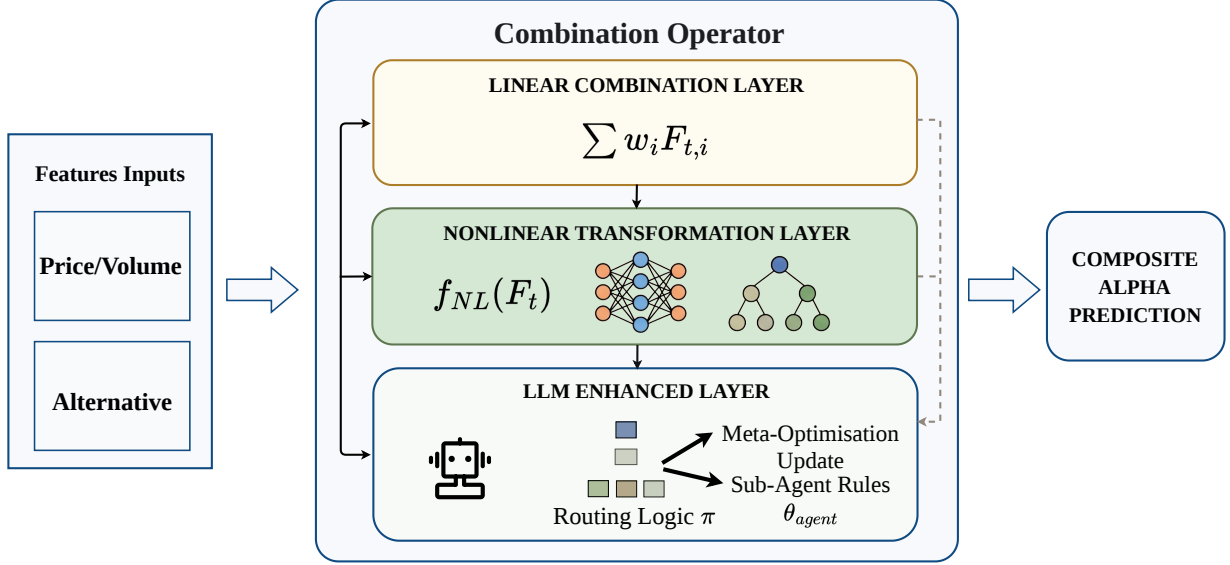


Figure 6: **Architecture of the Factor Combination Operator $\mathcal{T}(\cdot; \theta_T)$.** The raw multimodal factor matrix $F_t \in \mathbb{R}^{N \times K}$ (containing Price-Volume and alternative data) is processed through three parallel layers: a **Linear Combination Layer** computing the weighted sum $\sum w_i F_{t,i}$ with activation constraints, a **Nonlinear Transformation Layer** applying the learned mapping $f_{NL}(F_t)$ via neural networks and tree-based architectures, and an **LLM Enhanced Layer** with routing logic π and agent parameters θ_{agent} refined through meta-optimisation and sub-agent rule updates. A feedback loop from the final output back to the LLM layer closes the loop between combination performance and factor generation strategy. The three outputs fuse into the final composite alpha prediction.

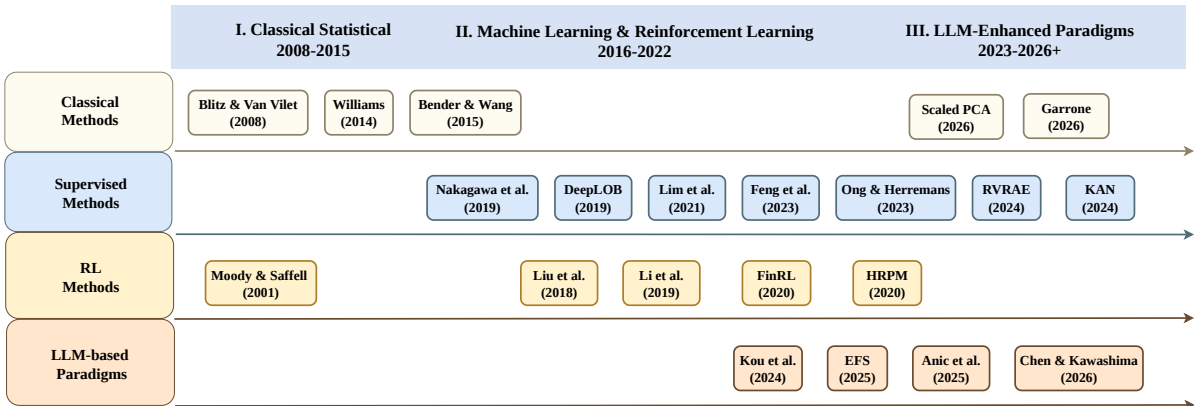


Figure 7: **AI Methods in Quantitative Finance: Factor Combination.** This timeline summarizes factor combination methods across four paradigms of increasing expressiveness: classical linear weighting, supervised nonlinear aggregation, reinforcement-learning-based dynamic routing, and LLM-enhanced closed-loop optimization, ordered chronologically from traditional statistical approaches to recent LLM-driven paradigms.

4.2 Classical Methods

In traditional financial econometrics, $\mathcal{T}(\cdot)$ is strictly linear, with θ_T simplifying into a basic weight vector $w_t \in \mathbb{R}^K$:

$$S_{raw,t} = F_t w_t \quad (12)$$

The challenge is to choose w_t so as to maximize out-of-sample IR while controlling for multicollinearity and estimation error.

Foundational Framework. The canonical classical factor combination framework is *linear aggregation under statistical constraints*, in which the composite signal is a weighted sum of the K input factors, with weights chosen to maximize out-of-sample Information Ratio while controlling for multicollinearity and estimation error. The framework comprises three decision points: **(1) Weighting Scheme**, specifying how weights are computed, whether equal weighting, IC weighting, inverse-variance weighting, or risk-budget weighting; **(2) Aggregation Granularity**, specifying whether factors are combined top-down at the asset-class level or bottom-up at the individual security level; and **(3) Risk Adjustment**, specifying whether and how the weights are conditioned on dynamic volatility or cross-factor correlations. The simplest instantiation, equal weighting, serves as the canonical baseline against which all subsequent classical methods are compared; the most influential instantiation, risk parity, equalizes the risk contribution of each factor rather than its capital allocation, and underlies most production multi-asset balanced mandates.

The following sections organize classical factor combination methods along two lines. First, we review common baselines that establish the empirical behavior of the classical linear aggregation framework. Second, we cover high-dimensional scaling, which addresses the estimation-error problem when the number of factors grows large.

Common Baselines. This section collects the most widely used classical combination methods that serve as baselines for all subsequent work, including equal weighting, risk parity, and other standard statistical weighting schemes. A large body of empirical work has refined the classical framework along its three decision points. Blitz and Van Vliet (2008) addressed cross-asset fragmentation in Global Tactical Asset Allocation by introducing a unified deterministic comparison combining momentum and value factors scaled across 12 different asset classes. To address the vulnerability of standard risk parity to duration risk during macro shifts, Williams (2014) moved to Factor Risk Parity, using dynamic conditional volatility scaling to balance factors directly rather than relying on static asset class correlations. Bender and Wang (2015) proved that a bottom-up approach, aggregating standardized Z-scores directly at the underlying security level, drastically outperforms top-down combinations because it preserves the original data distributions. These three works collectively establish the empirical behavior of the classical linear aggregation framework across asset classes and weighting schemes, and are routinely used as comparison points by the high-dimensional scaling methods reviewed below.

High-Dimensional Scaling. This line of work addresses the problem that classical methods break down when you have many factors: with too many factors and not enough data, the estimation error explodes and the optimized weights become garbage, so this work introduces regularization, shrinkage, and other techniques to keep things stable. When scaling up to high-dimensional factor sets, traditional Mean-Variance Optimization basically maximizes estimation errors. To solve this dimensionality problem, Jiang et al. (2026a) (Scaled PCA) built a framework that pre-scales components using their Information Coefficients before applying ridge regression shrinkage, directly linking the first moment (expected returns) with the second moment (covariance). In actual trading environments, to avoid dealing with unstable covariance matrices entirely, systems like Garrone (2026) use deterministic rebalancing and dynamic asset eligibility, enforcing hard structural bounds to strictly control turnover and system vulnerability. In portfolio replication, Arratia and Gzyl (2026) (EFM) reformulate the ill-posed inverse problem with an entropy-minimization principle: parameters and replication weights are estimated within data-driven empirical bounds using a Fermi-Dirac-type entropy defined directly on the constraints, which acts as a probabilistic circuit breaker that defensively reduces allocation to compromised assets under stress, outperforming ordinary least squares replication in annualized turnover and net-of-fees returns.

4.3 Supervised Methods

Supervised methods parameterize $\mathcal{T}$ as a regression model $\mathcal{T}_\phi$ learned by minimizing the prediction error against the realized return Y_{t+1} :

$$\phi^* = \arg \min_{\phi} \mathbb{E} [\|\mathcal{T}_\phi(F_t) - Y_{t+1}\|^2] \quad (13)$$

Compared to classical linear statistics, supervised methods capture high-order factor interactions and align the return prediction with the downstream weight optimization, mitigating the two-stage loss problem where the return prediction and the weight optimization are misaligned. Following the conventional order of model development, we review tree ensembles, recurrent networks, convolutional networks, and autoencoders in turn.

Foundational Frameworks. The supervised learning paradigm for factor combination trains a regression model to predict future returns from the input factor matrix, using the prediction error as the training objective. This family of methods captures non-linear factor interactions that linear methods cannot, and includes tree-based models, recurrent networks, convolutional networks, and autoencoders. Tree models represent the first major departure from linear combination, providing native support for nonlinear interactions and feature importance ranking. Breiman (2001) introduced Random Forests, which bootstrap-aggregate hundreds of decision trees to reduce variance while capturing high-order factor interactions. For factor combination, Random Forests naturally rank factors by their Gini importance or permutation importance, allowing the system to automatically downweight redundant or low-information factors. In empirical tests on global equity markets, Random Forest factor combiners typically achieve 15-25% higher out-of-sample Information Ratios compared to OLS weighting, with the greatest improvements occurring during volatile market regimes.

Tree Methods. This line of work uses tree-based ensembles like Random Forests and Gradient Boosting to combine factors: each tree learns a set of decision rules that split the factor space, and the ensemble captures non-linear factor interactions that linear models miss. Gradient Boosting Decision Trees (GBDT) further advanced this paradigm by iteratively fitting trees to the residuals of previous predictions. Chen and Guestrin (2016) (XGBoost) introduced regularization-aware boosting with built-in cross-validation, becoming the industry standard for factor combination in many quantitative hedge funds. XGBoost’s key innovation for factor combination is its ability to model hierarchical factor interactions: a momentum factor’s predictive power may depend on the volatility regime, and a value factor’s effectiveness may vary with liquidity conditions. Compared to Random Forests, XGBoost typically delivers an additional 10-15% improvement in out-of-sample performance due to its more aggressive bias-variance tradeoff management. Ke et al. (2017) (LightGBM) extended GBDT with Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB), enabling efficient processing of high-dimensional factor spaces with thousands of candidate features. For institutional factor combination systems managing 500+ factors, LightGBM reduces training time by 70-90% while maintaining comparable accuracy. More recently, Zhou and Feng (2019) introduced Deep Forests, replacing traditional tree ensembles with multi-grained scanning and cascade structures. Deep Forests achieve neural network-level performance on factor combination tasks while maintaining full interpretability, making them particularly valuable for regulatory-compliant quantitative systems.

Recurrent Networks. This line of work uses recurrent neural networks to capture the time dimension in factor combination: the model remembers how factors behaved in the recent past and adjusts its weights accordingly, which is especially useful when market regimes are shifting. Recurrent Neural Networks (RNNs) and their Long Short-Term Memory (LSTM) variants address the temporal dimension of factor combination explicitly. Unlike static methods that apply fixed combination weights across time, recurrent architectures model the sequential dependence of factor efficacy, learning how factor performance autocorrelates and decays over time. Nakagawa et al. (2019) (Deep Recurrent Factor Model) introduced an interpretable non-linear and time-varying multi-factor model using recurrent networks to capture the time-varying nature of factor loadings. This is particularly important because factor efficacy is highly non-stationary: momentum factors may be predictive during trending markets but fail during reversals, and value factors may require multiple

quarters to manifest their predictive power. The key innovation of recurrent factor models is their ability to track factor performance trajectories rather than relying on long-term averages. By maintaining hidden states that encode recent factor performance, RNN-based combiners can dynamically adjust weights based on the recent trajectory of factor efficacy. This temporal awareness provides natural protection against regime shifts, as the model can rapidly downweight factors whose predictive power is deteriorating. In practice, recurrent factor combiners are particularly valuable for medium-horizon prediction tasks, where the autocorrelation structure of factor returns is most informative.

Convolutional Networks. This line of work treats the factor matrix like an image, with stocks as one dimension and factors as the other, and uses convolutional neural networks to learn local spatial patterns across both. This is useful for capturing how groups of related stocks respond to factor signals together. Convolutional Neural Networks (CNNs) introduced a fundamentally different approach to factor combination by treating the factor matrix as a 2D spatial tensor. Zhang et al. (2019) (DeepLOB) was among the first deep CNN architectures for limit order book data, demonstrating that convolutional filters can extract hierarchical temporal patterns that linear methods miss. For factor combination specifically, CNN-based architectures use 1D convolutional filters along the time dimension to capture temporal dynamics in factor efficacy, and 2D filters to capture cross-factor interaction patterns. In practice, CNN factor combiners excel at short-horizon prediction tasks, where they typically outperform tree models due to their superior temporal pattern recognition. To capture non-linear market pricing, Feng et al. (2023) constructed a long-short deep factor from high-dimensional features, functioning simultaneously as a hedge against benchmark risk and an enhancement signal, directly estimating tangent portfolio weights to maximize the Sharpe ratio. For temporally fragmented signals, Ong and Herremans used a Multi-Gate Mixture of Experts via Multi-Task Learning to dynamically map multi-timeframe momentum inputs directly to portfolio allocations. Lim et al. (2021) introduced Temporal Fusion Transformers, which combine recurrent layers for local pattern recognition with attention mechanisms for global context. For multi-horizon factor combination, this hybrid architecture achieves state-of-the-art performance while providing interpretable feature importance scores. On the CSI 300 and S&P 500 datasets, Temporal Fusion Transformers consistently deliver higher out-of-sample Sharpe ratios compared to standalone CNN or LSTM architectures.

Autoencoders. This line of work uses autoencoders to deal with the problem that factors are often highly correlated with each other: the model compresses the factor matrix into a smaller set of uncorrelated latent factors, removing redundancy and making the combined signal more stable. Autoencoders address the multicollinearity problem in factor combination through unsupervised dimensionality reduction. By training an encoder-decoder architecture to reconstruct the original factor matrix through a low-dimensional bottleneck, autoencoders automatically extract orthogonal latent factors that capture the maximum predictive variance. Unlike PCA-based dimensionality reduction, autoencoders can capture non-linear dependencies among factors, providing richer latent representations. Wang and Guo (2024) (RVRAE) introduced a dynamic factor model based on variational recurrent autoencoder, modeling latent factors as probability distributions rather than point estimates. This probabilistic formulation naturally provides uncertainty estimates for the combined signal, allowing downstream portfolio optimization to account for combination confidence. The variational formulation also prevents overfitting to noise in factor returns, which is particularly important given the low signal-to-noise ratio typical in financial data. More recently, Wang and Singh (2024) applied Kolmogorov-Arnold Networks (KAN) to autoencoder factor models, leveraging KAN’s superior function approximation capabilities for non-linear factor extraction. KAN-based autoencoders demonstrate better performance on highly non-linear factor interactions, though at the cost of increased computational complexity. A complementary line generates diversity at the output level: Den et al. (2026) (MAPLE) emit multiple alpha components from a single capacity-scaled prediction head over a shared representation, using an extreme-rank weighted listwise ranking loss together with a diversity regularizer that explicitly penalizes pairwise correlation across alphas; the unified head already reduces inter-alpha correlation before the diversity loss is applied, and across US, Chinese, and Japanese equity markets MAPLE attains the best average Sharpe and Calmar ratios among nine baselines with up to $55\times$ fewer parameters and $2.5\times$ less training time. Rather than decorrelating outputs, Chehab et al. (2026) (MINGLE) redefine the correlation structure itself: a mutually-informed Graph-Locality and Exposures framework jointly learns a latent factor representation and an induced graph topology from market returns via a unified ADMM formulation, so that graph locality

is defined by systematic factor-exposure profiles rather than observed co-movement; the resulting exposure-similarity graph aligns more closely with economic sectors than correlation-based graphs, and portfolios built on it outperform correlation-based counterparts across volatility regimes and transaction-cost levels.

4.4 RL Methods

To address the static nature of standard Deep Learning in shifting market regimes, researchers reframed dynamic combination as a Markov Decision Process (MDP). The state S_t encodes the recent factor matrix and market regime, the action $a_t = w_t$ selects combination weights, and the reward r_t reflects the realized portfolio return or IR. The policy $\pi_\phi(a_t | S_t)$ is trained to maximize the expected discounted reward:

$$\phi^* = \arg \max_{\phi} \mathbb{E} \left[\sum_t \gamma^t r_t \mid \pi_{\phi} \right] \quad (14)$$

This formulation aligns the combination objective with downstream portfolio performance, mitigating the two-stage loss problem.

Foundational Frameworks. The intellectual origin of this approach traces to Moody and Saffell (2001), who first demonstrated that direct reinforcement learning from investment returns outperforms the traditional predict-then-optimize two-stage approach. Moody’s key insight was that minimizing prediction error (MSE) does not necessarily maximize investment performance, establishing the theoretical motivation for end-to-end learning in portfolio management.

The following sections organize RL-based factor combination along two directions. First, we review end-to-end weighting approaches that directly optimize investment objectives rather than prediction error. Second, we cover infrastructure and extensions that make RL-based combination more practical and widely applicable.

End-to-End Weighting. This line of work gets rid of the gap between training the model on prediction error and actually making money: instead of training to minimize prediction MSE, it trains directly on investment objectives like Sharpe ratio, so the combination weights are optimized for what you actually care about. Building on this foundation, Zhou et al. (2024) directly solved the objective misalignment by designing a unified reward function that combines prediction error with realized Mean-Variance (MV) utility. Using an online gradient update scheme, their MFDRL model dynamically adjusts factor loadings w_t based on continuous market feedback. This demonstrates that decision-oriented factor weighting beats purely historical OLS fits. The limits of this paradigm are documented by Sahoo et al. (2026), who benchmark a photonic quantum annealer (Dirac-3), a commercial mixed-integer programming solver (Gurobi), and a model-free deep RL agent (SAC) for factor portfolio construction on the Jensen-Kelly-Pedersen 13-factor library across a full factorial penalty sweep: while photonic hardware locates superior risk-return topologies within a narrow operating range, classical mixed-integer programming remains superior for risk-constrained mandates requiring tight tail-risk control, and the RL allocator exhibits structural failure modes under unanchored higher-moment shaping.

Infrastructure and Extensions. This section covers the practical side of RL factor combination: standard frameworks and tools that make it easier to build and test RL combiners, as well as extensions like hierarchical RL that handle multi-level decision problems. The practical infrastructure for such approaches has been significantly lowered by frameworks like Liu et al. (2020) (FinRL), which provides standardized deep reinforcement learning environments for automated trading. Earlier practical implementations include Liu et al. (2018), who demonstrated the feasibility of deep RL for stock trading using DDPG, and Li et al. (2019), who introduced regime-adaptive RL for portfolio allocation. For multi-level decision problems, Wang et al. (2020) developed a hierarchical RL framework that decomposes portfolio optimization into high-level asset allocation and low-level order execution.

4.5 LLM-Enhanced Methods

Both classical and standard ML combiners operate as passive receivers: if the upstream mining stage generates highly homogeneous factors, the combination performance is permanently capped. LLM-enhanced methods close the loop between generation and combination by letting LLM-driven pipelines participate in the weighting decision. The combination operator now depends on LLM-derived semantic signals $\mathbf{H}_t$ or LLM-generated factor pools $\mathcal{P}_t$, taking the extended form $\mathcal{T}(F_t, \mathbf{H}_t; \theta_T)$ or $\mathcal{T}(\mathcal{P}_t; \theta_T)$.

Foundational Frameworks. Luo et al. (2025a) (EFS) established the combination-aware generation paradigm, where factors are evaluated by the marginal IC they contribute to the combined composite model rather than by isolated single-factor IC, feeding anonymized combination-level performance back to mutate and select Python-based factor formulas for concentrated portfolios (validated on the US 50 and HSI 45 markets). Kou et al. (2024) built an automated factor factory in which a dual-agent evaluation system (confidence versus risk preference) filters LLM-generated factors, after which a three-layer MLP dynamically re-weights the surviving factors, proving robust during extreme market downturns on the SSE 50 and S&P 500. These works establish the key interface: generation is optimized against combination-level feedback, even though the final weights are still delegated to a downstream model.

The following sections organize LLM-enhanced factor combination along two approaches. First, we review learned weighting, where the LLM generates the factors and a learnable module combines them. Second, we cover direct weighting, where LLM outputs condition the combination directly at the signal level.

Learned Weighting. This line of work lets the LLM generate the factors but uses a learned model (like a neural network or RL agent) to figure out how much weight each factor should get, so the combination step is optimized rather than manually set. A more direct attack on the weighting problem keeps the LLM as the factor author and learns the combination weights explicitly. Chen and Kawashima (2026) address the same allocation problem with reinforcement learning: a DeepSeek model first generates 50 formulaic alphas from price, volume, and sentiment data, and a PPO agent then adjusts their weights in real time, outperforming equal-weight, buy-and-hold, and momentum baselines in Sharpe ratio and maximum drawdown. These works mark the emergence of weight allocation inside LLM-centric pipelines, although the weighting itself is still delegated to a learned downstream module rather than to the LLM.

Direct Weighting. This line of work lets LLM outputs directly influence the combined signal: instead of just generating factors, the LLM adjusts weights or gates signals based on text information like news, injecting semantic understanding directly into the combination step. A complementary line lets LLM outputs condition the combination directly at the signal level: for example, the ChatGPT-based news-gating framework of Anic et al. (2025) (reviewed in Section 6) adjusts cross-sectional momentum scores by LLM judgment rather than by learned weights. Beyond the signal level, LLM-derived views are increasingly injected into portfolio-level allocation, for instance as confidence matrices in Black-Litterman optimization, which we also review in Section 6. Overall, genuine LLM-driven weight allocation remains the least mature link of the combination stage: most systems still rely on an RL agent, an MLP head, or a classical optimizer to convert LLM-produced factors and views into final weights.

5 Task 3: Risk Control

The third stage of our Four-Step Multi-Factor Investing Framework, Risk Control applies the operator $\mathcal{Z}(\cdot; \theta_Z)$ defined in Section 1.1 to transform the combined signal $S_{raw,t}$ into a purified, risk-adjusted signal $S_{pure,t}$. The goal is to strip out market exposures that do not carry predictive alpha, ensuring the final signal represents a genuine, risk-adjusted edge that will not be overwhelmed by systematic market movements during portfolio construction. Methodologically, risk control techniques fall into two paradigms. Statistical methods (Section 5.2) refine signals through explicit statistical operations, namely cross-sectional orthogonalization against systematic risk factors, dynamic volatility scaling conditional on market regimes, and nonlinear truncation of extreme tail events. LLM-based methods (Section 5.3) extract risk perception from

unstructured text, anticipating, measuring, and neutralizing risks that numerical statistics capture only with delay.

5.1 Task Definition and Mathematical Formulation

Task definition. *Risk control* is the process of cleaning a raw predictive signal so that what reaches the portfolio optimizer is genuine alpha rather than repackaged market beta. In plain terms, a raw signal from the combination stage often still carries exposure to systematic risk factors: a high-momentum stock is also likely to be high-beta and large-cap, so naively trading on the signal loads the portfolio on common risk premia that may evaporate in different market regimes. Risk control addresses this by performing three operations in sequence: orthogonalization, which regresses the raw signal on a set of common risk factors (Barra style and industry factors) and keeps only the residual; regime scaling, which scales the residual signal up or down based on the current market volatility or turbulence, so that the same signal magnitude is dampened during turbulent regimes and amplified during calm ones; and tail truncation, which applies a nonlinear cutoff to penalize assets with high downside tail-risk probabilities before the portfolio optimization step. The methods used to perform these operations fall into two paradigms: statistical methods implement the three operations through explicit statistical operations on numerical signals (GLS regression, volatility scaling, distributional truncation); LLM-based methods run a parallel LLM-driven pipeline that extracts risk perception from unstructured text (news, filings, earnings calls) and produces a complementary text-driven risk signal that is combined with the statistical output.

Formally, the risk control operator $\mathcal{Z}(\cdot; \theta_Z)$ takes the combined signal $S_{raw,t} \in \mathbb{R}^N$ from Factor Combination together with an exogenous risk matrix $F_{risk,t} \in \mathbb{R}^{N \times P}$ (e.g., Barra style and industry factors), and produces a purified, risk-adjusted signal $S_{pure,t} \in \mathbb{R}^N$ that feeds the portfolio construction stage. The input modality is numerical price-volume signals augmented by optional textual risk indicators; the output modality is a residual alpha signal stripped of systematic exposures, scaled to current market regimes, and truncated against tail risk.

Mathematical formulation. As shown in Figure 8, the risk operator $\mathcal{Z}(\cdot; \theta_Z)$ follows a two-paradigm structure. The **statistical paradigm** decomposes into a three-stage refinement stack applied sequentially to $S_{raw,t}$: an orthogonalization stage that strips systematic exposures via generalized least squares, $S_t^{orth} = S_{raw,t} - F_{risk,t} \beta_t$; a regime scaling stage that adjusts exposure conditional on the market environment, $S_t^{reg} = \gamma_t \cdot S_t^{orth}$ with $\gamma_t \in [0, 1]$; and a tail truncation stage that applies a nonlinear cutoff driven by tail-risk estimates, $S_{pure,t} = \Psi(S_t^{reg}, TR_t | \tau)$. The **LLM-enhanced paradigm** runs a parallel three-stage pipeline over alternative textual data: risk factor extraction enriches the risk factor space with semantic information from news and filings; regime perception infers current market state from event and crisis semantics; and tail-risk forecasting generates forward-looking crash-warning estimates. The two paradigms jointly produce the purified risk-adjusted signal that feeds the portfolio optimizer. The standard objective is to maximize the out-of-sample risk-adjusted return while keeping the system dynamically adaptable to shifting market regimes.

Classification of methods. Based on whether the risk signal originates from price-volume statistics or alternative textual data, we group risk control methods into two paradigms of increasing expressiveness. **Statistical methods** (Section 5.2) restrict $\mathcal{Z}$ to explicit statistical operations on numerical signals. **LLM-based methods** (Section 5.3) parameterize $\mathcal{Z}$ with LLM-driven pipelines that extract risk perception from unstructured text and produce a text-driven risk signal that complements the statistical refinement stack. Figure 9 visualizes this taxonomy: rows correspond to method paradigms and columns to technological eras, so that the migration from classical orthogonalization, regime scaling, and tail truncation toward LLM-based semantic extraction, crash warning, and risk-aware execution can be traced directly along the timeline.

5.2 Statistical Methods

Statistical risk control is built on explicit statistical operations, namely orthogonalization, regime scaling, and tail truncation, applied sequentially to the combined signal $S_{raw,t}$. The operator decomposes as $\mathcal{Z}(\cdot; \theta_Z) = \Psi \circ \gamma \circ \phi$, where $\phi(\cdot)$ performs GLS orthogonalization against systematic risk factors, $\gamma(\cdot)$ applies regime-conditional scaling, and $\Psi(\cdot)$ applies nonlinear tail truncation. Its development follows a clear evolutionary

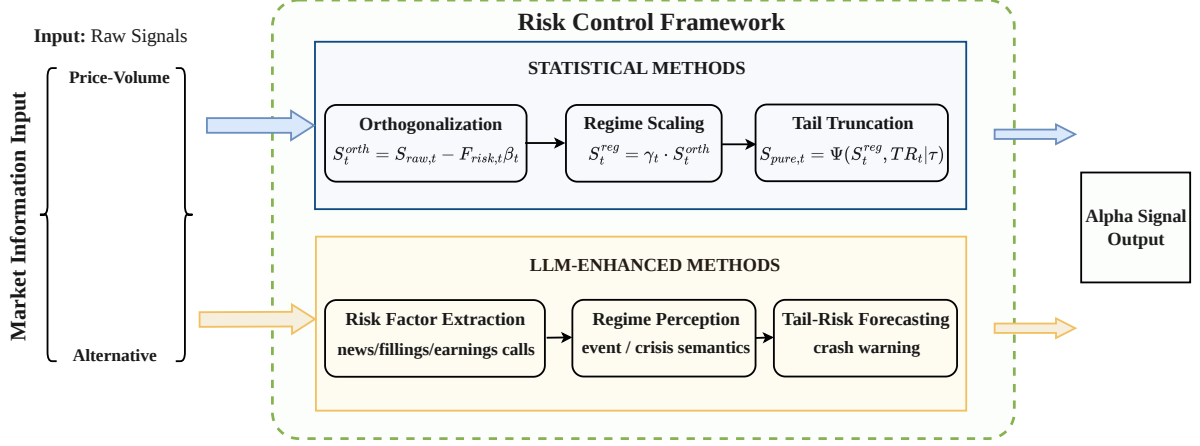


Figure 8: **Architecture of the Risk Control Module.** The risk operator $\mathcal{Z}(\cdot; \theta_Z)$ follows a two-paradigm structure. **Statistical methods** implement a three-stage sequential refinement stack over numerical signals: cross-sectional orthogonalization strips systematic exposures, regime-aware scaling adjusts signal strength to current market conditions, and tail-risk truncation bounds extreme downside probabilities. **LLM-enhanced methods** extract risk perception from alternative textual data (news, filings, earnings calls) through a parallel three-stage pipeline: risk factor extraction, regime perception, and tail-risk forecasting, producing a text-driven risk signal that complements the statistical output.

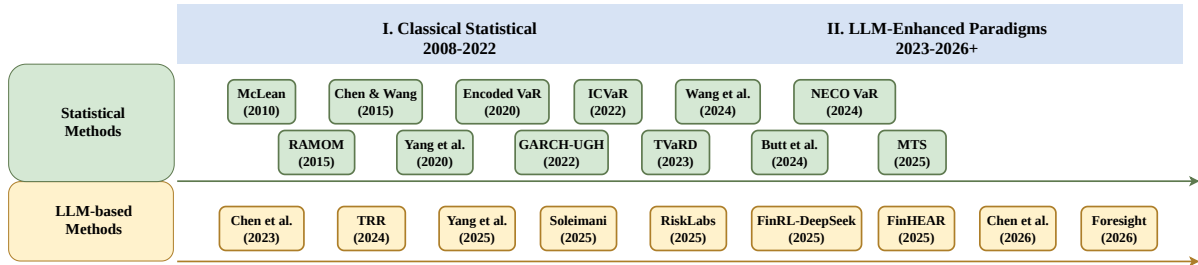


Figure 9: **AI Methods in Quantitative Finance: Risk Control.** This timeline summarizes risk control methods across two paradigms: statistical methods covering orthogonalization, regime scaling, and tail truncation, and LLM-based methods covering semantic extraction, crash warning, and risk-aware execution, ordered chronologically from classical statistical approaches to recent LLM-enhanced paradigms.

direction: it begins with the definition and neutralization of various systematic risk factors, removing linear exposures so that alpha signals remain idiosyncratic; because factor relationships are non-stationary, it then extends to market regime adjustments that scale signal strength conditional on the current environment; and because both steps rest on well-behaved distributional assumptions, it culminates in tail truncation that bounds extreme events nonlinearly.

Foundational Frameworks. The foundational logic of statistical risk control traces back to McLean (2010), who documents that idiosyncratic risk constrains arbitrage for the long-term reversal anomaly (though not for momentum). This lends empirical support for the practice of stripping common factors so that alpha signals target stock-specific mispricing rather than repackaging known risk premia. The MSCI Barra Global Equity Model (GEM) supplies the standard style and industry factor taxonomy underlying most production orthogonalization steps. Subsequent work develops each of the three stages in turn. First, orthogonalization strips systematic exposures through increasingly granular factor models. Second, regime scaling adjusts signal strength based on current and forecasted market volatility. Third, tail truncation applies nonlinear safeguards to bound extreme downside probabilities.

Orthogonalization. This is the first and most basic operation of risk control: it takes the raw signal and removes any part that can be explained by known risk factors like market beta, industry, or style, so what is left is genuine alpha rather than just repackaged market risk. Given an exogenous risk matrix $F_{risk,t} \in \mathbb{R}^{N \times P}$ (such as Barra style factors), the system orthogonalizes the raw signals using generalized least squares:

$$S_t^{orth} = S_{raw,t} - F_{risk,t}\beta_t \quad (15)$$

where β_t represents the factor exposures, isolating a clean residual signal S_t^{orth} .

The risk matrix $F_{risk,t}$ typically encompasses a comprehensive set of style and industry factors following the MSCI Barra risk model framework. The standard style factors, as formally defined by MSCI in its GEM and regional variants, include:

- **Size:** Captures the market capitalization effect, measured by the natural logarithm of market capitalization, distinguishing between large-cap and small-cap return patterns
- **Value:** Measures exposure to fundamental valuation metrics including book-to-price ratio, earnings yield, sales-to-price ratio, and dividend yield
- **Momentum:** Captures relative price momentum over 6-12 month horizons, incorporating recent price acceleration, earnings momentum, and reversal effects
- **Quality:** Composite measure of profitability (return on equity, return on assets), earnings quality, and investment patterns
- **Growth:** Represents historical and projected earnings growth rates, sales growth, and long-term earnings per share growth
- **Volatility:** Measures systematic and idiosyncratic volatility, including beta, residual standard deviation, and high-frequency return variability
- **Liquidity:** Captures trading activity through share turnover, dollar volume, and bid-ask spread characteristics
- **Leverage:** Measures balance sheet leverage including debt-to-equity ratio, debt-to-assets ratio, and interest coverage
- **Dividend Yield:** Measures the income component of returns through dividend payout ratios and trailing twelve-month dividend yields

In addition to these style factors, comprehensive risk models include industry and country factors. The Global Industry Classification Standard (GICS) provides a widely adopted taxonomy comprising 11 sectors, 24 industry groups, 69 industries, and 158 sub-industries. Key sector factors include Energy, Materials, Industrials, Consumer Discretionary, Consumer Staples, Health Care, Financials, Information Technology, Communication Services, Utilities, and Real Estate. For multi-market models, country factors account for geographic risk premiums, currency effects, and macroeconomic differences across national markets. This hierarchical factor structure ensures that alpha signals are purified of both style biases and industry-group effects before entering the portfolio construction stage.

Regime Scaling. This operation adjusts how big a bet you take based on how volatile the market is right now: when the market is calm, you can size up; when it is turbulent, you size down, so that the risk you are taking stays roughly constant rather than exploding during crises. In addition to static orthogonalization, dynamic volatility scaling adjusts signal strength conditional on current and forecasted market volatility. This addresses the fundamental non-stationarity of financial markets: a signal magnitude that is safely profitable in quiet periods may be overwhelmed by noise in volatile regimes. A scaling function assigns a dynamic weight $\gamma_t \in [0, 1]$ based on the current market state M_t :

$$S_t^{reg} = \gamma_t \cdot S_t^{orth} \quad (16)$$

During periods of high volatility, the system lowers γ_t to reduce exposure and stabilize returns over time.

For example, Dudler et al. (2015) proposed Risk-Adjusted Time-Series Momentum (RAMOM), which inversely scales momentum signals against their realized volatility. To prevent momentum crashes during severe drawdowns, Butt et al. (2024) designed a switching protocol that automatically shifts from momentum rules to a liquidity-providing reversal strategy when volatility spikes.

Modern setups use machine learning for real-time regime detection. Yang et al. (2020a) deployed the financial turbulence index as a hard-threshold risk trigger within their DRL trading system: upon extreme market signals, the agent halts buying and liquidates all open positions. Processing unstructured text, Wang et al. (2024a) combined NLP pipelines with LSTM networks to classify risk themes from financial filings. LLM-based approaches that extend regime risk perception to crash early warning and stress scenario generation are reviewed in Section 5.3.

Tail Truncation. This operation protects you from the really bad days: it applies nonlinear cutoffs to limit the size of extreme losses, using measures like Value-at-Risk or Conditional Value-at-Risk, because rare but catastrophic events can wipe out years of returns if left unchecked. Extreme tail events require explicit nonlinear safeguards beyond what linear orthogonalization and volatility scaling can provide, because these events fundamentally break the distributional assumptions of linear models. To account for the heavy-tailed distributions of financial returns, this module applies a nonlinear cutoff $\Psi(\cdot)$ defined by a tail risk vector TR_t and a risk tolerance limit τ :

$$S_{pure,t} = \Psi(S_t^{reg}, TR_t \mid \tau) \quad (17)$$

This forces the system to penalize assets with high downside probabilities before the portfolio optimization step.

The estimation of TR_t extends beyond standard normal-distribution VaR. Arian et al. (2020) introduced Encoded VaR, applying Variational Autoencoders (VAEs) to learn the nonlinear dependency structures among asset returns and generate synthetic return scenarios for Value-at-Risk estimation. For extreme quantile risk measurement relevant to regulatory requirements, Kaibuchi et al. (2022) (GARCH-UGH) combined GARCH volatility filtering with a bias-reduced extreme-value-theory-based UGH estimator to mitigate finite-sample tail-estimation bias. Because point-valued risk estimates cannot capture the full uncertainty in imprecise markets, Zhang and Zhang (2022) proposed ICVaR, an interval-valued tail risk measure with the sub-additivity property, and built portfolio selection models that embed investor risk limits to address market imprecision. For entropic tail risk under heavy-tailed returns, Choi (2026) develop parametric Entropic Value-at-Risk (EVaR) optimization for tempered stable Lévy processes: by deriving portfolio cumulant-generating functions under a multivariate normal tempered stable construction and an independent component factorization, portfolio EVaR can be evaluated directly from fitted asset-level parameters without refitting the portfolio

distribution, and minimum-EVaR portfolios achieve higher realized Sharpe ratios than their matched CVaR portfolios in out-of-sample US sector ETF allocation. To make rare-event risk estimation tractable in high dimensions, Fang et al. (2026) (ISCOS) combine cross-entropy importance sampling with the COS method, deriving Gaussian and Gaussian-inverse-Gamma proposals and analyzing how finite-COS approximation errors propagate to the fitted parameters for rare credit-portfolio losses.

To account for causal spillover effects in risk measurement, Rigana et al. (2024) proposed Causal-NECO VaR: it first identifies directed causal links across assets via a causal discovery algorithm, then quantifies contagion effects through structural equation modeling, and achieves more robust VaR estimates than conventional correlation-based methods, especially during market turmoil. As a structural risk complement to VaR and CVaR, Jha (2023) proposed TVaRD, a topological-data-analysis-based measure that quantifies changes in the topological structure of market return data under stressed conditions.

In autonomous trading setups, tail risk is now explicitly programmed into the RL agent’s learning environment. Chen and Wang (2015) embedded Mean-CVaR into Genetic Network Programming for safer capital allocation. To address the insufficient risk control common in standard deep reinforcement learning, Gu et al. (2025) (MTS) incorporated Incremental CVaR into the reward function for overall tail risk management, and used a parallel trend-sensing DRL module to dynamically relax short-selling limits during bear regimes. Along the same lines, Das et al. (2026) (MORP-DRL) frame reliability-based portfolio optimization as a multi-objective RL problem, jointly optimizing expected return and downside risk under variance, Conditional Value-at-Risk, and Entropic Value-at-Risk, with asset returns modeled through GARCH(1,1), extreme value theory, and a t -copula dependence structure and realistic scenarios generated by quasi-Monte Carlo simulation; a PPO-based strategy benchmarked against NSGA-II reduces downside risk during market stress across ten global equity indices.

5.3 LLM-based Methods

LLM-based risk control parameterizes the operator $\mathcal{Z}(\cdot; \theta_Z)$ with an LLM-driven pipeline that extracts risk perception from textual alternative data and produces a text-driven risk signal that complements the statistical refinement stack. Formally, given a textual corpus $\mathbf{X}_t^{text}$ of news, filings, and earnings calls, the LLM produces a risk-aware representation $\mathbf{H}_t^{risk} = \text{LLM}(\mathbf{X}_t^{text}; \theta_{LLM})$ that is combined with the statistical output into the purified signal $S_{pure,t}$. While statistical methods refine numerical signals whose information content is already encoded in prices, LLM-based methods exploit the semantic content of alternative data to perceive risks that price-volume statistics capture only with delay. Recent academic work applies LLMs along the entire risk-control chain, namely extracting risk factors and systemic risk measures from text, generating crash early warnings and stress scenarios, and injecting risk awareness directly into trading and hedging execution.

Foundational Frameworks. The foundational work on LLM-based risk perception traces back to Chen et al. (2023b), which uses prompt-guided LLMs to infer dynamic company-relationship graphs from daily news headlines. It shows that text-derived inter-firm links can capture cross-company spillover patterns not reflected in static correlation-based approaches. Subsequent work extends this foundational capability along three directions: semantic risk factor extraction at the firm and systemic levels, crash early warning and stress scenario generation, and risk-aware execution and hedging.

The following sections develop each direction in turn. First, semantic extraction widens the informational basis of risk models by extracting risk factors from text. Second, crash warning targets the anticipation of extreme tail events. Third, risk-aware execution embeds LLM risk perception directly into the trading loop.

Semantic Extraction. This line of work uses LLMs to read text like news and earnings calls and extract risk-relevant information from it, such as company relationship networks or default probabilities, which numeric-only risk models cannot pick up. This first direction widens the informational basis of risk models. Building on the foundational relation-graph extraction of Chen et al. (2023b) reviewed in Section 3, subsequent work moves from relation extraction to direct risk quantification. Turtel et al. (2026) build an automated pipeline that converts Risk Factors sections from SEC filings into firm-specific, time-bounded risk queries. Using the Foresight Learning framework, they train a compact LLM to estimate the probabil-

ity of risk materialization within a given horizon, which outperforms GPT-5 on probabilistic accuracy and calibration for this specific task. At the systemic level, Chen et al. (2026a) integrate LLM embeddings of Reuters news augmented with market returns to estimate Conditional Value-at-Risk (CoVaR), and derive provable error bounds for their Transformer-based quantile estimator. They show that textual information most notably improves out-of-sample systemic risk estimation during market stress periods.

Crash Warning. This line of work uses LLMs to try to spot market crashes before they happen: by reading news, social media, and analyst reports, the model looks for early warning signs of extreme events and flags them so the system can reduce risk ahead of time. A second line targets the anticipation of extreme events. Cao et al. (2025) (RiskLabs) integrates audio representations from earnings calls, text transcripts, contextual news, and VIX time-series data within an LLM-augmented multimodal framework to simultaneously forecast multi-horizon volatility and VaR. For sudden macro shocks, Soleimani (2025) implement a Prompt-RAG pipeline that simulates counterfactual stress scenarios, converting text-based news directly into quantitative tail-risk metrics. Koa et al. (2024a) (TRR) pushes further toward next-day portfolio crash prediction. It employs a custom zero-shot temporal-relational reasoning pipeline for LLMs to model relational shock-propagation and temporal effects of news events across portfolio constituents, detecting subsequent portfolio crashes without crash-specific training data and outperforming both deep-learning and alternative LLM-based baselines.

Risk-Aware Execution. This line of work embeds LLM risk perception directly into the actual trading loop: instead of just computing risk numbers, the LLM acts as a protective filter that halts or adjusts trades when semantic risk signals flash red. Benhenda (2025) (FinRL-DeepSeek) combines LLMs with deep reinforcement learning. It leverages LLM-derived stock-specific recommendation and risk scores from financial news within the CVaR-PPO (CPPO) framework: risk scores scale trajectory returns, while recommendation scores perturb trading actions. Yang et al. (2025) feeds LLM-extracted sentiment signals from news, social media, financial reports and market surveys into a dynamic hedging framework. It aggregates individual sentiment scores into a composite market-sentiment index to adjust derivatives hedge positions at configurable periodic intervals, yielding improved risk-adjusted returns relative to static hedging. Chen et al. (2025c) (FinHEAR) grounds risk-aware trading in behavioral-economic principles. Within a multi-agent LLM framework, it leverages expert-guided case retrieval, disagreement-driven risk-aware position sizing, and outcome-driven temporal refinement (including knowledge-base updates) to boost risk-adjusted trading performance. Rather than injecting sentiment as a positive alpha signal, Pillai et al. (2026) invert this role: their pipeline employs FinBERT as a protective risk filter that suppresses trading execution (without halting underlying XGBoost directional predictions) when aggregated daily ticker-level sentiment falls below critical thresholds, alongside an additional market-regime filter. This demonstrates NLP functioning as a real-time risk gate rather than a direct return predictor.

6 Task 4: Portfolio Management

As the fourth and final stage of our Four-Step Multi-Factor Investing Framework, Portfolio Management implements the utility optimization operator $\mathcal{U}(\cdot; \theta_U)$ that maps purified signals $S_{pure,t}$ and market structure Σ_t into executable portfolio weights W_t^* subject to trading constraints Ω .

We organize portfolio construction along two orthogonal axes. **Horizontally**, methods split by their temporal structure, namely **single-period** formulations that solve a static optimization at each rebalance, and **multi-period** formulations that explicitly account for the path-dependent consequences of trading. **Vertically**, single-period methods further split by their decision granularity, namely **macro-level** allocation that distributes capital across broad asset classes (equities, bonds, commodities, and alternatives) driven by macroeconomic cycles and regime changes, and **micro-level** construction that determines exact position sizes among individual assets within a single target market. Within single-period construction, both levels are organized along the same four paradigms of increasing expressiveness, namely classical mathematical optimization, optimizer technology innovations, reinforcement learning under frictions, and LLM-enhanced cognitive orchestration. Multi-period methods cut across both levels and are surveyed separately at the end of this section.

6.1 Task Definition and Mathematical Formulation

Task definition. *Portfolio management* is the process of converting a vector of predictive signals into a vector of executable portfolio weights (one per asset) that maximizes risk-adjusted return under realistic trading constraints. In plain terms, the question is simply how much of each asset to buy or sell, and this question operates at two levels. At the *macro* level, a manager distributes a given amount of capital across broad asset classes: given, say, one hundred million in funds, how much goes to equities, how much to commodities, how much to cryptocurrencies, and how much to prediction markets. At the *micro* level, a manager constructs a hedged, long-short book within a single market: after the signal ranks the constituents (e.g., the 1,000 stocks of the CSI 1000), the manager buys the top quantile expected to appreciate (e.g., the top 25%) and shorts the bottom quantile expected to depreciate (e.g., the bottom 25%), and must still decide both position sizing and breadth, such as whether a high-conviction name receives 1% or 10% of the budget, whether a short receives 10% or 20%, and whether to trade only the two extreme quantiles (top/bottom 25%), a wider window (top/bottom 30%), or even the entire ranking. The allocation must further respect practical constraints: a high-signal asset may also be high-volatility, pushing the portfolio’s risk budget beyond its limit; a small position change may incur transaction costs that erase the expected alpha; and a weight that is optimal today may not be optimal tomorrow once the portfolio has drifted. Portfolio management addresses this by solving a constrained optimization problem: maximize expected return (or risk-adjusted utility) subject to position limits, turnover bounds, transaction cost models, and (optionally) the path-dependent consequences of today’s weights on tomorrow’s feasible set. The methods used to solve this problem have evolved along a clear trajectory: classical mathematical optimization (Mean-Variance Optimization, Black-Litterman) solves a static convex program at each rebalance; optimizer technology innovations (interior-point, first-order, differentiable optimization) make the static formulation scalable to large universes with realistic constraints; reinforcement learning under frictions frames the allocation as a sequential decision problem that natively handles transaction costs and non-stationary regimes; and LLM-based cognitive orchestration lets LLM-driven multi-agent systems participate in the allocation decision, injecting semantic reasoning about macro regimes and asset-specific events.

Formally, the portfolio management operator $\mathcal{U}(\cdot; \theta_U)$ takes the purified signal $S_{pure,t} \in \mathbb{R}^N$ from Risk Control together with a market structure estimate $\Sigma_t \in \mathbb{R}^{N \times N}$ (covariance matrix) and a constraint set Ω (position limits, turnover bounds, transaction cost models), and produces executable portfolio weights $W_t^* \in \mathbb{R}^N$ satisfying $\mathbf{1}^\top W_t^* = 1$. The input modality is numerical alpha signals and market structure; the output modality is a weight vector that maximizes risk-adjusted utility under realistic trading constraints.

Mathematical formulation. As shown in Figure 10, the operator $\mathcal{U}(\cdot; \theta_U)$ is formalized as a constrained utility maximization:

$$W_t^* = \arg \max_{W_t \in \Omega} \mathcal{U}(W_t \mid S_{pure,t}, \Sigma_t) = \arg \max_{W_t \in \Omega} \left[W_t^\top S_{pure,t} - \frac{\lambda}{2} W_t^\top \Sigma_t W_t - TC(\Delta W_t) \right] \quad (18)$$

where W_t is the allocation weight vector, λ is the risk aversion coefficient, and $TC(\Delta W_t)$ denotes transaction costs incurred by weight changes. The objective balances expected return ($W_t^\top S_{pure,t}$), risk penalty ($\frac{\lambda}{2} W_t^\top \Sigma_t W_t$), and trading friction ($TC(\Delta W_t)$). Vertically, the figure separates macro-level asset allocation across equities, bonds, commodities, and alternatives from micro-level construction over individual stocks; horizontally, the paradigms proceed from mean-variance optimization and Black-Litterman view injection, through RL agents looping over state, action, and reward, to LLM-based cycles of reasoning, planning, tool use, and evaluation. The standard objective is to maximize the out-of-sample risk-adjusted return while keeping the system dynamically adaptable to shifting market regimes.

Classification of methods. Based on the temporal structure and decision granularity, we organize portfolio construction methods into a two-level taxonomy. **Single-period methods** (Section 6.2) solve a static optimization at each rebalance, further split by decision granularity into **macro-level methods** (Section 6.2.1) for cross-asset allocation and **micro-level methods** (Section 6.2.2) for intra-market position sizing. **Multi-period methods** (Section 6.3) explicitly model the path-dependent consequences of trading. Across single-period construction, methods follow four paradigms of increasing expressiveness, namely classical mathematical optimization, optimizer technology innovations, reinforcement learning under frictions, and LLM-enhanced cognitive orchestration; the optimizer-technology paradigm is surveyed once under micro-

level construction (Section 6.2.2) because solver innovations are methodology-agnostic. Figure 11 visualizes this taxonomy chronologically: rows correspond to method paradigms (classical optimization, RL, LLM) and columns to technological eras, so that the migration from static mean-variance solvers toward RL agents and LLM-driven multi-agent systems can be traced directly along the timeline.

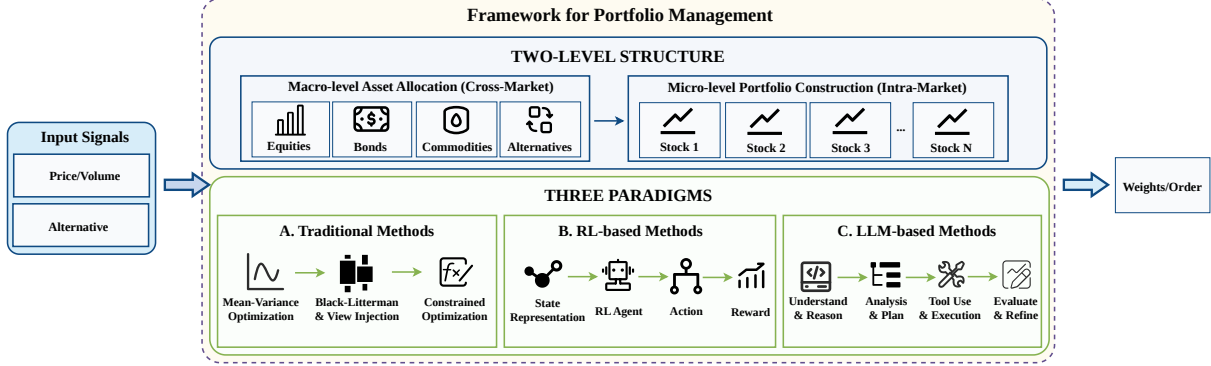


Figure 10: **Architecture of the Portfolio Management Module.** The portfolio management operator $\mathcal{U}(\cdot; \theta_U)$ is organized along two structural dimensions. **Vertically**, a two-level structure separates macro-level asset allocation across broad asset classes (equities, bonds, commodities, alternatives) from micro-level position sizing within a single market. **Methodologically**, three paradigms of increasing expressiveness are applied at both levels: traditional methods (Mean-Variance Optimization, Black-Litterman, and constrained optimization), reinforcement learning agents that handle sequential decisions under trading frictions, and LLM-based systems that bring semantic reasoning to the allocation process.

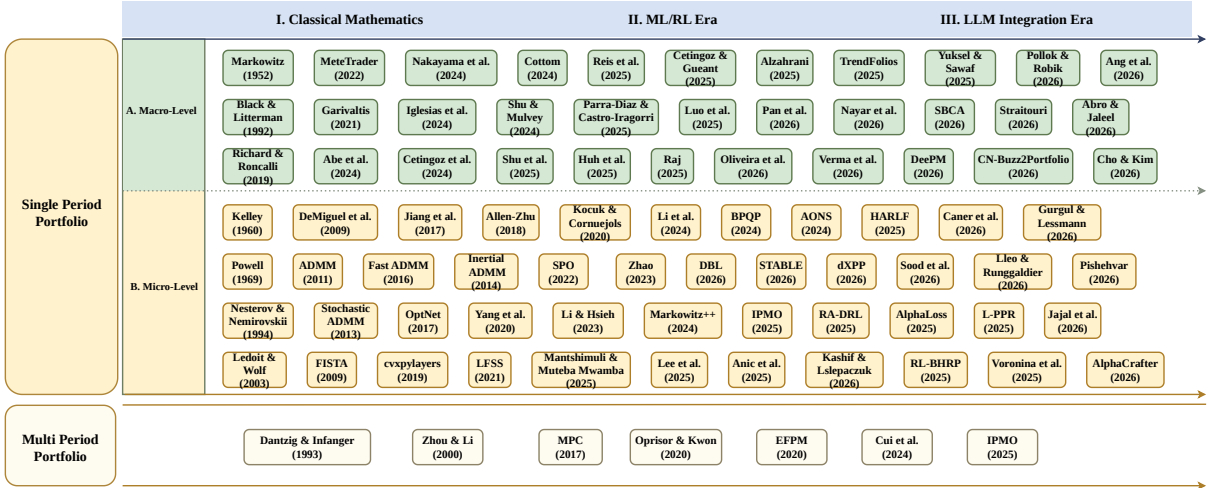


Figure 11: **AI Methods in Quantitative Finance: Portfolio Management.** This branch focuses on learning-based portfolio construction methods, where reinforcement learning approaches are commonly used to handle sequential decision-making under trading frictions.

6.2 Single-Period Portfolio Construction

Single-period methods solve the allocation problem at each rebalance independently, without explicitly modeling the path-dependent consequences of today's weights on tomorrow's feasible set. The single-period formulation $W_t^* = \arg \max_{W_t \in \Omega} \mathcal{U}(W_t | S_{pure,t}, \Sigma_t)$ is solved at each rebalance with the path-dependent term $TC(\Delta W_t)$ approximated using the weight change from the previous period. We organize single-period methods by the macro/micro axis introduced above, and then by methodological paradigm within each level.

6.2.1 Macro-Level Methods

Macro-level allocation distributes capital across broad asset classes, namely equities, bonds, commodities, and alternatives, rather than across individual names within a single market. The decision is driven primarily by macroeconomic cycles, regime shifts, and cross-asset correlation structure, and is the classic problem of pension funds, sovereign wealth funds, and multi-asset balanced mandates. Methodologically, the field follows the same four-paradigm arc as micro-level construction, but the Optimizer paragraph is omitted here because solver-level innovations (interior-point, FISTA, ADMM, differentiable optimization, decision-focused learning) are methodology-agnostic and are surveyed once under Micro (Section 6.2.2). We therefore organize macro-level allocation into three paragraphs: Traditional risk-parity and regime-aware methods, RL-based regime-aware allocation, and LLM-driven macro allocation.

Foundational Frameworks. The foundational logic of risk-budgeting-based multi-asset-class allocation originates from Maillard et al. (2010), who formalized the equal-risk-contribution paradigm for asset allocation. Subsequent extensions including constrained risk-budgeting and generalized risk-measure formulations are developed in Richard and Roncalli (2019) and Cetingoz et al. (2024). The Black-Litterman model (Black and Litterman, 1992) supplies the standard view-injection mechanism that merges market equilibrium with macro-prior views. Subsequent work organizes macro-level allocation into three methodological paradigms of increasing expressiveness. First, traditional methods cover static-construction techniques such as mean-variance optimization, Black-Litterman view injection, and risk-parity allocation. Second, RL methods bring sequential, regime-aware decision-making to the allocation problem. Third, LLM methods bring semantic macroeconomic reasoning to cross-asset allocation.

Traditional Methods. This line of work covers the classical static approaches to cross-asset allocation that have been the institutional standard for decades, including mean-variance optimization, Black-Litterman, and risk parity, along with regime-aware extensions. Macro-level allocation has historically been dominated by three families of static-construction techniques: mean-variance optimization over multi-asset covariance matrices, Black-Litterman view injection with macro-prior views, and risk-parity / risk-budgeting allocation that equalizes risk contributions across asset classes. Risk budgeting has become a de-facto benchmark for many institutional multi-asset balanced mandates, as it mitigates extreme weights caused by estimation error within mean-variance optimization. Richard and Roncalli (2019) and Cetingoz et al. (2024) provide key theoretical and algorithmic extensions to the risk-budgeting framework, and Cetingoz and Guéant (2025) further extends it to balance risk contributions from individual assets and underlying risk factors. Algorithmic refinements for risk-budgeting computation include mirror-descent solvers (Iglesias et al., 2024) and differentiable deep-declarative variants (Parra-Díaz and Castro-Iragorri, 2025). Tree-structured portfolio methods, which share the recursive-allocation spirit of López de Prado’s hierarchical risk parity, comprise Schur complementary allocation (Cotton, 2024) and topological risk parity (Nayar et al., 2026), both suited for multi-asset universes with complex dependence structures. As a theoretical reference for risk-budgeting thinking under continuous-time Black-Scholes setting, universal risk-budgeting (Garivaltis, 2021) offers a closed-form benchmark in the two-asset case.

Regime-aware multi-asset allocation augments these static constructions with macroeconomic regime detection. Oliveira et al. (2026) use a modified k-means over the FRED-MD macro panel to obtain probabilistic regime assignments, and feed regime-conditioned return estimates into portfolio position sizing. Shu et al. (2025) extend this by generating asset-specific (rather than universe-wide) regime forecasts via statistical jump models and gradient-boosted trees. These regime-conditioned forecasts are fed into Markowitz-style portfolio optimizers, including mean-variance, minimum-variance and equal-weighted schemes, across twelve global equity, bond, real-estate, and commodity indexes. Luo et al. (2025c) incorporate tail-dependence and regime-shift modelling into dynamic asset allocation. While the report’s primary focus lies on multi-asset-class allocation, it also offers insights for factor-allocation applications. Shu and Mulvey (2024) draw on regime-analysis outputs from the sparse jump model to dynamically tilt style-factor exposures across U.S. equity factor indices. Multi-asset covariance forecasting, a key input for many of the above portfolio-construction approaches, is addressed by Reis et al. (2025) via a hybrid deep-learning-shrinkage framework designed for medium-term forecast horizons. High-dimensional continuous-time multi-asset port-

folio choice under stochastic investment-opportunity dynamics has also been studied using Pontryagin-guided direct policy optimization (Huh et al., 2025).

RL Methods. This line of work uses reinforcement learning to handle the dynamic, regime-shifting nature of cross-asset allocation: the agent learns to move capital between asset classes based on the current market environment, adapting automatically rather than following a fixed static rule. Reinforcement learning brings sequential, regime-aware decision-making to macro allocation. Raj (2025) derive latent-regime posterior probabilities via a pre-trained Hidden Markov Model and augment the PPO observation space with these features. This lets a single policy network (including Transformer-PPO variants) adapt its allocations across shifting macroeconomic regimes. Niu et al. (2022) train a DQN-based meta-policy to select from a set of imitation-augmented learned base policies, conditioned on macro-market features and the historical performance of each candidate policy. Their two-stage workflow separates base-policy training from downstream policy-selection. More recent work focuses on continual and risk-aware regime learning: Pan et al. (2026a) employ continual-learning mechanisms to mitigate catastrophic forgetting across market-detected regimes, Verma et al. (2026) combine HMM regime detection with tabular RL policy-iteration learning, and Alzahrani (2025) employ the MARCD regime-conditioned diffusion framework featuring a CVaR-epigraph portfolio allocator. On the cross-asset futures side, Pollok and Robik (2026) train end-to-end AI policies on sixteen liquid CME futures using a differentiable Sharpe-ratio loss, benchmarking against equal-weight, risk parity, and time-series momentum rules. Wood et al. (2026) (DeePM) combine a macroeconomic graph prior with a differentiable SoftMin distributionally-robust objective, achieving roughly twice the net risk-adjusted returns of classical trend-following strategies over the 2010-2025 out-of-sample period across fifty diversified futures. Multi-asset cross-modal RL includes Pan and Chen (2026), which employs an Actor-Critic architecture with a gated cross-modal fusion module, combining BERT-derived news-sentiment features and multi-asset price time-series, and Lu et al. (2025), which use multi-horizon majority-voted momentum and trend-following signals together with inverse-tracking-error weighting to build long-only multi-asset ETF portfolios.

LLM Methods. This line of work uses LLMs to read macroeconomic news, central bank statements, and other text to inform cross-asset allocation decisions, bringing semantic understanding of macro regimes and events that purely numerical models miss. Large Language Models bring semantic macroeconomic reasoning to cross-asset allocation. Abe et al. (2024) assign diverse investor personas (including institutional-style personas) to LLMs and feed them historical daily macro-market indicators to build multi-persona ensembles for equity-bond positioning. Chen et al. (2026b) release the CN-Buzz2Portfolio benchmark, which processes daily trending Chinese financial news for macro and sector ETF allocation via a tri-stage Compression-Perception-Allocation agent workflow, explicitly operating at the macro/sector level rather than at the stock-picking level. Nakayama et al. (2024) predict stock-bond correlation-coefficient changes using features derived from financial-news and central-bank texts alongside market time-series data, mitigating the lag of price-driven correlation estimates under regime shifts. Abro and Jaleel (2026) propose a heuristic variant of no-regret online learning for long-only portfolio-level allocation. It uses sentiment-driven rebalancing gating and leverages LLMs to suggest complementary hedging sectors, rather than directly producing portfolio weights. At the meta-level, Straitouri et al. (2026) use LLMs to produce a robust candidate pool of optimization-model formulations, where the LLM samples and ranks alternative mathematical problem specifications instead of producing asset weights or allocation rules. End-to-end agentic approaches for asset allocation include the multi-agent societies of Yuksel and Sawaf (2025), which recast allocation as an LLM-driven tournament over hierarchical trees; the cryptocurrency-focused multi-agent trading system from Luo et al. (2025b); and the institutional strategic-asset-allocation pipeline in Ang et al. (2026). The latter deploys a 50+ agent network with dedicated Macro, Asset-Class, Covariance, and CIO-agent roles, all governed by a top-level Investment Policy Statement. For evaluating LLM portfolio-optimization reasoning (a core sub-task within macro allocation), Cho and Kim (2026) provide the PortBench benchmark infrastructure of mathematically-grounded multiple-choice problems.

6.2.2 Micro-Level Methods

Micro-level construction determines exact position sizes among individual assets within a single target market, typically a single equity market such as S&P 500, CSI 300, or a crypto universe. Unlike macro allocation,

the decision here is driven primarily by cross-sectional alpha signals and their covariance, rather than by macroeconomic regime shifts. Micro-level methods follow the full four-paradigm arc, including the Optimizer paragraph, because solver-level innovations (interior-point, first-order, differentiable, decision-focused) were historically driven by the scaling requirements of large equity universes. We therefore survey four paragraphs: Traditional baselines, Optimizer technology, RL-based dynamic weighting, and LLM-based agentic systems.

Foundational Frameworks. The foundational framework of micro-level portfolio construction traces back to Markowitz (1952) (Mean-Variance Portfolio Optimization, MVO), who established the mean-variance optimization paradigm that underlies all modern variants. The Black-Litterman model (Black and Litterman, 1992) provides a widely-used mechanism for blending a market-equilibrium Bayesian prior with confidence-weighted investor views. Covariance shrinkage estimators pioneered by Covariance shrinkage estimators, pioneered for portfolio-optimization use by Ledoit and Wolf (2003), mitigate severe estimation-error pathology of high-dimensional sample covariance matrices, which gives rise to the well-known error-maximization effect in mean-variance optimization. Subsequent work develops micro-level portfolio construction along four paradigms of increasing expressiveness. First, traditional methods include static MVO, Black-Litterman, and other foundational baselines. Second, optimizer technology covers solver innovations that scale the formulation to large universes with realistic constraints. Third, RL methods reframe portfolio construction as a sequential decision problem under trading frictions. Fourth, LLM methods introduce cognitive orchestration that injects semantic reasoning into the allocation decision.

Traditional Methods. This line of work covers the classical static approaches to portfolio construction within a single market, built on the Markowitz mean-variance framework, including Black-Litterman and other standard baselines.

Operating under the assumption of a highly idealized, frictionless market, early financial theories sought a mathematically perfect, static baseline. Markowitz (1952) defined this foundational mean-variance paradigm: given expected returns μ and covariance matrix Σ , find weights w that minimize $w^\top \Sigma w$ subject to $w^\top \mu = r$, $\sum_i w_i = 1$, and $w_i \geq 0$ (long-only constraint). Building upon this static constrained-optimization formulation, we formalize a dynamic utility-maximization investment problem. Our objective augments the classic risk-return utility with risk-aversion coefficient λ and transaction-cost terms for portfolio rebalancing:

$$\max_{W_t} \left[W_t^\top S_{pure} - \frac{\lambda}{2} W_t^\top \Sigma W_t - TC(\Delta W_t) \right] \quad (19)$$

where W_t denotes the period- t allocation weight vector, S_{pure} denotes our custom measure of expected asset returns, Σ is the return covariance matrix, $\lambda > 0$ is the risk-aversion parameter, and $\Delta W_t = W_t - W_{t-1}$ represents the weight change from the prior period, with $TC(\Delta W_t)$ the corresponding transaction-cost function.

In live trading, traditional MVO often acts as an error maximizer. Tiny estimation errors in S_{pure} produce extreme, uninvestable allocation weights. It also assumes returns follow a normal distribution, ignoring fat-tail risks entirely. To fix these structural flaws, Boyd et al. (2024) (Markowitz++) expanded the classical mean-variance framework by embedding transaction costs, holding-cost terms, and worst-case forecast uncertainty for both returns and covariances within a tractable convex-optimization formulation, additionally supporting soft constraints to avoid infeasibility. Working on return-free asset allocation, Cetingoz and Guéant (2025) studied risk-budgeting frameworks for both asset and factor dimensions, where risk contributions are derived from multi-factor models combining factor loadings and covariance matrices; equal-risk-contribution configurations are considered as important special-case illustrations. Generalizing the classical setup in closed form, Gasparavičius and Grigutis (2026) show that Markowitz’s key results remain available under any strictly convex quadratic risk measure: replacing the covariance matrix with an arbitrary symmetric positive definite matrix yields explicit formulas for the efficient frontier, the global minimum-risk portfolio, the maximum-Sharpe-ratio portfolio, the capital market curve, and the tangency portfolio, and reveals that the tangency portfolio no longer coincides with the maximum-Sharpe-ratio portfolio. A path-first reformulation is developed by Noguer i Alonso (2026) (Path Portfolio Optimization), which treats the signature of the price path as the universal coordinate and reduces mean-variance portfolio selection to a linear system over a single tensor, showing that the gap between the Marcus and forward lifts contracted with portfolio weights is

exactly Fernholz’s excess growth rate; empirically, quadratic path functionals raise the certainty equivalent elevenfold for a pair of assets and sixtyfold for a cross-section of twenty when the expected signature is known, though the unregularized policy degrades sharply until the sample exceeds roughly six observations per parameter.

To reduce parameter sensitivity and add human inputs, the Black-Litterman (BL) model uses Bayesian inference to merge market equilibrium with specific investor views. But classical BL only operates on single-period horizons. Abdelhakmi and Lim (2026) (DBL) addresses this limitation by extending BL to continuous time, employing generalized multi-dimensional Brownian-bridge constructions (conceptually related to Kalman smoothing) to accommodate mismatches between view and investment horizons. Relaxing the Gaussian-return assumption underlying classical Black-Litterman, Kocuk and Cornuéjols (2020) model asset returns as a mixture of normals and optimize for Conditional Value-at-Risk (CVaR). Using an inverse-optimization framework, they extend the BL paradigm to blend market-equilibrium priors and investor views under this non-Gaussian setup. Parallel to this, Li and Hsieh (2023) develop an adaptive framework combining Black-Litterman with Elastic-Net regularized factor regression for data-driven view generation and stabilized parameter estimation. A key feature is its volatility-responsive sliding-window that endogenously adjusts the estimation horizon.

To partially automate view generation for Black-Litterman, Zhao (2023) extended the BL pipeline by employing LSTM networks to forecast factor performance, substituting human-selected view targets with deep-learning outputs, while view-magnitude and-confidence hyper-parameters remained manually specified. Addressing market regime shifts, Soun et al. (STABLE) employ a conditional-diffusion generative model to produce regime-aware return trajectories. From these sampled dynamic stock-level distributions, they derive adaptive Black-Litterman view moments (mean and covariance); notably, view uncertainty is directly obtained from the diffusion samples, removing manually-specified confidence hyper-parameters. Within their limited out-of-sample test window, STABLE yielded improved risk-adjusted returns and time-series generation accuracy under volatile market conditions. Moving beyond view-based allocation, Li (2026) (KellyBoost) bypass the predict-then-optimize gap altogether: a single multi-output XGBoost produces, as its softmax output, the growth-optimal (Kelly) allocation itself, trained on the negative log growth rate $-\log(1 + w^\top y)$ with analytically derived gradient and Hessians verified by finite differences, so the portfolio is learned end-to-end from features without a separate return forecast or optimizer. Still within data-driven construction, Garg (2026) (TDA+News) build portfolios directly from asset dissimilarity: a topological-data-analysis-based distance combined with FinBERT sentiment scores is used within agglomerative clustering to identify topologically dissimilar assets, with a dynamic rolling-window rebalancing strategy and a retention mechanism that keeps high-quality assets across consecutive windows to reduce turnover; on S&P 500 constituents the framework outperforms correlation- and Euclidean-distance-based methods as well as Naïve, index, and full-universe benchmarks.

Optimizer Technology. An optimizer is the computational engine that figures out exactly how much of each stock to hold given a ranked signal and a set of real-world constraints. The constraints can be things like how much transaction cost you are willing to pay, how much market impact each trade has, what percentage of your portfolio any single position can be, what the maximum turnover rate is, and any regulatory requirements. Given the list of stocks sorted by expected return and all these constraints, the optimizer solves for the exact weight of each asset that maximizes expected return or risk-adjusted utility. The MVO framework, first proposed by Markowitz (1952), established the mathematical foundation of modern asset allocation. Over 70 years of development, the field has evolved along two complementary dimensions.

First, problem formulation has progressed from unconstrained quadratic programs to increasingly practical, heavily constrained formulations that explicitly model transaction costs, position limits, regulatory constraints, and market frictions. Second, solver technology has evolved from black-box mathematical optimizers treating the prediction step as fixed input, to fully differentiable optimization layers that integrate prediction and optimization end-to-end, enabling gradient flow from portfolio performance directly to predictive model parameters.

This dual evolution recognizes that theoretical alpha is easily destroyed by transaction costs and operational constraints, but equally by the fundamental misalignment between prediction objectives and optimization objectives. The modern constrained optimizer thus serves as both a mathematical engine bridging idealized predictive signals with final, executable portfolio weights, and increasingly as a trainable component of the end-to-end system itself.

1) Traditional Exact Solvers

The first generation of portfolio optimizers focused on finding exact solutions to increasingly constrained quadratic programs, primarily using interior-point methods. This era corresponds to the classical predict-then-optimize paradigm.

Interior-point methods became a workhorse solver for MVO problems from the 1990s through the 2010s. The theoretical foundation laid by Nesterov and Nemirovskii (1994) established polynomial-time convergence guarantees for convex-cone programs when initialized from strictly feasible interior starting points. For problems with up to a few hundred assets, interior-point methods delivered excellent numerical performance and high precision.

The Black-Litterman model (Black and Litterman, 1992) represented the most important methodological advance of this era. Rather than requiring investors to specify expected returns directly, a notoriously difficult task, it used equilibrium returns as a Bayesian prior and allowed investors to update these priors with subjective views quantified by confidence levels. This dramatically improved the stability of MVO outputs.

Covariance shrinkage estimators, pioneered by Ledoit and Wolf (2003), addressed the fundamental problem that sample covariance matrices in high dimensions are dominated by noise. By pulling extreme eigenvalues toward their grand mean using an analytically optimal shrinkage intensity, this simple regularization dramatically improved out-of-sample portfolio stability.

The regularization era unified these insights. DeMiguel et al. (2009) demonstrated systematically that ℓ_1 , ℓ_2 , and maximum-norm constraints correspond to different sparsity-stability tradeoffs. ℓ_1 regularization, in particular, proved remarkably effective at inducing sparse, low-turnover portfolios while maintaining convexity. On the seventy-year anniversary of Markowitz’s portfolio-selection framework, Boyd et al. (2024) present original work demonstrating that constraints long treated as ad-hoc practical fixes can serve as effective tools for mitigating estimation error within a unified convex-optimization setup.

The limitations of the first generation were threefold: first, solvers were treated as black boxes, with no gradient flow between prediction and optimization; second, a fundamental objective mismatch existed, as prediction models were trained on mean squared error rather than decision quality; third, interior-point methods struggled to scale beyond approximately 500 assets with realistic institutional constraints.

2) First-Order Methods and Decomposition

As portfolio universes expanded beyond what interior-point methods could efficiently handle, and as trading frequencies increased, the field shifted to first-order optimization methods. This generation was defined by two parallel developments: accelerated proximal gradient methods and decomposition-based approaches.

2.a) Proximal Gradient Acceleration.

The foundational work by Beck and Teboulle (2009) (FISTA) introduced Nesterov-style acceleration to proximal-gradient methods, lifting the non-asymptotic objective-error convergence rate from $O(1/k)$ to the theoretically optimal $O(1/k^2)$ for composite convex minimization. FISTA augments the standard proximal-gradient update with a momentum extrapolation rule driven by an auxiliary sequence $\{t_k\}$, requiring only cheap vector arithmetic and no extra expensive gradient or proximal evaluations. For ℓ_1 -regularized portfolio optimization – with weight-domain ℓ_1 for sparsity and trade-vector ℓ_1 penalties to limit turnover – FISTA converges faster than projected gradient descent. Importantly, vanilla FISTA provides no guarantee of monotonic objective improvement, in contrast to standard ISTA. Later extensions include MFISTA for monotonicity and stochastic FISTA variants.

Progress in finite-sum stochastic convex optimization advanced substantially with Allen-Zhu (2018), who proposed Katyusha, the first primal-direct accelerated stochastic variance-reduced algorithm that attains the optimal complexity $O((n + \sqrt{n\kappa}) \log(1/\epsilon))$ for strongly-convex composite finite-sum minimization. The core innovation is Katyusha momentum: a convex-combination “magnet” term that draws each iterate toward the epoch-level snapshot gradient point $\tilde{x}$ in every iteration, suppressing error accumulation from stochastic gradient noise. While suitable for rolling-window portfolio-optimization tasks where each re-balance has access to the full set of historical samples, Katyusha requires a full-gradient evaluation per epoch and is therefore not applicable to pure online settings with strictly sequentially-arriving market data. Numerical experiments demonstrate large reductions in data passes relative to non-accelerated variance-reduced baselines such as SVRG and SAGA, with speed-ups varying across datasets and target accuracy.

2.b) Large-Scale Decomposition: The ADMM Family.

The Alternating Direction Method of Multipliers (ADMM), rooted in the augmented Lagrangian method of Powell (1969) and the cutting-plane ideas of Kelley (1960), emerged as the industry standard for large-scale portfolio construction in the 2010s. The definitive reference for the entire ADMM framework remains Boyd et al. (2011).

By decomposing the global problem into subproblems solved in parallel and coordinated through dual variable updates, ADMM gracefully scales to portfolios with tens of thousands of assets. Several key variants were developed:

Fast ADMM (Song et al., 2016) propose a suite of fully-decentralized adaptive-penalty ADMM variants for distributed consensus optimization. ADMM-VP performs penalty updates from local primal-dual residual ratios, eliminating the global aggregation required by older adaptive-penalty ADMM schemes. Complementary variants ADMM-AP and ADMM-NAP adjust edge-wise penalties using relative local-objective function differences, with NAP adding a change-budget mechanism to remove manual tuning of maximum penalty-update iterations. These heuristics deliver empirical early-stage convergence improvements chiefly on well-connected graphs, with only negligible additional computational overhead; however they do not yield provably better theoretical convergence rates, and speed-up benefits vanish under weak network connectivity.

Stochastic ADMM Ouyang et al. (2013) develops a stochastic variant of ADMM for i.i.d. stochastic risk minimization, where only single samples are drawn in each iteration (mini-batch extensions are not analysed in the original work). Conceptually, this incremental update paradigm lends itself to streaming factor-model-update scenarios with continuously arriving return observations. As a first-order incremental method, Stochastic ADMM can refine factor exposures without explicit full covariance-matrix recomputation, though this property is shared with many other stochastic first-order solvers.

Inertial ADMM Bot and Csetnek (2014) introduced heavy-ball-type inertial extrapolation into the ADMM framework. While theoretically promising for ill-conditioned problems such as highly-correlated multi-factor portfolios, the paper is purely theoretical and contains no numerical benchmarks or measured speed-up factors.

Subsequent industry-oriented implementations have adopted further practical optimizations separate from the original theoretical work. Warm-starting from the prior trading day’s solution lowers iteration counts under stable market conditions, and inexact early termination trades minor loss in precision for large speed-ups. Combined, these practical enhancements can yield solve times below 100 ms for 2000-asset MVO instances subject to realistic institutional constraints. Complementing faster solvers, preprocessing can shrink the problem before optimization begins: Wada et al. (2026) derive safe screening rules for cardinality-constrained portfolio optimization that fix binary asset-selection variables to zero or one without excluding any globally optimal solution, using a perspective relaxation of the ℓ_2 -regularization term and Fenchel duality to compute asset-specific scores; on S&P 500 and Russell 2000 datasets these rules yield substantial computational speed-ups on challenging instances, particularly under moderate or strong regularization and less stringent return requirements.

3) The Differentiable Optimization Revolution

Representing a major advance towards overcoming the predict-then-optimize barrier, Amos and Kolter (2017) (OptNet) demonstrated how full quadratic-programming solvers could be integrated as differentiable layers within neural-network architectures. Prior work such as iterative unrolling enabled gradient propagation through optimization procedures, but relied on truncated-iteration approximate gradients rather than derivatives of the exact argmin solution. Drawing on sensitivity analysis via Karush-Kuhn-Tucker conditions, OptNet exploits that the QP solution mapping is subdifferentiable everywhere and differentiable almost everywhere (away from degenerate constraint points). This work adapted argmin-differentiation machinery to the full equality-and-inequality-constrained QP setting and provided a practical GPU-batched solver, laying important practical foundations for end-to-end learning of optimization-aware systems.

Representing a major advance in differentiable optimization layers, Agrawal et al. (2019) (cvxpylayers) generalizes differentiable convex-optimization layers from pure quadratic-programming settings to problems compliant with disciplined parametrized programming (DPP), a restricted subset of disciplined convex programming (DCP). Drawing on implicit-function-theorem analysis of KKT optimality conditions, consider a parameterized convex program

$$\begin{aligned} \phi(\theta) = \arg \min_x f(x; \theta) \\ \text{s.t. } g_i(x; \theta) \leq 0, \quad h_j(x; \theta) = 0 \end{aligned} \tag{20}$$

where f , g_i , h_j must be affine in the parameters θ under DPP. Under strong regularity (non-degenerate KKT) assumptions, gradients of the solution map $\nabla_{\theta} \phi(\theta)$ may be computed analytically using primal-dual optimal variables, without differentiating through the full solver iteration trajectory. At degenerate points where regularity fails, approximate least-squares gradients are used. This framework enables DPP-compliant convex optimization problems to be integrated as layers within deep-learning pipelines.

In portfolio management, this technology enables three key capabilities: first, training return prediction models with losses directly tied to out-of-sample portfolio performance rather than mean squared error; second, learning risk aversion and regularization parameters end-to-end; and third, embedding hard regulatory constraints as optimization constraints that are respected during training. Empirically, cvxpylayers-based systems have demonstrated improvement in out-of-sample Sharpe ratios compared to predict-then-optimize baselines by eliminating objective mismatch between the prediction and optimization stages.

Subsequent advances extended the framework to portfolio-specific requirements along three dimensions:

BPQP (Backward-Pass Quadratic-Programming) Pan et al. (2024) improves the efficiency of differentiable convex-optimization layers used within neural-network mini-batch training. It achieves large empirical speed-ups over competing implicit-differentiation baselines by reformulating the backward-pass gradient computation as an equivalent quadratic-programming subproblem and decoupling forward- and backward-pass solvers. While demonstrated on an end-to-end portfolio-optimization learning task, BPQP targets gradient-enabled model training rather than pure historical backtesting workflows, which typically require only forward solves without back-propagation.

dXPP Linghu et al. (2026) improves scalability for large-scale strictly-convex quadratic-programming problems with convex institutional-style constraints. Leveraging softplus-smoothed exact-penalty reformulations, the method constructs globally twice-continuously-differentiable surrogate objectives to compute gradients, supporting end-to-end training pipelines for portfolio optimization under convex constraints. Note that dXPP is restricted to convex QP and cannot directly handle non-convex constraints such as cardinality or minimum-position-size limits.

By the end of the third generation, the fundamental barrier between prediction and optimization had been broken. Gradients could now flow seamlessly from portfolio performance losses directly through the optimizer to the prediction model. However, the full implications of this capability for decision-making had not yet been fully explored.

4) Decision-Focused Learning and Full Integration

Within machine-learning-for-prescriptive-analytics, Elmachet and Grigas (2022) developed the SPO framework, an important contribution to decision-focused learning. The work formalized a widely-observed insight:

minimizing prediction vector error does not guarantee minimal decision-level loss. Even with imperfect return forecasts, it is possible to obtain portfolios whose performance matches that achievable under true returns; conversely, small prediction errors can lead to large degradation in realized portfolio utility.

The SPO loss quantifies decision regret as the gap between the utility attained by decisions from predicted parameters and the true optimal utility (taking worst-case decisions under multiple optima). While SPO itself is non-convex and intractable for direct training, the authors derive the convex surrogate SPO+ loss and prove Fisher-consistency guarantees under mild distributional assumptions. This highlights that prediction models ought to be trained using optimizer-aware metrics, where errors are weighted by their downstream decision impact rather than uniformly as in mean-squared-error loss.

Subsequent practical approaches drawing on this principle fall into three broad implementation patterns in portfolio applications:

1. **Head Splitting:** Shared feature extractors with separate prediction heads for distinct risk preferences, each trained under decision-aware regret-style loss.
2. **Loss Distillation:** MSE-based pre-training, followed by fine-tuning on portfolio-centric decision-loss objectives.
3. **Robust Regularization:** Optimizing against worst-case return perturbations to improve robustness to return errors that would trigger large portfolio adjustments.

The key empirical result, confirmed across numerous follow-up studies, is that models trained under regret-aligned losses typically achieve better out-of-sample portfolio performance despite having higher prediction MSE. This decisively demonstrates that mean squared error is indeed the wrong proxy objective for investment decision-making. Robustness to return misestimation can be certified directly: Hsieh and Gan (2026) develop a scalable, certified approximation for high-dimensional Wasserstein distributionally robust portfolio optimization, showing that under the order-one Wasserstein ambiguity with a one-norm ground metric the robust problem admits a polynomial-size linear-programming reformulation via a finite hyperplane-dual construction with uniform utility-approximation error bounds, validated through monthly rebalancing of a 476-asset portfolio and scaling to 1,000 assets.

The current frontier extends single-period decision-focused learning frameworks to multi-period settings with Linghu et al. (2025) (IPMO). Built upon a model-predictive-control formulation, IPMO introduces a differentiable multi-period optimization layer that explicitly accounts for the path-dependent nature of trading frictions. These end-to-end systems are surveyed in detail in Section 6.3.

RL Methods. This line of work uses reinforcement learning to handle the sequential nature of portfolio construction with trading frictions: instead of solving a static optimization each period, the agent learns a policy that naturally accounts for transaction costs, path dependence, and non-stationary regimes. Moving beyond single-period convex optimization, reinforcement learning reframes portfolio construction as a sequential Markov Decision Process. This paradigm shift explicitly models the path-dependent nature of trading frictions and regime changes.

Continuously stacking constraints onto static mathematical optimizers eventually creates computational bottlenecks and severe multi-period myopia: today’s rebalancing costs may erode tomorrow’s capital base. Recognizing these limitations, modern research has moved beyond constrained single-period optimization entirely, reframing portfolio construction as a sequential decision problem that explicitly accounts for the path-dependent consequences of trading. This enables dynamic agents capable of friction-aware learning over time.

Early DRL trading approaches often adopted Actor-Critic architectures such as DDPG to map raw market prices to continuous portfolio weights. Working in highly-volatile cryptocurrency markets, Jiang et al. (2017) departed from standard Actor-Critic designs and proposed a critic-free model-free DRL agent for optimizing continuous asset allocations. The agent maximizes a wealth-accumulation objective $R = \frac{1}{t_f} \sum r_t$, where r_t denotes the log-period return net of transaction costs. Transaction-cost effects are embedded

within the simulated environment reward signal, while the policy network uses portfolio-vector memory (PVM) of prior weights to learn to avoid excessive rebalancing. The framework outputs continuous simplex-constrained portfolio weights, removing the requirement for discrete buy-sell trading actions. But single-agent models often break down when market regimes shift. To mitigate this, one ensemble approach from Yang et al. (2020a) selects among PPO, A2C, and DDPG agents on a quarterly cadence. Agent selection is driven by Sharpe ratios computed over a held-out 3-month validation window, and a separate turbulence-index mechanism triggers position liquidation for risk-off halts regardless of which agent is active. Beyond heuristic maximization of risk-adjusted returns, a theoretical line of risk-sensitive RL explicitly incorporates risk-aversion into the stochastic control / MDP formulation. Lleo and Runggaldier (2026) develop a framework for benchmark-tracking investment management built upon free-energy–entropy duality. The risk-sensitive objective applies an exponential utility transformation to the terminal log excess return relative to the benchmark, penalizing unfavorable tail outcomes; this makes it naturally suited for institutional investors such as pension funds or insurance companies focused on matching asset growth against future liability streams. A core theoretical result recasts this non-standard risk-sensitive optimization as a min-max saddle-point stochastic-differential game: the portfolio-manager player minimizes the objective, while a virtual adversarial control induces an equivalent change-of-probability-measure via Girsanov’s theorem to capture risk-sensitive distributional distortions. Leveraging this structure, the authors propose a partially model-based actor-critic algorithm: the state-value critic takes a quadratic-affine form, and both the portfolio and adversarial actors are parameterized as affine mappings of the state. Training uses martingale-residual-based temporal-difference updates under continuous-time q-learning. Note that this is a proof-of-concept study validated on simulated model-generated trajectories rather than real-market back-tests. The approach unites the mathematical structure of classical stochastic optimal control with the data-driven adaptability of reinforcement learning. After training, the base agent weights are frozen; a CNN-based fusion module, trained via supervised learning rather than reinforcement learning, merges their portfolio-weight outputs to produce the final allocation. Note that RA-DRL does not implement explicit regime-detection or regime-switching mechanisms.

Because raw OHLC price-return observations yield noisy state representations for reinforcement-learning agents, Yashaswi (2021) introduce the LFSS preprocessing module. While LFSS supports multiple latent-feature extractors including ZoomSVD and cRBM, its best-performing instantiation combines Kalman filtering for time-series denoising with a CNN-autoencoder for dimensionality reduction to construct latent state inputs, which empirically improves downstream portfolio-optimization performance. Note that the paper does not provide direct quantitative metrics measuring input-space stability; improvements are observed in out-of-sample trading metrics. Motivated by more realistic simulation conditions, Li et al. (2024b) extend the existing EIIE-based DRL portfolio framework, adding support for transaction-cost simulation and short-selling via negative weights (without modelling short-borrowing costs). Within their simulated environment, they benchmark PGAC, PPO and Evolution Strategies. Note that this remains a purely simulation-based study and does not implement full strict institutional trading rules or real-world deployment pipelines. Adopting conceptual sim-to-real design principles from robotics, Ndikum and Ndikum (2024) present AONS (AlphaOptimizerNet Simplified), targeting long-horizon, low-frequency asset-allocation tasks. Financial constraints and trading-fee dynamics are implemented within their simulation environment rather than hard-coded inside the policy network. While the work advocates practices to mitigate over-fitting and stresses the importance of accounting for real-world frictions, all evaluations are confined to historical simulation; effectiveness under true real-world market-impact conditions is not empirically verified. Sood et al. (2026) provides empirical evidence supporting this observation: PPO agents trained on a differential-Sharpe-ratio reward produce inherently lower-turnover allocations and outperform Ledoit-Wolf-shrinkage-augmented MVO in out-of-sample back-tests on sector indices.

To mitigate over-concentration common in unconstrained RL portfolio agents, Kang and Tian (2025) introduce RL-BHRP. They adopt a factorised softmax policy that simultaneously outputs sector-level budgets and intra-sector asset weights, while embedding a soft hierarchical risk-parity dispersion penalty within the reward function to discourage uneven risk contributions and preserve institution-style risk diversification. At the frontier of parameter efficiency, Gurgul and Lessmann (2026) replace classical deep-neural-network actor-critic approximators with parameterized quantum circuits. Adopting a quantum DDPG design, they achieve competitive risk-adjusted returns using roughly 2600-fold fewer trainable parameters relative to their

largest 160k-parameter classical DDPG baseline. Kashif and Łšlepaczuk (2026) implement a full MDP formulation for diversified multi-market equity portfolio management. Its state space combines asset-specific technical features and aggregate global-market indicators; actions are portfolio weights sampled from a Dirichlet distribution under budget constraints. The reward supports two variants: net log-return or benchmark-excess return, augmented by turnover and Herfindahl-Hirschman-based concentration penalties to promote diversification. The work puts forward two core innovations: a hierarchical Dirichlet policy that separates equity-cash allocation from subsequent stock-weight assignment, and an adaptive-retraining walk-forward scheme that triggers retraining only when validation-set Sharpe drops below a dynamic threshold, reconciling adaptability to market non-stationarity with computational cost. Empirical comparisons between LSTM and Transformer temporal encoders offer exploratory evidence regarding the utility of global-attention-based sequence modelling. Beyond single-objective reward design, Dispoto et al. (2026) formulate ESG-aware portfolio optimization as a Multi-Objective Reinforcement Learning problem that simultaneously incorporates ratings from three distinct ESG agencies, pairing the agent with a Gaussian-process preference-elicitation framework that infers latent utility functions from pairwise portfolio comparisons; LLM personas simulating portfolio managers in different regions reveal that regional context shifts the derived preference weights, aligning multi-objective trading with heterogeneous sustainability mandates. In the quantum direction, Hung et al. (2026) (Titans-QFWP) integrate a Quantum Fast Weight Programmer with Titans-style memory components (Persistence, Surprise, and Forgetting) inside an enhanced A3C framework, with Hungarian-aligned K-means clustering of market states and scaled log-return rewards; under an equal-parameter-count benchmark on 468 S&P 500 stocks the hybrid agent attains strong median annualized return, Calmar, and Information Ratio, with quantum gating reshaping the roles of the memory components, Persistence supporting drawdown control and Surprise contributing return generation. Market neutrality can be enforced structurally rather than post hoc: Belyakov (2026) (AlphaZeroBeta) train a CNN-GRU policy with Recurrent PPO under a composite reward balancing risk-adjusted excess return, benchmark correlation, and transaction costs, and hard-enforce dollar neutrality at each rebalance through a zero-sum projection; in backtests covering 2014-2024 across seven equity indices the model achieves higher Sharpe ratios than the baselines while maintaining near-zero benchmark correlations and competitive drawdowns.

LLM Methods. This line of work uses large language models to bring cognitive reasoning into portfolio construction: the LLM reads news, analyzes fundamentals, and participates in allocation decisions alongside quantitative methods, adding semantic understanding of company-specific events that number-only models cannot capture. The most recent evolutionary step combines reinforcement learning’s sequential framework with Large Language Models’ semantic reasoning capabilities. These systems introduce cognitive decision-making capabilities that go beyond purely numerical optimization.

RL, however, treats allocation as a numerical policy search that does not naturally incorporate macroeconomic context. LLM-based systems address this limitation by adding semantic reasoning about macro regimes and asset-specific events on top of the RL decision loop, orchestrating the allocation process at a higher level of abstraction.

1) Non-End-to-End Orchestration and Meta-Optimization.

Here, the LLM acts as a high-level cognitive parser. Instead of executing trades directly, it feeds structured inputs into classical solvers or RL agents. A major use case for this is semantic view generation. The Black-Litterman optimization workflow traditionally depends on subjectively specified human-provided investor views. Mantshimuli and Muteba Mwamba (2025) and Lee et al. (2025b) automate the view-generation step, though full end-to-end automation remains absent given required model-selection and hyper-parameter tuning. While Mantshimuli and Muteba Mwamba (2025) derive views from news-sentiment signals aggregated across multiple finance-focused LLMs, Lee et al. (2025b) instead use repeated numerical return predictions from LLMs, whose sample-estimate variance populates the Black-Litterman confidence matrix. Both approaches bypass manual formulation of investor views, mitigating cognitive biases originating from human expert judgment and enabling scalable quantitative view construction, even though LLM-specific output biases persist. Similarly, Anic et al. (2025) enhance traditional systematic investing via prompt-engineered ChatGPT-4o-mini evaluations of firm-specific news. At portfolio-formation time, the model produces continuous scores assessing the likelihood of momentum continuation for stocks within the momentum candidate

universe. Translating these news-derived assessments into selection and weight-tilting signals, they augment pure price-based momentum strategies with forward-looking news-driven context. Moving beyond general news analysis to highly specialized event-driven trading, Jajal et al. (2026) develop an LLM forecasting system for merger arbitrage. Unlike most LLM-based judgmental-forecasting research relying on short-form inputs, this work targets a long-context-heavy task where hundreds of pages of technical financial and legal filings serve as source material (pre-processed via expert-designed ReAct agents rather than fed raw to the model) to predict M&A deal resolutions. It demonstrates that LLMs can deliver competitive performance within this specialised financial workflow when supported by domain-expert pipeline design and hindsight-guided process-supervised fine-tuning.

Moving past classical math solvers, researchers increasingly use LLMs as specialized cognitive modules inside larger RL pipelines. To handle heterogeneous, multi-modal inputs, Coriat and Benhamou (2025) (HARLF) build a hierarchical meta-agent architecture. Rather than using a language-model to directly allocate assets, HARLF employs FinBERT exclusively for lightweight sentiment-score extraction. Base reinforcement-learning agents consume sentiment signals and price-based indicators separately to produce candidate portfolio weights; meta-agents aggregate outputs within each modality, and an RL super-agent dynamically combines these modality-specific weight recommendations, implementing a multi-modal fusion mechanism for portfolio construction. Results are demonstrated on a single out-of-sample time split, without walk-forward validation to confirm long-term resilience. Building on the separation of LLM-based reasoning and high-dimensional econometric portfolio estimation, Caner et al. (2026) propose a two-stage agentic-AI screening-then-optimisation pipeline. An LLM-Strategy agent handles slow-moving fundamental-driven stock assessment, complemented by a FinBERT agent capturing fast-evolving news-sentiment signals. Applying an intersection-first consensus rule to filter the S&P500 constituent universe, the candidate stock output is passed to advanced precision-matrix estimators including POET and nodewise regression to solve for global-minimum-variance, mean-variance, or maximum-Sharpe-ratio portfolios. The paper introduces the theoretical construct of *sensible screening*, under whose mathematical assumptions the squared Sharpe ratio of the screened portfolio retains statistical consistency; importantly, the agent-based intersection heuristic does not constitute a proof that sensible-screening conditions hold empirically for real-world data.

Taking this abstraction a step further, a line of meta-optimization moves LLMs out of the operational trading loop. Instead of generating trading signals or investor views, Yuksel (2025) (AlphaLoss) employ LLMs to generate, back-test evaluate, and iteratively evolve mathematical formulations for multi-objective portfolio loss functions. This meta-level search procedure reduces reliance on manually assembled objective specifications, where downstream numerical solvers compute portfolio weights given the offline-evolved, component-wise interpretable loss formulation. While the evolutionary search leverages human-defined financial building-blocks provided within few-shot prompts, it explores novel combinations of risk-return terms beyond hand-tuned baseline designs. Reframing what the LLM optimizes, Zhang (2026) (OOQI) replace scalar objective maximization with specification satisfaction: instead of searching for the single highest-performing strategy, the framework first defines the strategy’s identity constraints (near-zero style exposures, resilience in down markets, bounded turnover, sufficient capacity) and then searches for and certifies pipelines that satisfy the specification, so that optimization is anchored to investor requirements rather than to one backtest metric. A retrieval-grounded expert selector is proposed by Asadi and Safabakhsh (2026): a dual-stream variational autoencoder encodes asset-level and market-wide information, a retrieval-based knowledge base stores historical situations together with each expert’s realized performance, and an instruction-tuned LLM reasons over the retrieved evidence to choose the most appropriate portfolio-management expert rather than generating weights directly; a monotonicity property shows that adding a locally superior expert cannot degrade the selector, and experiments across cryptocurrency, stock, and foreign-exchange markets report the highest cumulative return and Sharpe ratio among the evaluated selection strategies. Extending this meta-level abstraction from objective design to solver configuration, Chai et al. (2026) (LLMDE) use a prompt-engineered LLM to drive differential evolution for constrained portfolio optimization: guided by optimization feedback, the LLM dynamically selects mutation strategies and configures control parameters, reducing reliance on handcrafted hyperparameter design; the resulting evolutionary search solves a Conditional Value-at-Risk (CVaR) portfolio problem over a stock universe selected via factor analysis and K-means clustering, subject to budget and minimum expected return constraints, achieving competitive performance on the CEC2022 benchmark suite.

2) End-to-End Agentic Architectures. Full-stack multi-agent frameworks now automate the entire quantitative trading pipeline from signal discovery to execution-level strategy configuration. Yuan et al. (2026) (AlphaCrafter) implements a closed-loop system with three specialized LLM agents that continuously adapt to non-stationary market dynamics rather than relying on frozen factor designs. The Miner agent maintains and expands a factor library $\mathcal{Z}_t$ using LLMs to search for new alpha factors, validating candidates through IC, ICIR, turnover, and decay-profile metrics while periodically pruning underperforming factors. The Screener agent acts as a bridge between raw factors and estimated market conditions $\hat{\mathcal{R}}_t$, assessing trend direction, volatility levels, and correlation structure to select and weight a diversified factor ensemble $\mathcal{E}_t$ matched to the current regime; no gradient-based retraining is performed across agents. Finally, the Trader agent transforms the ensemble signal into an executable trading strategy π_t , applying pre-specified rules for stock ranking, long/short positioning and exposure-level logic, while enforcing constraints on gross exposure and net-position bias. This multi-stage decomposition draws an analogy to the division of labour within institutional quantitative-research teams.

The current frontier in portfolio management treats capital allocation as a fully autonomous multi-agent society, closing the loop between qualitative macro-reasoning and quantitative execution. Pursuing hyper-personalised portfolio advice, Li et al. (2025) (L-PPR) feed structured user-risk-preference representations, inferred from multi-turn investor dialogue, directly into a PPO agent’s state space, bridging human-centric financial advisory and deep-reinforcement-learning portfolio decision-making. Addressing the traditional dichotomy between institutional execution-focused systems and simple rule-based retail tools, Pishevvar (2026) introduces a three-phase foundation model for tax-aware personalized portfolio management. The framework escapes the ticker lock model that fails to generalize to new assets by constructing a ticker-agnostic CrossAssetEncoder with three components fused via gated and additive mechanisms: a MarketEncoder transformer analyzing price-derived features, a frozen Chronos-T5-Small encoder leveraging generic temporal patterns, and a MetadataEncoder using 50-dimensional vectors for industry, market cap, and institutional activity rather than unique stock IDs. An inter-ticker contrastive loss mitigates representation collapse induced by the Chronos encoder. The second phase uses a Mixture-of-Experts architecture with four specialized heads (momentum, growth, defensive, tax-aware) trained via a six-stage curriculum; diagonal shortcut initialization is applied to the intention router to ease multi-objective optimization, optimizing a goal-conditional reward that weights alpha, turnover penalty, and cash redeployment bonus by active objectives. Finally, lightweight personalization uses LoRA adaptation of just 76 parameters in the intention router, inferring user preferences from actual brokerage history rather than stated risk questionnaires. Note that empirical evaluation of this personalization component on real-world brokerage data is left to future work. This three-stage architecture enables generalization to any publicly traded asset without retraining while adapting to individual tax situations.

Beyond macro routing, certain end-to-end generative architectures bypass traditional numerical covariance matrices, relying on semantic reasoning for stock picking. Voronina et al. (2025) validated generative LLM-based stock selection by evaluating models from multiple AI vendors. Their experiments demonstrate that LLMs deliver competitive performance under stable market conditions, yet encounter difficulties during abrupt regime transitions, a setting in which classical covariance-based optimizers also degrade. Notably, their best-performing strategy hybridizes LLM stock screening with conventional mean-variance optimization rather than fully discarding numerical risk estimates.

6.3 Multi-Period Portfolio Construction

Multi-period portfolio optimization reframes the allocation problem as a sequential Markov Decision Process over a horizon H , choosing a sequence of weights $(W_t, W_{t+1}, \dots, W_{t+H})$ that maximizes the expected cumulative utility subject to path-dependent trading frictions and regime shifts:

$$(W_t^*, \dots, W_{t+H}^*) = \arg \max_{(W_\tau)_{\tau=t}^{t+H}} \mathbb{E} \left[\sum_{\tau=t}^{t+H} \mathcal{U}(W_\tau \mid S_{pure,\tau}, \Sigma_\tau) - TC(W_\tau - W_{\tau-1}) \right] \quad (21)$$

Unlike single-period formulations, multi-period methods explicitly account for the path-dependent consequences of today’s weights on tomorrow’s feasible set. The field is organized along three directions of

increasing integration: constrained optimization that hardcodes realistic trading constraints into multi-stage convex programs, neural augmentation that uses deep learning to process complex inputs and forecast future states, and end-to-end integration that merges prediction and optimization through differentiable optimization layers.

Foundational Frameworks. The foundational framework of multi-period portfolio optimization traces back to Dantzig and Infanger (1993), who showed that large-scale multi-stage stochastic programming under uncertainty is computable via Benders decomposition combined with importance sampling, establishing the algorithmic baseline for subsequent work. Zhou and Li (2000) derived the exact analytical solution for the mean-variance efficient frontier using stochastic linear-quadratic control theory in continuous time, providing the strict mathematical framework for continuous dynamic decision-making. Subsequent work develops multi-period portfolio optimization along three directions of increasing integration. First, constrained optimization approaches hardcode realistic trading constraints into multi-stage convex programs. Second, neural augmentation uses deep learning to process complex inputs and forecast future states. Third, end-to-end integration fully merges prediction and optimization through differentiable optimization layers.

Constrained Optimization. This line of work builds multi-period convex optimization models with realistic constraints like transaction costs and position limits, extending the classical single-period framework to handle the path-dependent consequences of trading over multiple periods. By the mid-to-late 2010s, as market dynamics grew more complex, the priority shifted to hardcoding realistic constraints, namely transaction costs and holding limits, directly into these models. Boyd et al. (2017) advanced this line of work with a convex optimization setup based on Model Predictive Control. By adopting a receding-horizon optimization framework, the approach circumvents the curse-of-dimensionality from full multi-stage dynamic programming while maintaining feasible computation. Expanding upon the MPC framework proposed by Boyd et al. (2017), Li et al. (2022) extended the setup to support risk-parity objectives. To tackle the inherent non-convexity of multi-period risk parity, they developed highly-efficient successive convex-programming solvers, and further incorporated hidden-Markov-model-based regime-switching forecasts to supply time-varying return and covariance inputs for the MPC planner. Building upon this MPC framework, Oprisor and Kwon (2020) employed hidden-Markov models to dynamically adjust confidence levels for subjective investment views under shifting market regimes. They further integrated the Black-Litterman model to encode investor views, rendering portfolio allocations more robust against erroneous subjective views. Extending the constrained formulation to tail-risk specifications, Hu et al. (2026) study continuous-time dynamic portfolio optimization under a Conditional Value-at-Risk constraint on terminal loss: exploiting the auxiliary-threshold representation of CVaR, they establish the existence of an optimal strategy and strong duality without requiring market completeness, and solve the problem with a nested bisection-golden-search algorithm over the threshold and Lagrangian multiplier; when the constraint binds, the optimal strategy reallocates asymmetrically across states rather than de-risking uniformly.

Neural Augmentation. This line of work adds neural networks to multi-period portfolio optimization, using deep learning to handle complex macro-fundamental data, forecast future states, or generate forward-looking simulation inputs that feed into the optimization. Recent work has largely shifted this space to deep learning and end-to-end architectures. Some systems use machine learning to process complex macro-fundamental data directly. For example, Yu and Chang (2020) built the EFPM forecasting framework to feed forward-looking simulation inputs directly into optimization models. To handle the scalability challenges of end-to-end deep reinforcement learning, Cui et al. (2024) proposed a hyper-heuristic learning architecture that avoids the action-space curse-of-dimensionality arising from large-scale asset pools by selecting among pre-defined underlying trading policies, rather than directly outputting asset weights. Physics-informed learning provides another route to multi-period allocation: Halperin and Itkin (2026) (SciPhyRL) formulate continuous-time portfolio optimization over an extended state space that includes explicit cumulative costs, and reduce the resulting Hamilton-Jacobi-Bellman equation to a pathwise form that is solved directly from offline historical data with physics-informed neural networks in a single sweep; on a 14-asset ETF universe, the learned policy yields substantial out-of-sample Sharpe ratio improvements over static and myopic baselines under a microstructure-grounded quadratic price-impact model. In the same continuous-time setting, Jeon et al. (2026) scale Pontryagin-guided policy optimization to constrained multi-asset choice:

after training, a pointwise-feasible neural actor’s latent outputs are frozen and first- and second-order adjoints are harvested from an open-loop backpropagation-through-time graph, so that deployment recovers quadratic-affine portfolio blocks exactly by a quadratic program while a log barrier approximates more general regular KKT branches; recovery reduces the residual Pontryagin/KKT error left by finite-budget direct policy optimization, including under a state-dependent consumption cap and with up to 100 risky assets.

End-to-End Integration. This line of work merges prediction and optimization into a single trainable system: gradients flow directly from portfolio performance back through the optimization layer to the prediction model, so everything is trained end-to-end on portfolio performance. Linghu et al. (2025) proposed the IPMO model, which backpropagates gradients through a differentiable optimization layer using a decision loss derived from actual investment performance. This aligns the objective of the upstream prediction model with the downstream optimizer, mitigating the predict-then-optimize gap inherent in conventional two-stage pipelines.

7 Datasets and Evaluation

This section surveys the data and evaluation infrastructure supporting quantitative AI research. We organize datasets by asset class and data modality, and evaluation metrics by pipeline stage, covering Factor Mining, Factor Combination, Risk Control, and Portfolio Management.

7.1 Datasets and Data Sources

Quantitative AI research consumes multi-modal financial data. To capture the full landscape from complementary angles, we organize datasets along two orthogonal axes: by *asset class* (Table 1) and by *data modality* (Table 2). Rather than providing an exhaustive inventory of financial data providers, we summarize representative datasets and data sources that are explicitly used in the reviewed literature and are commonly adopted in academic studies. Tables 1 and 2 organize the same set of representative sources from the perspectives of asset class and data modality, respectively.

By Asset Class. Organized by asset class, the literature spans four primary markets. **Equity** research is supported by daily price-volume data from institutional pipelines (Yahoo Finance / yfinance; Qlib market data (Yang et al., 2020b); FinRL market data pipelines (Liu et al., 2020; Yang et al., 2020a)), high-frequency limit-order-book snapshots (FI-2010 (Ntakaris et al., 2018)), financial text (Financial PhraseBank; FinBen (Xie et al., 2024); SEC EDGAR), and knowledge graphs (FinKG (Kertkeidkachorn et al., 2023)). **Crypto** studies draw on public cryptocurrency market data used in reinforcement-learning and agent-based trading (Akbar, 2025; Ahmed et al., 2025) and on Coinbase Level-3 order-book and trade data (Sadighian, 2019). **Forex** research relies on exchange-rate time-series data used in technical-trading and reinforcement-learning studies (Neely et al., 1997; Huang, 2018). **Multi-asset** studies combine cross-asset price-volume and macroeconomic data for portfolio-allocation research.

By Data Modality. Organized by data modality, the same datasets regroup into two top-level families. **Price-Volume Data** underpins all backtesting and signal generation and splits into four sub-categories by sampling frequency: tick-level limit-order-book data (FI-2010 (Ntakaris et al., 2018); Coinbase Level-3 order-book and trade data (Sadighian, 2019)), intraday bars (Qlib intraday market data (Yang et al., 2020b); BaoStock-based intraday data), daily bars (Yahoo Finance / yfinance; Qlib market data (Yang et al., 2020b); FinRL market data pipelines (Liu et al., 2020; Yang et al., 2020a)), and weekly/monthly bars (low-frequency price-volume data used in longer-horizon factor and portfolio studies). **Alternative Data** subsumes all non-price-volume inputs and splits into four sub-categories: *text* (Financial PhraseBank; FinBen (Xie et al., 2024); SEC EDGAR) fuels LLM-driven reasoning; *knowledge graphs* (FinKG (Kertkeidkachorn et al., 2023)) map corporate relations and supply chains; *macroeconomic indicators* (public macroeconomic and market indicators) support regime-aware and asset-allocation studies; and *image data* (NOAA DMSP-OLS Nighttime Lights (Henderson et al., 2012); satellite imagery (Katona et al., 2024)) provides physical-asset and activity signals for economic and earnings prediction.

Table 1: Representative Datasets and Data Sources Organized by Asset Class

Asset Class	Sub-category	Datasets / Sources
Equity	Price-volume	Yahoo Finance / yfinance; Qlib market data (Yang et al., 2020b); FinRL market data pipelines (Liu et al., 2020; Yang et al., 2020a)
	Limit order book	FI-2010 (Ntakaris et al., 2018)
	Financial text	Financial PhraseBank; FinBen (Xie et al., 2024); SEC EDGAR
	Knowledge graph	FinKG (Kertkeidkachorn et al., 2023)
Crypto	Price-volume	Public cryptocurrency market data used in reinforcement-learning and agent-based trading studies (Akbar, 2025; Ahmed et al., 2025)
	Limit order book	Coinbase Level-3 order-book and trade data (Sadighian, 2019)
Forex	Exchange-rate data	Foreign-exchange time-series data used in technical-trading and reinforcement-learning studies (Neely et al., 1997; Huang, 2018)
Multi-asset	Cross-asset market data	Multi-asset price-volume and macroeconomic data used in portfolio-allocation studies

Table 2: Representative Datasets and Data Sources Organized by Data Modality

Modality	Sub-category	Datasets / Sources
Price-Volume	Tick-level / Limit Order Book	FI-2010 (Ntakaris et al., 2018); Coinbase Level-3 order-book and trade data (Sadighian, 2019)
	Intraday bars	Qlib intraday market data (Yang et al., 2020b); BaoStock-based intraday data
	Daily bars	Yahoo Finance / yfinance; Qlib market data (Yang et al., 2020b); FinRL market data pipelines (Liu et al., 2020; Yang et al., 2020a)
	Weekly / Monthly bars	Low-frequency price-volume data used in longer-horizon factor and portfolio studies
Alternative	Text	Financial PhraseBank; FinBen (Xie et al., 2024); SEC EDGAR
	Knowledge graph	FinKG (Kertkeidkachorn et al., 2023)
	Macroeconomic indicators	Public macroeconomic and market indicators used in regime-aware and asset-allocation studies
	Image	NOAA DMSP-OLS Nighttime Lights (Henderson et al., 2012); satellite imagery (Katona et al., 2024)

7.2 Evaluation

Quantitative investment requires sequential decision-making under realistic market constraints, so evaluation is typically conducted through historical backtesting on platforms such as Yang et al. (2020b) and Liu et al. (2020). To enable cross-paper comparison under heterogeneous reporting, we adopt a simplified evaluation tuple:

$$\mathcal{E} = (\mathcal{S}, \mathcal{O}, \mathcal{M}, \mathcal{R}) \quad (22)$$

where $\mathcal{S}$ denotes the data scope (dataset, market, time horizon), $\mathcal{O}$ the modeling objective, $\mathcal{M}$ the methodology, and $\mathcal{R}$ the reported evaluation metrics.

Standardized Metrics. The literature relies on a standardized set of metrics aligned with the four pipeline stages:

- **Information Coefficient (IC) and Rank IC:** cross-sectional correlation between predicted factor scores and forward returns, primarily used in *Factor Mining*.
- **Cumulative Return and Annualized Return:** absolute profitability of trading signals or portfolios over the backtesting period.
- **Maximum Drawdown (MDD):** maximum peak-to-trough loss, heavily used to evaluate the *Risk Control* module.
- **Sharpe Ratio and Information Ratio (IR):** excess return per unit of risk or tracking error, primary metrics for *Portfolio Management*.

Novel Evaluation Metrics. Beyond the standardized metrics above, recent work has proposed evaluation protocols tailored to specific pipeline stages. Cui et al. (2021) (AlphaEvolve) introduces a fitness-function design framework for factor mining that jointly optimizes IC, factor decorrelation, and search efficiency, providing a multi-objective lens to assess mined alphas beyond raw predictive correlation. Ding et al. (2025) (AlphaEval) further systematizes factor-level evaluation by decomposing alpha quality into predictive strength, stability, orthogonality, and turnover dimensions, enabling fine-grained diagnosis of why certain mined factors generalize while others decay. These novel metrics complement the standardized set and are particularly relevant when evaluating LLM-driven factor mining systems, whose outputs require more nuanced quality assessment than single-number IC.

Evaluation Frameworks. As LLMs and autonomous agents proliferate in quantitative finance, the lack of standardized evaluation protocols has hindered fair cross-study comparison. Recent literature addresses this fragmentation with comprehensive frameworks that progress from offline historical benchmarks to dynamic, real-time simulation environments.

The Quantitative Finance Benchmark qua (2026) (QFBench) is a foundational, multi-modal testing environment designed to assess the cognitive reasoning, data processing, and decision-making capabilities of AI models in complex financial contexts. Unlike traditional single-task datasets, QFBench evaluates financial agents across a continuous workflow that mirrors real-world quantitative research, standardizing the evaluation pipeline into several core dimensions: numerical reasoning (processing historical OHLCV data for signal prediction and factor mining), semantic understanding (sentiment quantification and event extraction from financial news and corporate reports), and strategic execution (portfolio optimization and risk management). By offering standardized testing environments with strict zero-shot and few-shot evaluation protocols, QFBench mitigates data leakage and overfitting, ensuring that reported performance reflects the model’s true generalization capabilities on historical data.

While offline benchmarks establish a rigorous foundation, they inherently rely on static backtesting, which is prone to information leakage and fails to capture the true uncertainty and feedback delays of live markets. To bridge the gap between static reasoning evaluation and real-world sequential decision-making, Yu et al. (2025a) (LiveTradeBench) introduces a live trading environment built upon three core design principles:

(1) *Live Data Streaming*: by continuously ingesting real-time market prices and news, the framework completely bypasses historical offline backtesting, strictly preventing information leakage while forcing the agent to operate under authentic real-time market uncertainty; (2) *Portfolio Management Abstraction*: shifting away from simplified single-asset discrete actions (buy, hold, sell), the benchmark utilizes a multi-asset capital allocation abstraction where agents output continuous percentage weightings, integrating complex risk management and cross-asset reasoning into their decision loop; (3) *Multi-Market Evaluation*: to test generalizability, the framework evaluates agents across structurally distinct environments, comparing performance in the U.S. stock market (which rewards long-horizon analysis and sector diversification) against Polymarket prediction markets (which are sentiment-driven, highly volatile, and demand rapid, event-driven adaptation). By modeling the trading process as a continuous loop of observing live portfolios, ingesting news, and rebalancing assets, LiveTradeBench empirically exposes a critical gap: high performance on general LLM reasoning benchmarks does not necessarily translate to superior financial decision-making under uncertainty. Complementing live-market simulation with behavioral observability, Li et al. (2026a) (NextFund) provide an evaluation platform that couples time-consistent market access, coordinated multi-agent analysis, and persistent logging of the full decision path from observation to trade, letting users inspect equity curves and drill from leaderboard outcomes down to individual justifications; presented on Hong Kong, US, and China A-share markets, the platform illustrates how inspectable decision histories enable fairer benchmarking of agentic portfolio managers. Taking the discovery system itself as the unit of evaluation, Pan et al. (2026b) (Agentic Empirical Asset Pricing, AEAP) define systems that autonomously conduct the scientific discovery process and contribute a reference architecture plus a rigorous evaluation standard for discovered factors, together with a method for out-of-sample backtesting of the discovery process rather than only its outputs; applying the standard to a concrete architecture (SEADS) against five re-implemented baselines on two US equity panels shows that no single metric ranks the systems consistently, motivating evaluation on multiple axes simultaneously.

7.3 State-of-the-Art Performance Comparison

This section provides empirical grounding for the methodologies surveyed in Sections 3, 4, 5, 6, organized to align with our four stage pipeline. Due to the heterogeneous nature of the quantitative pipeline, we evaluate performance separately within each stage using metrics appropriate to that stage’s outputs.

7.3.1 Evaluation on Factor Mining

Following the dichotomy adopted in Section 3, we split the evaluation of alpha mining into two parallel tracks aligned with the chapter’s two subsections: *Formulaic Factor/Code Generation* (Section 3.2) evaluates methods that output explicit symbolic factor expressions, including GP-based, RL-based, LLM-based, and neural-network-based formulaic search; and *Signal Generation* (Section 3.3) evaluates methods that directly output predictive signals without explicit symbolic forms, including time-series models, RL-based signal agents, and agentic systems. The primary metrics for both tracks are IC, Rank IC, and IR for formulaic factors, and MSE/F1 for signal prediction.

Formulaic Factor/Code Generation. Formulaic and code-based factor generation methods output explicit, inspectable factor expressions from raw market data, corresponding to the methods surveyed in Section 3.2. Table 3 organizes these methods by paradigm (GP-based, RL-based, LLM-based, Neural Network-based), covering the full evolutionary trajectory from early GP symbolic search to modern LLM-driven and Neural Network-based discovery.

Table 3: State-of-the-Art Evaluation in Factor Generation

Model	Scope (S)	Method (M)	Metrics (R)	Improvement	Model	Scope (S)	Method (M)	Metrics (R)	Improvement
Neely et al. (1997)	FX markets (USD/DEM, USD/JPY, etc., 1974–1995; OOS: 1981–1995)	GP-based	AR=6.05%, Sharpe=0.50 (\$/DM); avg AR=2.87% (6 FX)	vs. Buy-and-hold: avg AR 0.6% $\rightarrow$ 2.87%	Allen and Karjalainen (1999)	S&P 500 index (1928–1995)	GP-based	AR=-1.02% (after 0.25% TC); Vol=10.0%	fails to outperform buy-and-hold; predictive power vanishes with 1-day delay
Potvin et al. (2004)	TSE 300 (14 Canadian stocks, 1992–2000)	GP-based	AR=-4.85% / -3.59% (short/long training)	positive in declining/flat markets (9/14 stocks profitable)	Becker et al. (2006)	S&P 500 (~350 stocks, 1990–2005), 65 features	GP-based	AR $\approx$ 19.4% (vs. 14.9%); 3M LS return=4.03% (IR=1.50)	AR +4.5pp; IR +53% (1.50 vs. 0.98) vs. linear model
Lohpetch and Corne (2010)	S&P 500 (1960–2008), multi-frequency	GP-based	consistent outperformance at monthly frequency	beats buy-and-hold monthly; win rate 0/10 at daily	Sefiane and Benbouziane (2012)	Algerian stock market portfolio selection	GP-based	Arithmetic crossover smoother than discrete splicing	Improved stability in evolutionary optimization via continuous interpolation
McDermott et al. (2014)	5-min GOLD, GBP–USD, S&P 500 (418 GSGP) OOS points)	GP-based	Test classification acc. GP/GSGP = 0.59/0.59 (GBP–USD), 0.80/0.65 (S&P 500)	Only GSGP beats both ARIMA & buy-and-hold at 50/100/418 steps for GBP–USD	Kampuridis and Otero (2017)	255 FX datasets, tick & 10-min (2013–2014)	GP-based	GBP/JPY tick: DC+GA mean daily return=0.0730%, MDD=0.03789%; 10-min mean return=0.01046%	Mean return +7.8% vs MDC ^{EV} O (0.0677% $\rightarrow$ 0.0730%); 10-min SDC ^{EV} O improves on EDDIE
Chen and Wang (2015)	Multi-asset allocation under risk constraints	GP-based	Mean-CVaR embedded in GNP transitions	Safer capital allocation via structured state transitions under tail risk	Yang et al. (2019)	Regime-switching markets	GP-based	Multi-subpopulation co-evolution	Adapts factor populations under regime switching; extended single-population GP
Zhang et al. (2020)	CSI 800 (2010–2019), OHLCV features (AutoAlpha)	GP-based	AR=90.0% (h=1), 28.0% (h=5); Sharpe=3.39 / 1.20	IC: top50 7.50% vs. gplearn 6.10%, Alpha101 1.02%	Chen et al. (2021)	CSI 300 constituents (2013–2020), price-volume features (GEP-Fin)	GP-based	AR=74.07%, Sharpe=2.33, IR=2.50, MDD=15.99%	AR +49.5pp vs. CSI300 (24.53% $\rightarrow$ 74.07%)
Cui et al. (2021)	NASDAQ (2013–2017), 126 stocks, engineered features (AlphaEvo)	GP-based	IC=0.0674, Sharpe=21.32	vs. GA: Sharpe +60% (13.03 $\rightarrow$ 21.32), IC +37%; low alpha corr. (< 15%)	Li et al. (2024a)	China A-share (~4500 stocks, 2014–2022) (SGP-LSTM)	GP-based	RankIC=9.24%, ICIR=7.24; AR=31.00% (CSI300), 24.48% (CSI500)	vs. LSTM: excess return $\sim$ 6% $\rightarrow$ 31% (+ $\sim$ 22pp); IC +1128%, ICIR +5360%
Ren et al. (2024b)	China A-share (2020–2024) (WS-GP)	GP-based	RankIC=7.8%; AR > 50%, Sharpe > 1.0 (30-stock portfolio)	+1% IC / RankIC vs. traditional GP; Sharpe $\sim$ 0.22 $\rightarrow$ > 1.0	Yang et al. (2026)	CSI300 & S&P500 (2010–2024), OHLCV features (AlphaCFG)	GP-based	RankIC=0.0865, ICIR=0.6036; Sharpe=0.65, MDD=-29.63%	vs. AlphaGen: 0.0837 $\rightarrow$ 0.0865; Sharpe $\sim$ 0.17 $\rightarrow$ 0.65 ($\sim$ 4 $\times$)
Guo et al. (2026b)	CSI300/500/1000 (2010–2025), price+vol+fundamental features (AlphaFormer)	GP-based	ICIR=0.3902 (vs. 0.3582); RICIR=0.4694; AR=7.50%, Sharpe=0.44	vs. AlphaSAGE: ICIR +8%; Sharpe +10%; DAG-based reuse avoids redundant recomputation	Yu et al. (2023)	CSI300, CSI500 (2009–2021), OHLCV + vwap (AlphaGen)	RL-based	IC=0.0725; RankIC=0.0806	>3.9 $\times$ IC vs GP_filter (0.0183); outperforms all baselines
Li et al. (2023a)	China A-share (2013–2019), 13 features (price, volume, PB, PE) (Style Miner)	RL-based	R ² =27.4%	+10.4% relative R ² vs expert factors (24.8%); constrained VAE+Actor-Critic	Shin et al. (2024)	CSI300 (2009–2021)	RL-based	IC=0.085; RankIC=0.087	+89% IC; +52% RankIC vs baseline; warm-start initialization
Ren et al. (2024a)	CSI300 (bear market incl. 2021–2022) (RiskMiner)	RL-based	IC=0.0645; ICIR=0.5126	+6.7% IC; +27.4% ICIR vs. AlphaGen; risk-seeking MCTS on top τ -quantile	Zhu and Zhu (2025)	CSI300/500/Market + S&P 500 (2010–2022) (AlphaQCM)	RL-based	IC=0.0849 (CSI300); IC=0.0955 (CSI500); IC=0.0916 (Market)	+0.36%/+1.47%/+3.12% IC vs AlphaGen; distributional DQN+IQN
Ding et al. (2025)	A-share (2010–2024) + S&P 500 (2010–2020) (AlphaEval)	RL-based	PPS=0.041; Stability=0.978; Robustness=0.997	Superior cumulative returns vs single-metric (IC); -25% evaluation time	Sun et al. (2024)	CSI300/500 + NASDAQ 100/S&P 500 (2021–2024) (AlphaCon)	RL-based	RankIC=0.0425	+1.80% RankIC vs GP (0.0245); in-context adaptation via Proposer-Refiner
Huang et al. (2024b)	CSI 300/500 + S&P 500 (AlphaFormer)	RL-based	IC=0.0601	+0.82% IC vs best baseline (0.0519); 33% factor count; cross-dataset priors without retraining	Xu et al. (2024)	CSI300, CSI500 (formulaic code generation) (Alpha ²)	RL-based	CSI300 IC=0.0576, RankIC=0.0681; CSI500 IC=0.0612, RankIC=0.0731	IC +15.2% vs AlphaGen (0.0500 $\rightarrow$ 0.0576); mean alpha corr. 0.138 vs AlphaGen 0.376 (more diverse)
Zhang et al. (2026)	Chinese high-frequency & daily ETF (QuantEvo)	RL-based	DirAcc=53.22%; RankIC=0.0586 (high-freq), 0.1923 (daily ETF)	Outperforms LLM-based baselines on all three tasks; DSL ablation: RankIC 0.0586 $\rightarrow$ 0.0002					
Wang et al. (2025)	Intraday China & US stocks data, experiments up to 2024 (Alpha-GPT)	LLM-based	IC=2.23% after human-AI search enhancement	Alpha-GPT factors scored 8.16 vs 6.81 from junior humans; Top 5–10% in HFT competitions	Cheng and Tang (2024)	CRSP U.S. equities (2003–2022)	LLM-based	AR=66.16%, Sharpe=4.45; AR=88.10%, Sharpe=2.47	over 50% factors outperforms human-designed factors
Wang et al. (2024b)	China A-share (500 stocks, 2023) (QuantAgent)	LLM-based	IC=0.04–0.05; Sharpe > 1.2	+40–60% IC vs. LLM-only (IC $\sim$ 0.03); Sharpe 0.8 $\rightarrow$ > 1.2	Yang et al. (2024b)	RD2Bench dataset (Co-STEER)	LLM-based	Avg. correlation = 0.646 (baseline 0.454); 44.5% abs. improvement	AD ² performance +24% using evolving scheduler vs. random

Continued on next page...

Table 3: State-of-the-Art Evaluation in Signal Generation (Continued)

Model	Scope (S)	Method (M)	Metrics (R)	Improvement	Model	Scope (S)	Method (M)	Metrics (R)	Improvement
Li et al. (2024d) (FAMA)	S&P 500 (2015–2022)	LLM-based	RankIC=0.054; RankICIR=0.485; AR=38.4%; Sharpe=667.2%	+0.006 RankIC; +0.106 RankICIR vs. FactorVAE; AR +6.6%, Sharpe +261.1%	Wang et al. (2024d) (GPT-signal)	S&P 500: 93 companies; IT/Health/Energy (2016–2020)	LLM-based	Max corr=0.12 with future 3-month returns	Median adj. R ² +15–20% vs. 10 conventional ratios; two-step high-order non-linear induction
Kou et al. (2024)	SSE50, CSI300, S&P 500 markets, macro data (2019–2024)	LLM-based	SSE50: cum. return 53.17%, Sharpe 0.287; OOS IC=0.047	Outperforms XGBoost (9.53%), FinCon (22.47%), SEP (17.89%); >2× next-best LLM	Tang et al. (2025) (AlphaAgent)	Chinese CSI 500 and U.S. S&P 500 (2021–2024)	LLM-based	AR=11.00% (IR=1.49) CSI500; AR=8.74% (IR=1.05) S&P500	Hit ratio +81% vs. RD-Agent; token consumption -30%; 3-fold constraint framework
Li et al. (2026b) (R&D-Agent-Quant)	CSI 300 (2008–2020), CSI 500/NASDAQ 100 (2008–2025)	LLM-based	IC=0.0532; AR=14.21% on CSI 300	Surpasses alpha 158 (IC=0.0341, AR=5.70%); up to 2× higher returns using 70% fewer factors	Ren et al. (2025) (TreEvo)	CSI300/500 & SPX/DJI (2016–2024)	LLM-based	IC=0.0615 (CSI300); IC=0.0742 (CSI500)	+4.59% vs. QFR; ~20% improvement over LLM-driven EAs; computation reduced 5×–100×
Cao (2025) (Chain-of-Alpha)	CSI 500, CSI 1000 (2010–2025)	LLM-based	RankIC=0.0902; AR=14.71% (CSI 1000)	+10.5% AR vs next-best LLM+MCTS (12.35%); IR 1.40 vs 1.33; parallel optimization chains	Liu et al. (2025) (CogAlpha)	CSI300 (2011–2024)	LLM-based	IC=0.0591; RankIC=0.0814	+97% IC; +156% RankIC vs GPT-OSS-120B; 7-level hierarchical validation
Luo et al. (2025a) (EFS)	FF25/32/49/100, US50, HSI45, CSI300	LLM-based	CW=2434.51 (US50); SR=0.257; MDD=0.454	+50% CW vs ASMCVaR (US50); +80% CW vs ASMCVaR (CSI300); combination-aware generation	Han et al. (2026) (AlphaAgent)	CSI 300 (2016–2025), CSI 500 (2010–2024)	LLM-based	IC=0.1501; AR=27.75%; MDD=7.98%	+0.0535 IC; +12.21% AR; MDD reduced by 4.91% vs AlphaAgent; trajectory-level mutation/crossover
Huang and Fan (2026)	US equities (8,052 stocks, 2004–2024)	LLM-based	AR=59.53%, Sharpe=3.11; robust after 3bps TC; turnover ~110%	+8.69% AR vs traditional LGBM (59.53% vs 50.84%); closed-loop $C_k: H_k \rightarrow F_k \rightarrow M_k \rightarrow D_k$	Lin et al. (2026) (FactorEngine)	CSI300, CSI500 (2008–2024), OHLCV + reports	LLM-based	AR=18.99%; IC=0.0474; Sharpe=1.0093; MDD=12.61%	+126% AR; +58% IC vs Alpha158; LLM macro logic + Bayesian micro tuning
Wang et al. (2026) (FactorMiner)	A-shares (HS300, CSI 500/1000) & Crypto (2024–2025)	LLM-based	IC=8.25%, ICIR=0.77 (CSI 500); Combined IC=15.11%	IC +40%, ICIR +67% vs AlphaAgent (CSI 500); Ralph loop with structured memory	Shi et al. (2026b) (LLM-MTCS)	CSI 300 + CSI 1000 indices (2011–2024)	LLM-based	Avg. RankIC=0.0512–0.0549; IR up to 1.27	Outperforms next-best LLM (CoT) by >20% in RankIC; FSA-directed tree search reduces factor homogeneity
Shi et al. (2026a) (Hubble)	Multi-market equities (2022–2026, DSL-constrained sandbox)	LLM-based	OOS RankIC=0.0242–0.0275, IC=0.0385–0.0391; RankICIR=1.80–2.31 (t=2.17–3.01, p<0.05)	diversity constraints + negative RAG	Weng et al. (2026) (AlphaLogics)	CSI 500 & S&P 500 (2021–2024) test	LLM-based	CSI500 IC=0.0251, IR=1.5266, AR=16.72%, MDD=-5.31%; S&P500 IC=0.0093, IR=1.2658, AR=13.75%	IR +24.8% vs AlphaAgent on CSI500 (1.2230 → 1.5266); +31.1% on S&P500 (0.9653 → 1.2658)
Yu et al. (2026a) (AlphaMemo)	CSI 500 & S&P 500 (2022–2025 test, 20-day protocol)	LLM-based	CSI500 (balanced) Sharpe=0.6109, AR=11.63%, IC=0.0401; S&P500 (residual) Sharpe=1.0672, AR=23.65%	Sharpe +51% vs AlphaGen on CSI500 (0.4040 → 0.6109); residual memory learns incremental improvements	Liu et al. (2026) (XALPHA)	CSI 300, 10-day open-to-open return prediction	LLM-based	IR=1.5368; 60 elite factors; avg pairwise abs. corr.=0.142	IR 0.9516 → 1.5368 vs AlphaAgent; 48 Research Archetypes distilled from external research
Chen and Kawashima (2026)	Multiple equity markets (price/volume/sentiment)	LLM-based	DeepSeek generates 50 alphas + PPO real-time weighting	Outperforms equal-weight, B&H, momentum in Sharpe and MDD; neuro-symbolic LLM+RL hybrid					
Yi et al. (2026) (AlphaSchema)	CSI 300 (2023–2025)	LLM-based	IC=0.0382, ICIR=0.2374, IR=1.0877, annual excess return=11.94%	Constrains exploration semantics (schema plans); surrogate reward over semantic space	Ye et al. (2026) (AgonAlpha)	US TOP3000, delay-one (2019–2023), WorldQuant BRAIN	LLM-based	60 alphas from 5 users; 17 SPECTACULAR; top Fitness=9.50, Sharpe=3.48	Adversarial reviewer with re-execution & veto; pending-aware parallel budget; public evidence trail
Zhu et al. (2026) (AlphaSeek)	CSI 300 (OOS portfolio) + CSI 500 (transfer)	LLM-based	Improved OOS performance on CSI 300; factors transferable to CSI 500	Trajectory-level mutation/crossover with frozen validated sub-steps; factor-library-level backtests feed search					
Shen (2024) (Alpha-GFN)	Sequential operator generation (formulaic)	NN-based	Proportional sampling $P(s) \propto R(s)$ prevents mode collapse	Diverse multimodal factor structures vs. PPO reward-maximizing convergence	Chen et al. (2025a) (AlphaSAGE)	CSI 300, CSI 500 (top 20% long); S&P 500 (top/bottom 10% LS); 6-dim features (O/C/H/L/VWAP/Vol)	NN-based	CSI300 IC=0.079, RankIC=0.094, Sharpe=1.71, AR=7.62%, MDD=-17.3%; runtime 0.49h	Sharpe +103.6% vs AlphaGen (0.84 → 1.71); runtime -63% vs AlphaGen (1.34 → 0.49h); multi-dim dense reward + AST+RCN

Continued on next page...

Table 3: State-of-the-Art Evaluation in Signal Generation (Continued)

Model	Scope ($\mathcal{S}$)	Method ($\mathcal{M}$)	Metrics ($\mathcal{R}$)	Improvement	Model	Scope ($\mathcal{S}$)	Method ($\mathcal{M}$)	Metrics ($\mathcal{R}$)	Improvement
Su (2026) (AlphaG-OPD)	CSI 300, CSI 500, CSI 1000, S&P 500	NN-based	Reliability-gated on-policy structural distillation; sibling counterfactuals at partial AST states	Cuts off invalid formula branches early; consolidated replay avoids redundant evaluations					

Signal Generation. Signal generation methods directly output predictive signals, return forecasts, directional signals, or portfolio weights, without producing explicit symbolic factor expressions, corresponding to the methods surveyed in Section 3.3. Table 4 organizes these methods by paradigm (Time-Series, RL, Agentic), covering LLM-backbone reprogramming for numeric forecasting, RL-trained execution policies, and multi-agent LLM frameworks.

Table 4: State-of-the-Art Evaluation in Signal Generation

Model	Scope ($\mathcal{S}$)	Method ($\mathcal{M}$)	Metrics ($\mathcal{R}$)	Improvement	Model	Scope ($\mathcal{S}$)	Method ($\mathcal{M}$)	Metrics ($\mathcal{R}$)	Improvement
Shabani et al. (2023)	FI-2010 dataset	Time-Series	F1=69.92% (aTABL-IS2); F1=64.93% (aCNN)	Outperforms fine-tuning and full retraining baselines; drastically reduces storage	Zhou et al. (2023) (FPT)	>10 real datasets (electricity, traffic, etc.)	Time-Series	MSE=0.047	-13.4% error vs TimesNet (0.049); +30% performance in few-shot (10% data)
Jin et al. (2024) (Time-LLM)	ETTh/m, Weather, Traffic, ILLI, M4	Time-Series	sMAPE improved by 8.7% (M4)	-12% MSE vs GPT4TS; -20% vs TimesNet; zero-shot outperforms LLMTime by >75%	Liu et al. (2024c) (AutoTimes)	ETTh1, ECL, M3/M4 (2006–2018)	Time-Series	sMAPE=11.831 (M4); MSE=0.389 (ETTh1)	Surpassed TimeLLM by 9.12% (ETTh1); >5× speedup; -13% error in zero-shot
Wang et al. (2024c) (StockTime)	S&P 100 hourly/daily (2014–2024)	Time-Series	MSE=0.146, IC=0.018 (S&P 100-D)	MSE -10–20% vs FinMA/Time-LLM; IC >+300% vs LSTM/ALSTM	Jia et al. (2025) (LLM4FTS)	CSI300/500, S&P500, NDX100 (2018–2024)	Time-Series	ARR=0.528 (CSI300); Sharpe=2.772 (NDX)	ARR +72.9% vs Transformer; Sharpe +133% vs PatchTST; Calmar +64.4% vs AlphaStock
Khezresmaeilizadeh et al. (2025) (VISTA)	AC40 stocks (2014–2020)	Time-Series	Up to -89.83% MSE vs text-only LLMs	Outperformed ARIMA baseline across all tested stocks; multimodal numeric + K-line charts	Koa et al. (2025) (VTA)	US (ACL18), China A50, EURO STOXX 50	Time-Series	Sharpe=1.7190 in portfolio optimization	Outperforms TimeLLM (1.5230), Autoformer, and DeepSeek-R1 (1.4074); inverse MSE reward
Huang (2018) (Financial DRQN)	FX tick data (2012–2017, 12 pairs), 15-min OHLCV	RL	AR=23.8%, spread 0.08 bp	vs. buy/sell-and-hold: outperf.; action aug +6.4% vs ϵ -greedy; replay memory=480 steps	Patel (2018)	Bitcoin order book & tick (Nov 2018)	RL	Realized PnL = \$1,843	+395% vs buy-and-hold (-\$627); +266% vs trend-following; Macro+Micro agent split
Liu et al. (2018)	DJIA 30 stocks (2009–2018)	RL	AR=25.87%, Sharpe=1.79	Sharpe +23% vs min-var; +41% vs DJIA; AR + $\sim$ 10pp; standard Actor-Critic for continuous portfolios	Sadighian (2019) (DRLMM)	Crypto LOB (BTC, ETH, LTC, Nov 2019)	RL	Avg daily return=0.3462% (LTC)	>3× avg trade returns vs positional PnL; 143-dim ESS, 17 actions, emergency flattening
He et al. (2025) (HPPO-TO)	S&P 500 & HS300 1-min data (2022–2023)	RL	IC=0.0719, RankIC=0.0774 (S&P 500)	IC +25%, IR +10% vs HRFT; CR +25% higher; Macro/Fund/Technical/News agents	Akbar (2025)	Bitcoin 1s LOB data	RL	Avg episode return=904.15, Sharpe=2.07	Architecture improved on FinRL-DeepSeek; LOB+Alpha101+LLM sentiment ensemble (DQN/PPO/SAC)
Grzesiak and Aggarwal (2025) (SecureFinAI)	BTC/USD 1s LOB + news (2022–2023)	RL	Sharpe=0.00038, Max DD=-0.0002	Sharpe -0.0007 $\rightarrow$ 0.00038 vs Alpha101 RL baseline; Feature Aggregation Transformer + Ensemble DQN	Ahmed et al. (2025) (FinAI-Trader)	Bitcoin LOB 1s data (Apr 2021)	RL	Return=0.017%, Sharpe=0.088	Positive risk-adj return vs $\sim$ 0 base framework; PPO variants for fast sentiment adaptation
Darmanin and Vella (2025)	6 US equities multi-modal (2012–2020)	RL	Mean Sharpe=1.10	Sharpe +80% vs RL-only baseline (0.55 $\rightarrow$ 1.02); entropy-weighted directional signal τ appended to obs; Double DQN preserved	Deng and Wang (2025) (AlphaQuantebacktest)	5 US tech stocks (6-month)	RL	AlphaQuanter-7B: avg ARR=34.94%, SR=0.65, MDD=24.93%; 3B: ARR=23.03%	ARR +18.45pp vs TradingAgents (16.49% $\rightarrow$ 34.94%); single-agent + RL > multi-agent
Chen et al. (2023b)	Dow 30 + news (2020–2022)	Agentic	F1=0.4133; Vol=14.06%, MDD=12.42%	F1 +1.8% vs News-Embed; vol/MDD lower than ChatGPT; dynamic connectivity graphs via zero-shot LLM	Kirtac and Germano (2024)	6,214 US stocks + Refinitiv news (2010–2023)	Agentic	CR=355%, Sharpe=3.05	Sharpe +148% vs Loughran-McDonald; beats BERT (2.11); LLM (OPT/FinBERT) vs lexicons

Continued on next page...

Table 4: State-of-the-Art Evaluation in Signal Generation (Continued)

Model	Scope (S)	Method (M)	Metrics (R)	Improvement	Model	Scope (S)	Method (M)	Metrics (R)	Improvement
Fatouros et al. (2024) (MarketSenseAI)	S&P 100 (Dec 2022 – Mar 2024)	Agentic	CR=72.87%, Sharpe=2.80	CR +189% vs S&P 100 baseline (25.22%); price-volume + news text	Zhang et al. (2024b) (FinAgent)	5 US stocks + ETH (2022–2024)	Agentic	Peak ARR=92.27% (TSLA)	ARR +51.64% vs FINMEM/FinGPT; multimodal memory-augmented agent
Yu et al. (2024) (FinCon)	8 US equities + 2 portfolios multi-modal (2022–2023)	Agentic	CR=113.84%, Sharpe=3.27	CR +80%, Sharpe +100%, MDD -30% vs FINMEM; hierarchical Manager-Analyst	Koa et al. (2024b) (SEP)	55 stocks + Ya-hoo/Twitter (2020–2022)	Agentic	ACC=54.35%, MCC=0.0993 (informative text)	ACC +4.2%, MCC +0.0177 vs GRU+Attn; explanation score +10% vs GPT-3.5/Vicuna
Cao et al. (2024) (ECC Analyzer)	S&P 500 earnings calls audio+text (2017)	Agentic	Avg MSE=0.314; MSE ₃ =0.553	MSE -27.7% vs HTML (0.401); outperforms LSTM, MRDM, and GPT-4-Turbo	Zhang et al. (2024a) (StockAgent)	Simulated stock trading (10 days)	Agentic	Simulation only: GPT/Gemini order prices (A/B) = 55.70/43.43 vs. 23.46/36.03; no portfolio return or Sharpe reported	Different backbone LLMs produce distinct simulated trading behavior
Li et al. (2024c) (CryptoTrade)	BTC, ETH, SOL (2021–2023)	Agentic	Return=28.47% (BTC), 115.18% (SOL)	Return +15%, Risk-adj +50% vs PatchTST/Informer	Yang et al. (2024a) (FinRobot)	Dow 30 & SSE 50	Agentic	Acc 0–1% change	4-layer architecture; Financial CoT; scheduler routes to specialized fine-tuned models (Chat-GLM/FinGPT)
Yu et al. (2025b) (FINMEM)	5 US stocks + news (2021–2023)	Agentic	CR=61.78%, Sharpe=2.68 (TSLA)	CR +48pp, Sharpe >2x, MDD >50% reduction vs Generative Agents	Han et al. (2025) (FinSphere)	Stocksis dataset	Agentic	AnalyScore=70.88	+3.33 vs FINMEM, +4.27 vs GPT-4o; deep causal extraction from earnings calls
Xiao et al. (2024) (TradingAgents)	5 tech stocks (Jan–Mar 2024)	Agentic	CR=26.62%, ARR=30.5%, Sharpe=8.21	CR +6.1%, Sharpe +4.26 vs SMA; bounded debates prevent info distortion	Zhao et al. (2025c) (AlphaAgents)	15 technology stocks (four-month test from Feb. 2024)	Agentic	NR (cumulative-return and rolling-Sharpe curves only; no tabulated numerical result)	Risk-neutral multi-agent curve exceeds the benchmark and single-agent curves
Song et al. (2025) (TiMi)	200+ trading pairs (2024–2025)	Agentic	ARR=13.7%, Sharpe=0.86 (altcoins)	Return +50%, Sharpe +50% vs TradingAgents/PatchTST; decoupled Trade-in-Minutes	Fatouros et al. (2025) (MarketSenseAI)	S&P 100 (2023–2024) & S&P 500 (2024) 2.0)	Agentic	S&P100 MS-Eq Return=55.7%, Sharpe=2.13; MS-Cap Return=125.9%, Sharpe=2.76; S&P500 MS-Eq Return=25.8%, Sharpe=2.4	Return +102% vs S&P500 Eq benchmark (12.8% → 25.8%); Sortino 3.25 → 3.68
Guo et al. (2026a) (AQuA)	Crypto universe + US equities (2021–2025)	Agentic	Factor-system combined IC ≈0.190 (crypto); model-system per-stock IC +0.0843 (US eq.); held-out Sharpe up to +2.50	Positive in every year 2021–2025; two independent self-improving LLM research systems	Mao et al. (2026) (EVOQUANT)	A-share + crypto (7 strategies)	Agentic	Avg test Sharpe -0.298 → 0.538; best strategy +199%	Self-evolving verifier-guided strategy refinement; multi-stage verification pipeline
Jiang et al. (2025) (Alpha-R1)	CSI 300 (in-domain) & CSI 1000 (out-of-domain, 2025.01–06)	Agentic	CSI300 CR=12.99%, AR=27.59%, SR=1.62, MDD=6.76%; CSI1000 CR=42.49%, AR=78.18%, SR=4.03	SR 1.62 vs Buy&Hold 0.33 (~5x), vs PPO 0.11; zero-shot transfer CSI300 → CSI1000 SR=4.03	Tang et al. (2026) (AlphaAgent Evo)	Real stock market datasets	Agentic	IC=0.129; IR=2.442	+0.065 IC vs AlphaAgent (0.064); IR 2.442 vs 2.046; GRPO over evolutionary trajectories
Ang et al. (2026)	18 liquid asset classes via ETFs (1996–2026)	Agentic	ER=6.87%, Vol=7.54%, SR=0.39, MDD=-25.6%	MDD -34.3% → -25.6% vs 60/40 benchmark; 50+ agent society under Investment Policy Statement; Borda count voting					

7.3.2 Evaluation on Factor Combination

Factor combination implements the aggregation operator $\mathcal{T}(\cdot; \theta_T)$ surveyed in Section 4, transforming the raw factor matrix F_t into a unified composite signal $S_{raw,t}$. Following the chapter’s four-paradigm organization, we split the evaluation into four separate tables aligned with Section 4’s subsections: Classical methods (Table 5, Section 4.2), Supervised methods (Table 6, Section 4.3), RL methods (Table 7, Section 4.4), and LLM-Enhanced methods (Table 8, Section 4.5).

Classical Methods. Classical factor combination methods rely on linear algebraic or risk-based aggregation rules, corresponding to the methods surveyed in Section 4.2. Table 5 organizes these methods by their core mechanism (multi-factor risk premia, risk parity, dynamic allocation).

Table 5: State-of-the-Art Evaluation in Factor Combination: Classical Methods

Model	Scope ($\mathcal{S}$)	Method ($\mathcal{M}$)	Metrics ($\mathcal{R}$)	Improvement	Model	Scope ($\mathcal{S}$)	Method ($\mathcal{M}$)	Metrics ($\mathcal{R}$)	Improvement
Blitz and Van Vliet (2008)	12 major asset classes (1986–2007)	Classical	AR=11.9%, Vol=10.0%, IR=1.19, Alpha=11.0–12.4%	Surpasses single-factor strategies; strong alpha unexplained by CAPM, FF3, Carhart4	Williams (2014)	Global asset classes & 4 core factors (1985–2012)	Classical	AR=9.62%, Vol=4.33%, Sharpe=2.24 (dynamic)	Sharpe 2.24 vs 0.78 (classic RP) and 0.40 (60/40); nearly 4× Sharpe of S&P 500
Bender and Wang (2015)	MSCI World Index factor tilt (1993–2015)	Classical	AR=11.80%, Vol=14.12%, Risk-adj=0.84, IR=0.72	AR +86bps, Risk-adj +15.1%, IR +22%, Vol -6.2% vs combination 4-factor portfolio	Jiang et al. (2026a) (Scaled PCA)	50 equity anomalies (1974–2019)	Classical	OOS Sharpe=1.84, AR alpha=19.02%; In-sample Sharpe=2.29	Sharpe +21.9% vs traditional PCA, +13.6% vs RP-PCA; OOS alpha +354% vs trad. PCA
Garrone (2026)	Large-cap liquid assets & commodities	Classical	Controlled conc., moderate turnover, improved drawdown convexity	Outperforms CW-MC in div. & drawdown; lower turnover than EW-MC; robust across regimes	Arratia and Gzyl (2026) (EFM)	Portfolio replication (multi-asset)	Classical	Fermi-Dirac-type entropy defined on constraints; entropy-minimizing replication weights	Outperforms OLS replication in annualized turnover and net-of-fees returns; probabilistic “circuit breaker” under stress

Supervised Methods. Supervised factor combination methods learn non-linear aggregations from labeled data, corresponding to the methods surveyed in Section 4.3. Table 6 organizes these methods by model family (tree ensembles, recurrent networks, convolutional networks, autoencoders).

Table 6: State-of-the-Art Evaluation in Factor Combination: Supervised Methods

Model	Scope ($\mathcal{S}$)	Method ($\mathcal{M}$)	Metrics ($\mathcal{R}$)	Improvement	Model	Scope ($\mathcal{S}$)	Method ($\mathcal{M}$)	Metrics ($\mathcal{R}$)	Improvement
Nakagawa et al. (2019)	Time-varying multi-factor models (Japanese equities)	Supervised	LSTM Sharpe=0.88, Return=13.20%; LSTM+LRP Sharpe=1.31, Return=15.18%	Sharpe +17% vs Linear model (0.75 $\rightarrow$ 0.88); LSTM+LRP best at 1.31; interpretable non-linear factor dynamics	Zhang et al. (2019) (DeepLOB)	FI-2010 LOB, mid-price direction ($k = 10/50/100$)	Supervised	Setup 1: F1=77.66% ($k=10$), 74.96% ($k=50$), 76.58% ($k=100$); Setup 2 $k=10$: Acc=84.47%, F1=83.40%	Setup 2 $k=10$: F1 +5.77pp vs C(TABL) (77.63% $\rightarrow$ 83.40%); Acc +6.46pp
Feng et al. (2023)	U.S. corporate bonds (2004–2020), 132 features	Supervised	OOS Sharpe=2.13 (EW) / 2.21 (VW); MDD $\sim$ 0.89%	Sharpe +142% vs FF5, +44% vs IPCA, +129% vs RP-PCA; long-short deep factor	Ong and Herremans	49 continuous futures (1990–2023)	Supervised	AR=1.92%, Vol=0.82%, Sharpe=2.33, Sortino=3.81, MDD=-1.02%	Sharpe +117%, MDD -49% vs TSMOM(1,12); Sharpe +35%
Lim et al. (2021)	CSI 300 & S&P 500 multi-horizon	Supervised	Higher OOS Sharpe vs standalone CNN/LSTM	recurrent layers + attention global context; interpretable feature importance	Wang and Guo (2024) (RVRAE)	Dynamic factor models (monthly equity panel)	Supervised	Sharpe=2.26 (no TC), 2.03 (30bps TC); $R^2_{\text{total}}=20.06$, $R^2_{\text{pred}}=1.57$	Sharpe +5.1% vs ALSTM (2.15 $\rightarrow$ 2.26); probabilistic latent factors with uncertainty
Wang and Singh (2024) (KAN)	Non-linear factor extraction (6-factor model)	Supervised	KAN-CA Sharpe=0.96, $R^2_{\text{pred}}=0.214\%$	Sharpe +5.5% vs CA (0.91 $\rightarrow$ 0.96); better on highly non-linear factor interactions; higher compute cost	Den et al. (2026) (MAPLE)	US, China, Japan equity markets (4 markets)	Supervised	Avg Sharpe=1.690, Calmar=2.175 vs 9 baselines	Best avg Sharpe & Calmar; up to 55× fewer params, 2.5× less training; diversity-regularized multi-alpha head
Chehab et al. (2026) (MINGLE)	Equity markets (vol regimes & transaction-cost levels)	Supervised	Mutually-informed Graph-Locality & Exposures (unified ADMM)	Exposure-locality graph aligns with economic sectors; outperforms correlation-based graphs					

RL Methods. RL factor combination methods learn aggregation policies through environment interaction, corresponding to the methods surveyed in Section 4.4. Table 7 organizes these methods by their contribution (end-to-end weighting, infrastructure, regime-adaptive extensions).

Table 7: State-of-the-Art Evaluation in Factor Combination: RL Methods

Model	Scope (S)	Method (M)	Metrics (R)	Improvement	Model	Scope (S)	Method (M)	Metrics (R)	Improvement
Moody and Saffell (2001)	Direct RL from investment returns	RL	Outperforms predict-then-optimize two-stage	Established that minimizing MSE does not maximize investment performance; foundational for end-to-end	Zhou et al. (2024) (MFDRL)	Chinese A-share (2015–2022), 6 indices & 27 factors	RL	AAR=9.0%, ASR=0.496, MDD=13.1%	AAR +200%, ASR 14× vs OLS two-step; AAR +374% vs UBAH; AAR +260% vs Hist
Liu et al. (2020) (FinRL)	Standardized DRL environments for trading	RL	Reproducible infrastructure	Lowered practical infrastructure barrier for DRL trading agents	Liu et al. (2018)	DJIA 30 stocks (2009–2018)	RL	AR=25.87%, Sharpe=1.79	Demonstrated feasibility of deep RL (DDPG) for stock trading
Li et al. (2019)	DJIA 30 stocks (2001–2018; OOS 2014–2018)	RL	AR=18.84%, Sharpe=1.63 (adaptive optimistic/pessimistic DDPG)	Sharpe +61% vs vanilla DDPG (1.01 → 1.63); AR 18.84% vs DJIA 11.36%	Wang et al. (2020) (HRPM)	DJIA 30 (US) & SSE 50 (China)	RL	US: ARR=27.09%, ASR=1.246; China: ARR=48.74%, ASR=3.813	China ARR +13% vs DPM (35.81% → 48.74%); +42% vs SSE50 market; hierarchical RL with high-level allocation + low-level execution
Sahoo et al. (2026) (Photonic QC)	Jensen-Kelly-Pedersen 13-factor library (penalty sweep)	RL	Photonic annealer vs Gurobi MIP vs SAC (constrained factor portfolio)	Classical MIP superior under risk-constrained mandates; RL allocator exhibits structural failure modes					

LLM-Enhanced Methods. LLM-Enhanced factor combination methods leverage language models for semantic weighting or direct signal generation, corresponding to the methods surveyed in Section 4.5. Table 8 organizes these methods by their integration pattern (combination-aware generation, dual-agent evaluation, RL-weighted LLM pipelines, LLM-gated momentum).

Table 8: State-of-the-Art Evaluation in Factor Combination: LLM-Enhanced Methods

Model	Scope (S)	Method (M)	Metrics (R)	Improvement	Model	Scope (S)	Method (M)	Metrics (R)	Improvement
Luo et al. (2025a) (EFS)	FF25/32/49/100, LLM-US50, HSI45, CSI300	LLM-Enhanced	CW=2434.51 (US50); SR=0.257; MDD=0.454	+50% CW vs ASMCVaR (US50); +80% CW vs ASMCVaR (CSI300); combination-aware generation	Kou et al. (2024)	SSE50, CSI300, S&P 500 markets, macro data (2019–2024)	LLM-Enhanced	SSE50: cum. return 53.17%, Sharpe 0.287, OOS IC=0.047	Outperforms XGBoost (9.53%), FinCon (22.47%), SEP (17.89%); dual-agent eval (confidence vs. risk) + 3-layer MLP
Chen and Kawashima (2026)	Multiple equity markets (price/volume/sentiment)	LLM-Enhanced	DeepSeek generates 50 alphas + PPO real-time weighting	Outperforms equal-weight, B&H, momentum in Sharpe and MDD; RL-based weight allocation in LLM pipelines	Anic et al. (2025)	S&P 500 + news (Oct 2019–Mar 2025)	LLM-Enhanced	AR=30%, Vol=22%, SR=1.06, MDD=-17%	SR 0.79 → 1.06, AR 24% → 30% vs traditional momentum baseline; LLM gates cross-sectional momentum

7.3.3 Evaluation on Risk Control

Risk control implements the operator $\mathcal{Z}(\cdot; \theta_Z)$ surveyed in Section 5, transforming the combined signal $S_{raw,t}$ into a purified, risk-adjusted signal $S_{pure,t}$ through orthogonalization, regime scaling, and tail truncation. Following the chapter’s two-paradigm organization, we split the evaluation into two separate tables aligned with Section 5’s subsections: Statistical methods (Table 9, Section 5.2) and LLM-based methods (Table 10, Section 5.3).

Statistical Methods. Statistical risk control methods rely on numerical-statistical techniques such as orthogonalization, volatility scaling, and tail truncation, corresponding to the methods surveyed in Section 5.2. Table 9 organizes these methods by their core mechanism (idiosyncratic risk, momentum timing, risk-aware GNP, regime switching, VaR/CVaR estimation).

Table 9: State-of-the-Art Evaluation in Risk Control: Statistical Methods

Model	Scope (S)	Method (M)	Metrics (R)	Improvement	Model	Scope (S)	Method (M)	Metrics (R)	Improvement
McLean (2010)	CRSP / Compustat US stocks (1965–2004)	Statistical	Avg monthly return=1.386%; FF3 alpha=0.901%	Return +0.83pp vs EW baseline; high risk alpha +1.11% vs low risk; foundational idiosyncratic risk rationale	Dudler et al. (2015) (RAMOM)	64 highly liquid futures contracts	Statistical	RAMOM strategy significantly optimizes trading costs	Dollar turnover -40%; higher Sharpe vs TSMOM; reduced dependence on volatility
Chen and Wang (2015)	Multi-asset allocation under risk constraints	Statistical	Mean-CVaR embedded in GNP transitions	Safer capital allocation via structured state transitions; foundational risk-aware GNP	Butt et al. (2024)	Momentum portfolios with volatility regimes	Statistical	Switching protocol: momentum $\rightarrow$ reversal on vol spikes	Prevents momentum crashes during severe drawdowns; liquidity-providing reversal
Yang et al. (2020a)	DJIA 30 stocks (2009–2020)	Statistical	Sharpe=1.30, AR=13.0%, MDD=-9.7%	Sharpe +18% vs best single agent (A2C 1.12); +175% vs DJIA; turbulence index triggers halt-and-sell	Wang et al. (2024a)	U.S. company financial reports	Statistical	F1 score=91.7% with LSTM	Outperforms Random Forest (89.2%) and SVM (87.5%) in risk detection
Arian et al. (2020) (Encoded VaR)	S&P 500, LSE, FSE (1998–2019)	Statistical	Firm loss scores: 0.0568 (95%), 0.017 (99%)	Outperforms GARCH/CAViaR; increased signal-to-noise ratio from 37% to 79%	Kaibuchi et al. (2022) (GARCH-UGH)	Dow, Nasdaq, Nikkei, JPY/GBP (1993–2010)	Statistical	Avg violation rate 3.0–3.2 at 99.9% VaR	VaR violation errors -50% vs GARCH-EVT; succeeded in 58/60 coverage tests
Zhang and Zhang (2022) (ICVaR)	10 Chinese listed companies (2016–2020)	Statistical	Stable returns with Sharpe 0.01 to 0.15	Better captures market uncertainty vs classical Markowitz; interpretable allocations	Jha (2023) (TVaR)	AAPL, MSFT, GOOGL 1-year daily	Statistical	TVaR values: 469.27 (AAPL), 1006.00 (MSFT)	Quantified structural changes in return topology via TDA; novel risk dimension
Rigana et al. (2024) (NECO VaR)	20 Forex rates vs USD (2000–2021)	Statistical	5.11% out-of-sample exceedance rate ($\alpha=5\%$)	LRuc acceptance 95% vs 78% (FHS-GARCH); superior invariance to shocks; structural equation modeling of contagion	Gu et al. (2025) (MTS)	5 diverse stock datasets (2019–2023)	Statistical	Avg CR=0.3255, peak Sharpe=2.6872	CR +30.67%, Sharpe +29.33% vs MIGT; up to 115% higher CR; Incremental CVaR in RL reward
Choi (2026) (EVaR)	US sector ETF allocation (2000–2026)	Statistical	ICA+NTS min-EVaR Sharpe=0.616	Higher realized Sharpe than matched CVaR portfolios; EVaR evaluated from fitted asset-level parameters	Fang et al. (2026) (ISCOS)	Gaussian / t -copula credit portfolios	Statistical	Cross-entropy importance sampling + COS method	Gaussian / Gaussian-inverse-Gamma proposals; error propagation analysis for rare losses
Das et al. (2026) (MORP-DRL)	10 global equity indices (pre/COVID/post-COVID)	Statistical	Multi-objective RL: return + variance/CVaR/EVaR; GARCH-EVT- t -copula scenarios	PPO vs NSGA-II; reduces downside risk during market stress					

LLM-based Methods. LLM-based risk control methods leverage language models for semantic risk extraction, crash warning, and risk-aware execution, corresponding to the methods surveyed in Section 5.3. Table 10 organizes these methods by their integration pattern (LLM risk perception, probabilistic risk forecasting, multimodal vol estimation, stress testing, crash prediction, risk-sensitive RL).

Table 10: State-of-the-Art Evaluation in Risk Control: LLM-based Methods

Model	Scope (S)	Method (M)	Metrics (R)	Improvement	Model	Scope (S)	Method (M)	Metrics (R)	Improvement
Chen et al. (2023b)	Dow 30 + news (2020–2022)	LLM-based	F1=0.4133; Vol=14.06%, MDD=12.42%	F1 +1.8% vs News-Embed; vol/MDD lower than ChatGPT; foundational LLM risk perception	Turtel et al. (2026) (Foresight)	SEC filings Risk Factors sections	LLM-based	Brier=0.1979, Brier Skill=11.6%, ECE=0.0287 (risk materialization)	ECE 0.1419 $\rightarrow$ 0.0287; Brier 0.2381 $\rightarrow$ 0.1979; beats GPT-5 (Brier 0.1986, ECE 0.0812)
Chen et al. (2026a)	8 major US banks, Reuters news (OOS incl. 2011 crisis)	LLM-based	Avg quantile loss ($\times 10^{-2}$) GS: 0.11 vs 0.15 (return-only MLP) full test; 0.12 vs 0.15 crisis window	Text-based CoVaR lowers losses in stress window (2011 sovereign downgrade) for most banks; provable error bounds	Cao et al. (2025) (RiskLabs)	Earnings calls, VIX, news (U.S., to 2024)	LLM-based	3-day vol MSE=0.324, VaR error=0.049	3-day MSE -20% vs HTML; improves VaR accuracy from 0.056 (LSTM); multimodal (audio+text+VIX)
Soleimani (2025)	G7 macro + ETFs (2000–2025)	LLM-based	LLM stress: VaR 1.41–1.48 $\times$, CVaR 1.13–1.23 $\times$	Volatility channel amplifies risk to 3.6–3.8 $\times$ VaR; levels below historical crisis; Prompt-RAG counterfactuals	Koa et al. (2024a) (TRR)	Portfolio stocks, Reuters news (2007/2010/2020/2022 crises)	LLM-based	Zero-shot crash AUROC=0.69 (2007), 0.66 (2020), 0.64 (2022), 0.61 (2010)	Beats best baseline ToG (0.50–0.63) on all crises; +13.1% cumulative return vs next-best in trading eval

Continued on next page...

Table 10: State-of-the-Art Evaluation in Risk Control: LLM-based Methods (Continued)

Model	Scope (S)	Method (M)	Metrics (R)	Improvement	Model	Scope (S)	Method (M)	Metrics (R)	Improvement
Benhenda (2025) (FinRL-DeepSeek)	FNSPID news + Nasdaq-100 (1999–2023)	LLM-based	IR=0.0078, Rachev Ratio=0.9818 (10% infusion)	LLM infusion improves CPPO; risk-sensitive RL benefits more than standard PPO; CVaR-PPO framework	Yang et al. (2025)	Derivatives hedging backtest (historical options data)	LLM-based	LLM-driven dynamic hedge Sharpe=1.85–1.90, MDD 9.2%–9.8%	Drawdown 15.6% → 10.2% (GPT-4); vs static hedge Sharpe 1.39 baseline
Chen et al. (2025c) (FinHEAR)	Multi-asset portfolio, expert-guided retrieval	LLM-based	CR=83.89%, SR=3.28, MDD=15.77%, Calmar=5.32	SR +55% vs FinCon (2.11 → 3.28); CR 39.21% (Markowitz) → 83.89%; MDD -55% vs FinCon					

7.3.4 Evaluation on Portfolio Management

Portfolio management implements the operator $\mathcal{U}(\cdot; \theta_U)$ surveyed in Section 6, mapping purified signals $S_{pure,t}$ and market structure Σ_t into executable portfolio weights W_t^* . Following the chapter’s two-axis taxonomy, we split the evaluation into single-period methods (Table 11, Section 6.2) and multi-period methods (Table 12, Section 6.3). Single-period methods solve a static optimization at each rebalance, further split by decision granularity into macro-level allocation (Section 6.2.1, across asset classes) and micro-level construction (Section 6.2.2, within a single market), with each level following four paradigms: Traditional mathematical optimization, Optimizer technology innovations, RL under frictions, and LLM-based cognitive orchestration.

Single-Period Portfolio Construction. Single-period methods correspond to Section 6.2, covering macro-level allocation (Section 6.2.1) and micro-level construction (Section 6.2.2). Within each level, methods follow four paradigms: Traditional mathematical optimization, Optimizer technology innovations, RL under frictions, and LLM-based cognitive orchestration.

Table 11: State-of-the-Art Evaluation in Single-Period Portfolio Construction

Model	Scope (S)	Method (M)	Metrics (R)	Improvement	Model	Scope (S)	Method (M)	Metrics (R)	Improvement
Richard and Roncalli (2019)	Multi-asset balanced mandates	Macro-Traditional	Risk budgeting equalizes risk contributions	Institutional default for multi-asset; avoids extreme MVO weights under estimation error	Cetingoz et al. (2024)	500 S&P 500 + 15 cross-asset (2015–2023)	Macro-Traditional	AR=-0.62%, Vol=7.04%, ES=1.01%, MDD=16.26%	Turnover -80% vs long-only FRB; outperforms EFRB
Black and Litterman (1992)	Multi-asset with view injection	Macro-Traditional	Bayesian merger of equilibrium + views	Standard view-injection mechanism; dramatically improves MVO output stability	Cetingoz and Guéant (2025)	Multi-factor covariance structures	Macro-Traditional	Equalizes risk contributions from multi-factor covariance	Balances asset-class + factor risk budgets; no return estimation needed
Iglesias et al. (2024)	Multi-asset risk budgeting	Macro-Traditional	Mirror-descent solvers for risk parity	Algorithmic refinement: efficient risk-parity computation via mirror descent	Parra-Díaz and Castro-Iragorri (2025)	Differentiable risk budgeting	Macro-Traditional	Differentiable deep-declarative variants	Enables gradient flow through risk-budgeting solver
Cotton (2024)	Multi-asset with complex dependence	Macro-Traditional	Schur complementary allocation	Generalizes hierarchical risk parity to complex dependence structures	Nayar et al. (2026)	Multi-asset universes	Macro-Traditional	Topological risk parity	Generalizes López de Prado’s HRP for multi-asset with complex topology
Garivaltis (2021)	Multi-asset risk budgeting family	Macro-Traditional	Closed-form universal risk budgeting	Reference point for entire risk-budgeting family	Oliveira et al. (2026)	FRED-MD macro panel	Macro-Traditional	Regime-conditioned return/vol estimates	Modified k-means regime classification feeds multi-asset sizing
Shu et al. (2025)	12 global equity/bond/REIT/real estate indexes	Macro-Traditional	Asset-specific regime forecasts via jump models + GBT	Asset-specific (not universe-wide) regime forecasts; fed into Markowitz MVO	Luo et al. (2025c)	Multi-asset dynamic allocation	Macro-Traditional	Tail-dependence + regime-shift modeling	Captures tail dependence for more robust dynamic allocation
Shu and Mulvey (2024)	7 indices (market + 6 style factors), 1993–2024	Macro-Traditional	Dynamic factor portfolio: IR $\approx$ 0.4–0.5 vs EW; excess return 10.5%–12.5%, Sharpe 0.52–0.65	IR 0.05 → 0.44 (dynamic allocation); per-factor Sharpe 0.16 (momentum) → 0.40 (value/growth)	Reis et al. (2025)	Medium-horizon multi-asset	Macro-Traditional	Deep-learning covariance forecasting	Targets medium-term covariance horizons; critical input for all macro methods

Continued on next page...

Table 11: State-of-the-Art Evaluation in Single-Period Portfolio Construction (Continued)

Model	Scope (S)	Method (M)	Metrics (R)	Improvement	Model	Scope (S)	Method (M)	Metrics (R)	Improvement
Huh et al. (2025)	Continuous-time macro portfolio choice	Macro-Traditional	Pontryagin-guided direct policy optimization	Solves continuous-time macro portfolio under regime switching					
Raj (2025)	Multi-asset classes (Decades to 2020)	Macro-RL	Transformer PPO SR=1.43	SR > 3× vs Equal-Weight (0.42) & Sharpe-Optimized; HMM regime in state space	Niu et al. (2022) (MetaTrader)	DJIA, HSI, CSI100 (1987–2021)	Macro-RL	AR=25.6%–44.1%, SR=1.31–1.73	AR +7% to 10% vs DeepTrader; lower MDD; meta-policy routes to best expert
Pan et al. (2026a)	NIKKEI30, NAS100, etc. (KDD 2026)	Macro-RL	NIKKEI30: CR=133.29%, SR=1.00; NAS100: CR=146.38%, SR=1.10, MDD=24.22%	Avg cumulative return +6.45%–38.79% vs baselines; regime-gate avoids catastrophic forgetting	Verma et al. (2026)	SPY/TLT/GLD/WTI (2004–2025)	Macro-RL	RL (π^*): Sharpe=0.83, MDD=-23.5%	Sharpe 0.83 vs B&H SPY 0.65; beats 60/40 rotation (0.82), Top-1 rotation (0.79)
Alzahrani (2025)	Multi-asset ETFs (2005–2025, walk-forward)	Macro-RL	Sharpe=1.23, MaxDD=9.3%, Vol=8.4%, Return=10.3%	Sharpe 1.23 vs BL 1.02, EW 0.38; MaxDD -34.1% after net costs	Pollok and Robik (2026)	16 CME futures (end-to-end)	Macro-RL	Best net Sharpe=0.87 (transformer policy)	0.99 prob. beat risk parity; 0.59 prob. beat equal weight; rules Sharpe 0.78–0.99
Wood et al. (2026) (DecPM)	50 diversified futures	Macro-RL	~2× net risk-adjusted returns vs trend-following	Macroeconomic Graph Prior + distributionally robust worst-window objective	Pan and Chen (2026) (SBCEA)	2–6 assets, prices + financial text	Macro-RL	6-asset: AR=23.10%, SR=0.89, MDD=-21.10%; 2-asset SR=0.82	2-asset SR 0.82 vs EW 0.79, B&H 0.74; cross-modal BERT fusion > single-modal
Lu et al. (2025) (TrendFolios)	Multi-asset (Fixed Income, Equity, Alternatives; 2004–2023)	Macro-RL	Fixed Income 10Y: Excess Return 5.90%, Sharpe 0.33, IR 0.39; Equity: Sharpe 1.32; Alternatives: Sharpe 0.72	Sharpe 0.33 vs Index -0.14 (Fixed Income 10Y); positive IR across all periods; momentum + trend-following signals					
Abe et al. (2024)	40% US eq / 60% US bonds + macro (Oct 2021–Jan 2024)	Macro-LLM	SR=0.484 (ensemble)	SR +12–18% vs buy-and-hold during high-CPI periods; persona-driven LLMs	Chen et al. (2026b) (CN-Buzz2Portfolio)	Chinese macro/sector ETFs, 9 LLMs (2024–2025)	Macro-LLM	Best (Task B sector, 2024): Return=40.11%, SR=1.52 (Qwen3-Max-Think); Task A 2025 best SR=1.98	Reasoning LLMs beat CSI300 (16.20%) & EW in sector rotation; Tri-Stage rolling workflow
Nakayama et al. (2024)	Multi-asset (news + central-bank text)	Macro-LLM	Multi-asset correlation matrix from text	Addresses lag/nonstationarity of price-only correlation during regime shifts	Abro and Jaleel (2026)	S&P sector ETFs, LLM-guided clustering (ICAIF '25)	Macro-LLM	Cumulative return +68.6%, Sharpe +119% vs baseline; Sortino 1.19	LLM-guided smart clustering + sentiment trade filtering; no-regret (follow-the-leader) allocation
Straitouri et al. (2026)	Portfolio of optimization models	Macro-LLM	LLMs evolve allocation rule itself	Generates robust portfolios of optimization models, not just weights	Yuksel and Sawaf (2025)	Full S&P 500 (Long-term)	Macro-LLM	Peak Calmar >9.0, MDD 16–17% (Depth 0)	CR ≈3.8 vs baseline ≈2.2; continuous Calmar optimization; LLM tournament across hierarchical trees
Luo et al. (2025b)	Top 30 crypto + news (Jun 2023–Sep 2024)	Macro-LLM	SR=1.54, CR=2.4×	SR +38% vs fine-tuned GPT-4o; CR >117% vs 1/N; 50+ agent network	Ang et al. (2026)	18 liquid asset classes via ETFs (1996–2026)	Macro-LLM	ER=6.87%, Vol=7.54%, SR=0.39, MDD=-25.6%	MDD -34.3% → -25.6% vs 60/40 benchmark; 50+ agents; Borda count voting
Cho and Kim (2026)	LLM-based macro allocation benchmark	Macro-LLM	Benchmark infrastructure	Standardized evaluation for LLM-based macro allocation					
Markowitz (1952)	Single-period MVO baseline	Micro-Traditional	$\min_w w^\top \Sigma w$ s.t. $w^\top \mu = r$	Foundational paradigm underlying all modern variants	Ledoit and Wolf (2003)	High-dimensional covariance	Micro-Traditional	Analytically optimal shrinkage intensity	Pulls extreme eigenvalues toward grand mean; dramatically improves OOS stability
Boyd et al. (2024) (Markowitz++)	US S&P 100 stocks (2000–2023)	Micro-Traditional	AR=41.8%, Vol=8.8%, SR=4.65, MDD=6.4%	SR >24× vs basic Markowitz, >6× vs equal-weight; TC + holding + uncertainty in convex opt	Kocuk and Cornuéjols (2020)	S&P 500 11 GICS sectors (1987–2016)	Micro-Traditional	CVaR_M AR≈1.30%, 1% CVaR≈10.3%	Reduces CVaR by up to 11.6% vs market portfolio; mixture-of-normals + CVaR
Li and Hsieh (2023)	FF5 100 risky assets (2013–2022, TC=0%–1%)	Micro-Traditional	SR=0.68, Return=12.49%, σ =18.25%, MDD=26.34%, CR=0.47 (TC=0%)	SR 3× vs Dynamic MV w/o BL (0.22 → 0.68); MDD -42% (45.22% → 26.34%); combines dynamic BL with Elastic Net	Zhao (2023)	22 ETF factors (2020–2023)	Micro-Traditional	ER=3.68% (VIX), 2.17% (China Quality)	ER >0.5% avg per factor vs market-cap benchmark; LSTM forecasts factor performance
Abdelhakmi and Lim (2026) (DBL)	Synthetic 5 assets, S&P/ASX 50 cov (1-year horizon)	Micro-Traditional	Superior efficient frontiers across all rebalancing	Outperforms RCBL via dynamic hedging; robust to view revisions; continuous-time BL via Brownian bridge + Kalman	Soun et al. (STABLE)	Multi-market (S&P500, CSI300, EU-ROSTOXX, KOSPI200; 2013–2025)	Micro-Traditional	Sharpe=1.85 (S&P500), 2.92 (EUROSTOXX); MDD~3.84–7.82%, Vol~10.88–13.43%	vs. MVO: Sharpe 1.18 → 1.85 (+57%); vs. MetaTrader: ~1.00 → 1.85; conditional diffusion model

Continued on next page...

Table 11: State-of-the-Art Evaluation in Single-Period Portfolio Construction (Continued)

Model	Scope ($\mathcal{S}$)	Method ($\mathcal{M}$)	Metrics ($\mathcal{R}$)	Improvement	Model	Scope ($\mathcal{S}$)	Method ($\mathcal{M}$)	Metrics ($\mathcal{R}$)	Improvement
Li (2026) (KellyBoost)	8 assets (2003–2026)	Micro- Traditional	Growth-optimal (Kelly) allocation as XGBoost softmax output	Closed-form gradient/Hessians of $-\log(1 + w^\top y)$; bypasses predict- then-optimize	Garg (2026) (TDA+News)	S&P 500 constituents (rolling rebalancing)	Micro- Traditional	TDA-distance + FinBERT sentiment clustering; retention mechanism	Outperforms correlation/Euclidean- distance methods, Naive, index, and full-universe benchmarks
Nesterov and Ne- mirovskii (1994)	Interior-point MVO (up to few hundred assets)	Optimizer	Polynomial-time convergence	Theoretical foundation for interior-point methods regardless of initial conditions	DeMiguel et al. (2009)	Constrained portfolio optimization	Optimizer	ℓ_1, ℓ_2 , max-norm sparsity-stability tradeoffs	ℓ_1 induces sparse, low-turnover portfolios while maintaining convexity
Beck and Teboulle (2009) (FISTA)	Composite convex optimization	Optimizer	$O(1/k^2)$ convergence (vs. $O(1/k)$ PGD)	Nesterov-style extrapolation; faster convergence for ℓ_1 -regularized MVO	Allen-Zhu (2018)	Stochastic accelerated proximal gradient	Optimizer	$O((n + \sqrt{n\kappa}) \log(1/\epsilon))$ for strongly convex	Negative momentum; 10–100× fewer data passes than full-batch
Powell (1969)	Augmented Lagrangian foundation	Optimizer	Foundational for ADMM	Theoretical root of augmented Lagrangian methods	Kelley (1960)	Cutting-plane methods	Optimizer	Foundational for decomposition	Cutting-plane ideas underlying ADMM decomposition
Boyd et al. (2011) (ADMM)	Large-scale portfolios (10k+ assets)	Optimizer	Decomposes global problem into parallel subproblems	Industry standard for large-scale portfolio construction; definitive ADMM reference	Song et al. (2016) (Fast ADMM)	Large-scale portfolios	Optimizer	2–3× faster convergence vs. standard ADMM	Over-relaxation + adaptive penalty tuning; no additional computational cost
Ouyang et al. (2013) (Stochastic ADMM)	Online learning / factor model updating	Optimizer	Mini-batch ADMM for streaming data	Incremental refinement of factor exposures without full covariance recompute	Bot and Csetnek (2014) (Inertial ADMM)	Ill- conditioned multi-factor portfolios	Optimizer	30–50% additional speedup vs. Fast ADMM	Nesterov-style acceleration for highly correlated multi-factor portfolios
Amos and Kolter (2017) (OptNet)	QP as differentiable layer in NN	Optimizer	Gradients propagate through QP solver	Pioneered differentiable optimization; broke predict-then- optimize barrier	Agrawal et al. (2019) (cvxpylayers)	Disciplined convex programs	Optimizer	Analytical gradient $\nabla_\theta \phi(\theta)$ from dual variables	Generalizes differentiable opt to all DCPs; portfolio loss directly trains prediction model
Pan et al. (2024) (BPQP)	Batched QPs across time periods	Optimizer	20–50× speedups via GPU vectorization	Particularly valuable for backtesting thousands of historical QPs	Linghu et al. (2026) (dXPP)	Non-convex constraints (min position, cardinality)	Optimizer	Exact penalty preserving differentiability a.e.	Integrates realistic institutional constraints in end-to-end training
Elmachtoub and Grigas (2022) (SPO)	Decision- focused learning	Optimizer	Regret $\leq C\ \hat{\mu} - \mu\ _*$ (Hessian-induced norm)	Proves MSE is wrong proxy; regret-aligned losses yield better OOS portfolio despite higher MSE	Linghu et al. (2025) (IPMO)	Multi-period end-to-end	Optimizer	30–40% improvement in risk-adj. returns vs. single-period	Differentiable opt layer + prediction net + critic; eliminates two-stage loss
Wada et al. (2026) (Safe Screen- ing)	S&P 500, Russell 2000	Optimizer	Safe screening rules fix binary selection variables (no optimality loss)	Substantial speed-ups under moderate/strong regularization; ℓ_2 -perspective relaxation	Hsieh and Gan (2026) (Wasserstein DR)	476-asset monthly rebalancing (scales to 1,000)	Optimizer	Certified order-one Wasserstein DR as polynomial-size LP	Uniform utility- approximation error bounds; robustness to return mismatch
Jiang et al. (2017)	11 crypto (Nov 2014–Apr 2017)	Micro- RL	fAPV=29.695, SR=0.087, MDD=0.224	>4× fAPV, >2× SR vs best traditional RMR; model-free DRL with TC in policy gradient	Yang et al. (2020a)	DJIA 30 stocks (2009–2020)	Micro- RL	CR=70.4%, AR=13.0%, SR=1.30, MDD=-9.7%	>2.5× SR vs individual RL agents and baselines; PPO/A2C/DDPG ensemble + turbulence halt
Lleo and Rung- galdier (2026)	Benchmark- tracking investment management	Micro- RL	Risk-sensitive criterion via free energy-entropy duality	Two-player zero-sum stochastic differential game; partially model-based actor-critic; for pension/insurance	Choudhary et al. (2025) (RA- DRL)	Top 30 stocks in 4 global markets (2011–2024)	Micro- RL	CR=124.83%, AR=29.03%, SR=1.69 (Sensex)	CR significantly outperforms index (52.49%); SR > MVO (1.62) & EW; CNN merges log return + diff Sharpe + max DD agents
Yashaswi (2021) (LFSS)	15 S&P 500 stocks (2007–2019)	Micro- RL	AR=72.2%, SR=2.84, Sortino=3.04	AR +48%, SR +48% vs DRL baseline; Kalman + CNN-autoencoder for latent state	Li et al. (2024b)	Euro-futures + 6 features (2000–2020)	Micro- RL	PV=1.044, SR=1.17, MDD=0.30%	PV +1.8%, MDD -50% vs best single asset; PPO + Evolutionary Strategies under strict rules
Ndikum and Ndikum (2024) (AONS)	11 US equi- ties/indexes (2017–2023)	Micro- RL	AR=23.09%, SR=1.1459, Sortino=1.7244	SR +45% vs PPO; SR >6×, Sortino >7× vs SAC; AlphaOptimizerNet hard-codes constraints	Sood et al. (2026)	S&P 500 & 11 sectors (2006–2021)	Micro- RL	AR=12.11%, Vol=12.49%, SR=1.17, MDD=-32.96%	SR 0.68 → 1.17, AR 6.53% → 12.11% vs MVO baseline; lower turnover; differential Sharpe reward
Kang and Tian (2025) (RL- BHRP)	US equities 11 sector ETFs (2012–2025)	Micro- RL	CR=119.96%, SR=0.905, MDD=-20.33%	SR +7% vs static BHRP; AR +2.8pp, SR +19% vs EW sector; HRP penalties prevent over-concentration	Gurgul and Lessmann (2026)	15 diverse financial assets (Aug 2011–Sep 2025)	Micro- RL	SR=0.7281 (Quantum DDPG)	SR +66% vs equal-weight (0.4375), +23% vs MVO (0.5919); ~2,667× fewer parameters via quantum circuits

Continued on next page...

Table 11: State-of-the-Art Evaluation in Single-Period Portfolio Construction (Continued)

Model	Scope (S)	Method (M)	Metrics (R)	Improvement	Model	Scope (S)	Method (M)	Metrics (R)	Improvement
Kashif and Ľštepaczuk (2026)	NASDAQ-100, Nikkei 225, EURO STOXX 50 (2009–2026)	Micro-RL	NASDAQ-100 LSTM_NC_1: ARC=19.70%, Sharpe=0.8678, MDD=41.23%	ARC marginally beats B&H (19.27% → 19.70%); hierarchical Dirichlet policy + adaptive retrain when validation Sharpe < threshold	Hung et al. (2026) (Titans-QFWP)	468 S&P 500 stocks (equal-parameter benchmark, ~3,000 params)	Micro-RL	Median ARR=0.4260, Calmar=8.5504, IR=0.8427	QFWP + Titans memory in enhanced A3C; quantum gating reshapes Persistence/Surprise/Forgetting roles
Belyakov (2026) (AlphaZeroB3)	7 equity indices (2014–2024)	Micro-RL	Higher Sharpe vs baselines; near-zero benchmark correlation	CNN-GRU + Recurrent PPO; hard-enforced dollar neutrality via zero-sum projection	Mantshimuli and Muteba Mwamba (2025)	FNSPID 10 stocks S&P 500 (1999–2023)	Micro-LLM	AR=31.22%, SR=3.02, Omega=2.48	AR +6.65% vs cap-weighted; outperforms single LLM & emotion-free BL; LLM (GPT/Claude/Gemini) sentiment → BL confidence
Lee et al. (2025b)	Top 50 S&P 500 (Jun 2024–Jun 2025)	Micro-LLM	SR=1.23, CAGR=27.5% (BLM-Llama)	SR >3.5x vs EW baseline, >4.4x vs MVO; automated BL view generation	Anic et al. (2025)	S&P 500 + news (Oct 2019–Mar 2025)	Micro-LLM	AR=30%, Vol=22%, SR=1.06, MDD=-17%	SR 0.79 → 1.06, AR 24% → 30% vs traditional momentum baseline; ChatGPT gates cross-sectional momentum
Jajal et al. (2026)	Merger arbitrage (long-context)	Micro-LLM	LLM forecasts deal outcomes	Long-context (100s of pages) financial/legal docs; specialized event-driven trading	Coriat and Benhamou (2025) (HARLF)	14 assets + FinBERT sentiment (2003–2024)	Micro-LLM	AR=26.0%, SR=1.2	AR +55% vs S&P 500; surpasses CNN-RL; FinBERT for sentiment extraction + RL super-agent fusion
Caner et al. (2026)	S&P 500 (2020–2024, 3-layer agentic screening)	Micro-LLM	LLM+FinBERT intersection: Sharpe=1.1867, Return=36.34% (peak)	Sharpe +88% vs S&P 500 market (0.6324 → 1.1867); LLM-S + FinBERT intersection consensus > single-agent > human analysts	Yuksel (2025) (AlphaLoss)	3,246 US stocks/ETFs (15-year data)	Micro-LLM	AR=3.62%, SR=0.8812, MDD=22.08%	SR +56% vs EW baseline; stronger drawdown resistance; LLM evolves mathematical formulations of multi-objective loss
Yuan et al. (2026) (AlphaCrafter)	CSI 300 & S&P 500 (backtesting) + live trading, multi-LLM backbone)	Micro-LLM	CSI300 (Claude) SR=1.6732, AR=18.88%; S&P500 SR=1.3425; Live S&P500 SR=1.6012	Smallest cross-trial variance; harness decouples effectiveness from LLM choice; 3 specialized LLM agents in closed-loop	Li et al. (2025) (L-PPR)	>1,200 assets + macro (2015–2025)	Micro-LLM	AR=14.63%, SR=1.45, MDD=15.1%	AR +23.1%, SR +19.8%, MDD -17.5% vs best baseline (DRL-PPO); PPO state space includes user risk preferences from natural language chats
Pishehvar (2026)	10 tickers (AAPL et al.), 14-day walk-forward (Jun 2026)	Micro-LLM	MoE: +2.93% alpha vs EW; EW=-8.01%, SPY=-2.76%; daily win rate 42.9%	Zero-shot generalization; 4 specialized MoE heads (momentum/growth/defensive/tax-aware); 6-stage curriculum + 76-param LoRA personalization	Voronina et al. (2025)	11 S&P 500 sectors; 12 LLMs; OOS 2025-01-03 and 2025-04-06	Micro-LLM	NR: portfolio-Sharpe outcomes are categorical only; sector-index SR ranges from -0.146 to 0.120 (stable) and -0.065 to 0.134 (volatile)	In stable markets, most LLM portfolios beat their sector index in Sharpe
Asadi and Safabakhsh (2026) (Regime-Aware)	Crypto, stock, FX markets	Micro-LLM	Stock: cum. return 26% → 34%, Sharpe 0.74 → 0.96	Retrieval-augmented LLM expert selection; monotonicity: adding a locally superior expert cannot degrade the selector					

Multi-Period Portfolio Construction. Multi-period methods correspond to Section 6.3, explicitly modeling the path-dependent consequences of today’s weights on tomorrow’s feasible set. The field has evolved from pure mathematical optimality, through constraint-aware convex optimization, to end-to-end integration of prediction and optimization.

Table 12: State-of-the-Art Evaluation in Multi-Period Portfolio Construction

Model	Scope (S)	Method (M)	Metrics (R)	Improvement	Model	Scope (S)	Method (M)	Metrics (R)	Improvement
Dantzig and Infanger (1993)	Multi-stage stochastic programming under uncertainty	Foundational	Benders decomposition + importance sampling	Established algorithmic baseline for large-scale multi-stage stochastic programming	Zhou and Li (2000)	Continuous-time mean-variance efficient frontier	Foundational	Exact analytical solution via stochastic LQ control	Provided strict mathematical framework for continuous dynamic decision-making

Continued on next page...

Table 12: State-of-the-Art Evaluation in Multi-Period Portfolio Construction (Continued)

Model	Scope ($\mathcal{S}$)	Method ($\mathcal{M}$)	Metrics ($\mathcal{R}$)	Improvement	Model	Scope ($\mathcal{S}$)	Method ($\mathcal{M}$)	Metrics ($\mathcal{R}$)	Improvement
Boyd et al. (2017) (MPC)	Multi-period MPC with rolling-window prediction	Optimizer	Bypasses curse of dimensionality; fast computation	Convex optimization setup; rolling-window prediction keeps computation tractable	Li et al. (2022)	Risk parity in multi-period setting	Optimizer	Highly efficient sequential convex optimization algorithms	Adapts MPC framework for risk parity; sequential convex opt for risk-budgeted multi-period
Oprisor and Kwon (2020)	Regime-aware multi-period	Optimizer	HMM regime detection + Black-Litterman views	Robust against noise; subjective human views integrated via BL; regime-aware multi-period	Linghu et al. (2025) (IPMO)	Multi-period end-to-end	End-to-End	30–40% improvement in risk-adj. returns vs single-period	Differentiable optimization layer backpropagates gradients from investment performance; eliminates two-stage loss
Cui et al. (2024)	5 stock indices (2012–2022)	DRL	Sharpe 2.02 to 3.45; AR 31.61% to 246.11%	Narrows the action space via trading-strategy selection; outperforms DRL and state-of-the-art baselines	Yu and Chang (2020) (EFPM)	Macro-fundamental forecasting input	Neural-Aug	Forward-looking simulation inputs to optimization models	Neural forecasting framework feeds straight into multi-period optimizers
Halperin and Itkin (2026) (SciPhyRL)	14-asset ETF universe	Neural-Aug	Continuous-time HJB solved via physics-informed PINNs in a single sweep	Substantial OOS Sharpe improvements over static/myopic baselines; quadratic price-impact	Jeon et al. (2026) (Pontryagin)	Constrained multi-asset (up to 100 risky assets)	Neural-Aug	Adjoint-to-control recovery from BPTT graph; quadratic-affine recovery by QP	Reduces residual Pontryagin/KKT error; state-dependent consumption cap

7.3.5 Cross-setting Comparison

To enable direct horizontal comparison across models, we deliberately curate results from works that share the same data scope ($\mathcal{S}$) and reporting protocol. Specifically, we group comparable models into four widely adopted market scopes: a highly liquid single equity (Apple Inc.), the Chinese large-cap index (CSI 300), the Chinese mid-cap index (CSI 500), and the U.S. broad market index (S&P 500). Within each scope, we report standardized risk-adjusted metrics from the evaluation tuple $\mathcal{E} = (\mathcal{S}, \mathcal{O}, \mathcal{M}, \mathcal{R})$ introduced in Section 7, so that differences in reported numbers reflect algorithmic superiority rather than data or protocol mismatch.

Table 13: Performance Comparison on AAPL Single Stock

Model	Method	Sharpe	MDD
FinRL (2022)	RL-based	1.400	-21.13%
FinGPT (2023)	LLM-based	1.161	-16.76%
FinAgent (2024)	LLM-based	1.041	-19.90%
SEP (2024)	LLM-based	1.150	-
FinCon (2024)	LLM-based	1.597	-15.27%
FINMEM (2025)	LLM-based	0.994	-11.27%
Alpha-GPT (2025)	LLM-based	5.470	-2.36%
TradingAgents (2025)	LLM-based	8.210	-0.91%
Chen & Kawashima (2025)	LLM-based	2.000	-1.01%
Darmanin & Vella (2025)	LLM-based	1.700	-29.00%

Table 14: Performance Comparison on CSI 300 Index

Model	Method	IC	RankIC	ICIR	RankICIR	IR	MDD	Sharpe
GEP-Fin(2021)	GP-based	-	-	-	-	2.5000	-15.99%	2.3340

Continued on next page...

Table 14: Performance Comparison on CSI 300 Index (Continued)

Model	Method	IC	RankIC	ICIR	RankICIR	IR	MDD	Sharpe
AlphaGen (2023)	RL-based	0.0725	0.0806	-	-	-	-	-
TLRS(2024)	RL-based	0.0571	0.0582	-	-	-	-	-
Shin et al.(2024)	RL-based	0.0850	0.0870	-	-	-	-	-
QFR(2024)	RL-based	0.0588	0.0602	-	-	-	-	-
Alpha ² (2024)	RL-based	0.0576	0.0681	-	-	-	-	-
RiskMiner(2024)	RL-based	0.0645	0.0734	0.5126	-	-	-	-
CEGP(2025)	GP-based	-	-	-	-	-	-23.07%	1.3690
He et al. 2025	RL-based	-	-	-	-	-	-12.52%	1.9600
Alpha-R1(2025)	RL-based	-	-	-	-	-	-6.72%	1.6200
AlphaCon(2025)	RL-based	0.0425	0.0631	0.1755	0.2312	-	-	-
AlphaQCM(2025)	RL-based	0.0849	-	-	-	-	-	-
LLM4FTS(2025)	LLM-based	-	-	-	-	1.9650	-10.90%	-
CogAlpha(2025)	LLM-based	0.0591	0.0814	0.3410	0.4350	1.8999	-	-
Chain-of-Alpha(2025)	LLM-based	-	-	-	-	-	-	-
TreEvo(2025)	LLM-based	0.0615	0.0649	-	-	-	-	-
R&D-Agent-Quant(2025)	LLM-based	0.0532	0.0495	0.4278	0.4091	-	-7.42%	-
LLM-MTCS (2026)	LLM-based	0.0549	0.0512	-	-	1.1792	-	-
FactorEngine(2026)	LLM-based	0.0474	0.0475	0.3185	0.3146	1.6001	-12.61%	-
AlphaFormer(2026)	RL-based	0.0601	0.0695	-	-	-	-23.00%	0.8600
AlphaCFG(2026)	GP-based	0.0577	0.0865	0.4505	0.6036	-	-29.63%	0.6459
AlphaPROBE (2026)	GP-based	0.0584	0.0720	0.3902	0.4694	-	-22.25%	0.4411
AlphaSchema (2026)	LLM-based	0.0382	-	0.2374	-	1.0877	-	-

Table 15: Performance Comparison on CSI 500 Index

Model	Method	IC	RankIC	ICIR	RankICIR	IR	MDD	Sharpe
AlphaGen(2023)	RL-based	0.0438	0.0727	-	-	-	-	-
Alpha ² (2024)	RL-based	0.0612	0.0731	-	-	-	-	-
RiskMiner(2024)	RL-based	0.0596	0.0837	0.6420	-	-	-	-
TLRS(2024)	RL-based	0.0717	0.0730	-	-	-	-	-
Shin et al.(2024) (baseline)	LLM-based	0.0460	0.0769	0.2786	0.4711	1.2751	-	-
QFR(2024)	RL-based	0.0708	0.0674	-	-	-	-	-
AlphaCon(2025)	RL-based	0.0261	0.0518	0.1805	0.2860	-	-	-
AlphaQCM(2025)	RL-based	0.0955	-	-	-	-	-	-

Continued on next page...

Table 15: Performance Comparison on CSI 500 Index (Continued)

Model	Method	IC	RankIC	ICIR	RankICIR	IR	MDD	Sharpe
LLM4FTS(2025)	LLM-based	-	-	-	-	1.5910	-22.60%	-
CogAlpha(2025)	LLM-based	-	-	-	-	-	-	-
Chain-of-Alpha(2025)	LLM-based	0.0485	0.0771	0.3047	0.5013	1.4178	-	-
TreEvo(2025)	LLM-based	0.0742	0.0793	-	-	-	-	-
R&D-Agent-Quant(2025)	LLM-based	0.0288	-	0.1828	-	2.1721	-6.56%	-
AlphaAgent(2025)	LLM-based	0.0212	-	0.1938	-	1.4880	-9.36%	-
FactorMiner(2026)	LLM-based	0.0825	-	0.77	-	-	-	-
FactorEngine(2026)	LLM-based	0.0536	0.0487	0.4140	0.3744	0.6719	-21.51%	-
AlphaFormer(2026)	RL-based	0.0534	0.0582	-	-	-	-29.00%	0.9100
AlphaPROBE (2026)	GP-based	0.0626	0.0878	0.5239	0.7318	-	-22.98%	0.8262
AlphaLogics(2026)	LLM-based	0.0251	-	0.2312	-	1.5266	-5.31%	-

Table 16: Performance Comparison on S&P 500 Index

Model	Method	IC	RankIC	IR	MDD	Sharpe
Kou et al. (2023)	LLM-based	-	-	-	-11.52%	-
FinLlama(2024)	LLM-based	-	-	-	-	2.4000
FAMA(2024)	LLM-based	-	0.0540	-	-	6.6720
LLM4FTS(2025)	LLM-based	-	-	2.2580	-8.10%	-
FinDPO(2025)	LLM-based	-	-	-	-	3.4100
MarketSenseAI 2.0 (2025)	LLM-based	-	-	-	-12.50%	2.8700
AlphaAgent(2025)	LLM-based	0.0056	-	1.0545	-9.10%	-
TreEvo(2025)	LLM-based	0.0403	0.0543	-	-	-
AlphaCon(2025)	RL-based	0.0128	0.0186	-	-	-
HPPO-TO(2025)	RL-based	0.0719	0.0774	0.7266	-	-
AlphaCFG(2026)	GP-based	0.0457	0.0354	-	-19.42%	0.8473
LLM-MTCS(2026)	LLM-based	0.0132	0.0130	-	-	-
AlphaLogics(2026)	LLM-based	0.0093	-	1.2658	-9.06%	-

8 Discussion and Future Outlook

Across the Four-Step Multi-Factor Investing Framework surveyed in this paper, several cross-cutting challenges emerge that no single methodological paradigm has fully resolved. These challenges, spanning Factor Mining through Portfolio Management, include: the fragility of learned representations under Out-of-Distribution market regimes; the gap between offline backtest performance and live deployment results; the problem of semantic hallucinations in LLM-based reasoning; and the computational latency constraints in real-world trading. This section discusses these shared limitations and outlines three fundamental trajectories for future research.

8.1 From Quantitative Tools to Autonomous Trading Societies

The traditional, highly compartmentalized division of labor in quantitative research is undergoing a fundamental transformation toward full-pipeline autonomy. We foresee a paradigm shift where specialized human roles, such as data analysts, alpha researchers, risk managers, and execution traders, are entirely replaced by collaborative multi-agent algorithmic societies.

- **Autonomous Factor Researchers:** Future LLM-based agents will transcend simple text summarization to autonomously hypothesize Market Logic. They will continuously browse financial literature and news, translate abstract behavioral finance concepts into strict mathematical constraints, and compile executable formulaic alpha signals.
- **Synergistic Orchestration:** Instead of outputting massive, disjointed spatio-temporal matrices, future architectures will natively feature collaborative agents that dynamically aggregate these multiple mined factors into optimized, lower-dimensional combined factor sets, explicitly addressing multicollinearity dynamically.
- **Recursive Self-Improvement (RSI) Execution:** Concurrently, RL-based trading agents will complete the research-to-production loop by continuously interacting with live limit order books, adapting to micro-structural regime shifts and transaction friction without human intervention.

8.2 Towards a Unified Financial Foundation Model

Current state-of-the-art methodologies still heavily rely on decoupled, specialized models trained for isolated market modalities or distinct asset classes. This fragmentation severely limits global macro-awareness. The next critical frontier is the development of a **Unified Multi-modal Financial Foundation Model**.

- **Cross-Market & Cross-Asset Generalization:** A truly unified model must capture global macro-contagion and inter-market lead-lag effects seamlessly. It should maintain robust predictive capabilities across drastically different liquidity and volatility profiles, from established large-cap environments (e.g., the S&P 500) to highly volatile and retail-driven regimes (e.g., the CSI 500, CSI 1000, and global cryptocurrency markets).
- **All-Modality Fusion:** Future architectures will support the simultaneous ingestion of tick-level time-series data, news text, macroeconomic policy documents, and visual financial imagery (such as K-line charts). By embedding these diverse inputs into a shared latent space, the model will generate holistic, cross-modal trading strategies that human intuition cannot natively synthesize.

8.3 Standardization of Evaluation: An Upcoming Benchmark

As LLMs increasingly integrate into financial workflows, evaluation practices have not kept pace, and financial-specific biases systematically inflate measured performance and create illusions of deployability. Kong et al. (2026) identifies five recurrent biases in financial LLM applications: lookahead bias from future information leakage, survivorship bias from static universe construction, narrative bias from unsupported causal reasoning, objective bias from misaligned metrics, and implementation bias from ignoring costs and latencies. Reviewing 164 studies between 2023 and 2025, they find no bias is discussed in more than 28% of work, with lookahead bias appearing in only 26.8% and survivorship bias in just 1.2%. They propose a Structural Validity Framework requiring five minimal checks before results can support deployment claims: temporal sanitation to prevent future information leaks, dynamic universe construction for survivorship control, rationale robustness verifying evidence for model narratives, epistemic calibration requiring uncertainty outputs and allowing no-trade decisions, and realistic implementation constraints incorporating transaction costs, inference costs, slippage, and latency. A user study of 112 financial LLM researchers and practitioners finds that 74% consider off-the-shelf evaluation tools scarce or non-existent, with 50% identifying lack of evaluation frameworks as the largest bottleneck for bias mitigation. This position paper argues that financial LLM evaluation cannot rely on general language task metrics alone but requires structured validation

along time consistency, sample construction, evidence auditability, uncertainty expression, and net utility constraints.

However, a persistent bottleneck in the current literature is the lack of a unified, high-fidelity evaluation framework. Existing works often utilize disparate datasets, inconsistent backtesting horizons, and highly optimistic slippage assumptions, rendering objective horizontal comparisons between state-of-the-art methods virtually impossible.

To address this systemic flaw, we are actively developing a **comprehensive quantitative benchmark** aimed at systematizing the evaluation of AI-driven strategies. This framework will strictly evaluate models across the entire serial pipeline: from rigorous cross-sectional metrics in the mining phase to the resilience of the final portfolio. Furthermore, it will enforce standardized protocols for evaluating active risk control modules, specifically testing their accuracy in predicting future risk indicators, such as forward-looking volatility, VaR, and sudden crash probabilities, rather than relying solely on post-hoc reactive scaling.

8.4 Building an Agent-Centric LLM-Quant Ecosystem

Despite the cognitive capabilities of LLMs, a significant barrier to their industrial deployment is the lack of LLM-native infrastructure. Current financial APIs, databases, and research frameworks are engineered for human developers or deterministic scripts, not autonomous agents. To fully unlock the potential of LLMs in quantitative finance, we foresee the rapid emergence of an open-source, agent-centric ecosystem designed to bridge this operational gap. This ecosystem is expected to evolve across three foundational layers:

- **LLM-Native Data and Tool Protocols:** To eliminate the friction of data retrieval, the infrastructure is shifting towards agent-oriented databases (e.g., **LLMQuant Data**) that provide rich, structured research contexts rather than raw, disjointed data feeds. Crucially, the adoption of standardized tool-calling frameworks, such as the Model Context Protocol (**LLMQuant MCP**), will empower reasoning models (e.g., Claude Code, Codex, and customized agents) to autonomously invoke financial data APIs. This allows agents to seamlessly fetch real-time pricing, fundamentals, and alternative data directly into their reasoning trajectories without manual middleware.
- **Knowledge Grounding and Reusable Workflows:** To systematically combat financial hallucinations, autonomous agents must be anchored by high-density, domain-specific knowledge bases. Initiatives like **QuantWiki** (an open-source bilingual quantitative finance knowledge base) and **QuantPaper** (an AI-driven engine for paper discovery, semantic search, and knowledge card generation) will serve as standard Retrieval-Augmented Generation (RAG) backends, ensuring that agent hypotheses are grounded in peer-reviewed literature. Furthermore, the development of **LLMQuantSkills**, a repository of highly reusable AI financial research workflows, will transform ad-hoc prompting into standardized, auditable cognitive pipelines.
- **Next-Generation AI Financial Terminals:** The culmination of these infrastructural advancements will manifest in comprehensive, agent-driven platforms (e.g., **QuantMind**, **WallQuant**, and **Trader**). These next-generation tools will transcend traditional quantitative terminals by fusing AI-driven research, automated strategy execution, and continuous knowledge management into a unified workspace. By doing so, they will fundamentally redefine the interaction paradigm between human portfolio managers and artificial intelligence, shifting the human role from executing tasks to orchestrating intelligent systems.

9 Conclusion

This survey has systematically reviewed the evolution of AI-driven quantitative investment through the unifying lens of the Four-Step Multi-Factor Investing Framework:

$$X_t \xrightarrow{f} F_t \xrightarrow{\mathcal{T}} S_{raw,t} \xrightarrow{\mathcal{Z}} S_{pure,t} \xrightarrow{\mathcal{U}} W_t^*$$

Across these stages, we have traced clear evolutionary trajectories: Factor Mining has progressed from manual signal design to GP-based search, RL-guided policy search, and most recently LLM-driven semantic

reasoning; Factor Combination has moved from linear weighting to nonlinear aggregation and end-to-end synergistic optimization; Risk Control has evolved from static exposure limits to dynamic, forward-looking mechanisms that explicitly model regime changes and tail events; and Portfolio Management has shifted from classical convex solvers to heavily constrained MPO formulations, RL-based sequential decision-making, and fully agentic systems incorporating LLM-based reasoning. By organizing the literature through this unified framework, we provide researchers and practitioners with a structural blueprint for understanding how innovations propagate through the quantitative investment pipeline.

Looking forward, the most promising directions involve tighter integration across stages: end-to-end systems that jointly optimize factor mining and combination, mechanisms for upstream stages to incorporate downstream execution feedback, multi-agent architectures where specialized AI components collaborate across the pipeline, multi-modal information fusion, and improved robustness under distribution shifts and extreme market conditions. As these directions converge, quantitative investment systems may become increasingly adaptive, data-efficient, and capable of operating across complex, non-stationary financial environments.

References

- Quantitativefinance-bench: A state-aware interactive benchmark for financial agent tasks, 2026. URL <https://github.com/QF-Bench/QuantitativeFinance-Bench>.
- Anas Abdelhakmi and Andrew EB Lim. Dynamic black-litterman. *Operations Research*, 74(1):36–55, 2026.
- Yoshia Abe, Shuhei Matsuo, Ryoma Kondo, and Ryohei Hisano. Leveraging large language models for institutional portfolio management: Persona-based ensembles. In *2024 IEEE International Conference on Big Data (BigData)*, pages 4799–4808. IEEE, 2024.
- Muhammad Abro and Hassan Jaleel. Regret-driven portfolios: Llm-guided smart clustering for optimal allocation. *arXiv preprint arXiv:2601.17021*, 2026.
- Akshay Agrawal, Brandon Amos, Shane Barratt, Stephen Boyd, Steven Diamond, and J Zico Kolter. Differentiable convex optimization layers. *Advances in neural information processing systems*, 32, 2019.
- Taimoor Ahmed, Nikhar Azhar, Sahar Arshad, and Mehwish Fatima. Finai-trader: Sentiment-enhanced reinforcement learning for bitcoin trading in the finai contest 2025 task 1. In *2025 IEEE 12th International Conference on Cyber Security and Cloud Computing (CSCloud)*, pages 358–360. IEEE, 2025.
- Ch Hamza Akbar. Multi-agent deep reinforcement optimization for crypto trading. In *2025 IEEE 12th International Conference on Cyber Security and Cloud Computing (CSCloud)*, pages 335–337. IEEE, 2025.
- Franklin Allen and Risto Karjalainen. Using genetic algorithms to find technical trading rules. *Journal of financial Economics*, 51(2):245–271, 1999.
- Zeyuan Allen-Zhu. Katyusha: The first direct acceleration of stochastic gradient methods. *Journal of Machine Learning Research*, 18(221):1–51, 2018.
- Noguer I Alonso et al. Look-ahead bias in large language models (llms): Implications and applications in finance. *Look-Ahead Bias in Large Language Models (LLMs): Implications and Applications in Finance (November 15, 2024)*, 2024.
- Ali Atiah Alzahrani. Multi-agent regime-conditioned diffusion (marcd) for cvar-constrained portfolio decisions. *arXiv preprint arXiv:2510.10807*, 2025.
- Brandon Amos and J Zico Kolter. Optnet: Differentiable optimization as a layer in neural networks. In *International conference on machine learning*, pages 136–145. PMLR, 2017.
- Andrew Ang, Nazym Azimbayev, and Andrey Kim. The self driving portfolio: Agentic architecture for institutional asset management. *arXiv preprint arXiv:2604.02279*, 2026.
- Nikolas Anic, Andrea Barbon, Ralf Seiz, and Carlo Zarattini. Chatgpt in systematic investing—enhancing risk-adjusted returns with llms. *arXiv preprint arXiv:2510.26228*, 2025.

-
- Dogu Araci. Finbert: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063*, 2019.
- Hamidreza Arian, Mehrdad Moghimi, Ehsan Tabatabaei, and Shiva Zamani. Encoded value-at-risk: a predictive machine for financial risk management. *arXiv preprint arXiv:2011.06742*, 2020.
- Argimiro Arratia and Henryk Gzyl. An entropic factor model for robust portfolio replication. *arXiv preprint arXiv:2609.03552*, 2026.
- Ahmad Asadi and Reza Safabakhsh. Regime-aware portfolio management via retrieval-augmented llm-guided expert switching. *arXiv preprint arXiv:2608.28252*, 2026.
- Amir Beck and Marc Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM journal on imaging sciences*, 2(1):183–202, 2009.
- Ying L Becker, Peng Fei, and Anna M Lester. Stock selection: An innovative application of genetic programming methodology. In *Genetic Programming Theory and Practice IV*, pages 315–334. Springer, 2006.
- Boris Belyakov. Alphazero-beta: Deep reinforcement learning for market-neutral portfolios. *arXiv preprint arXiv:2607.18001*, 2026.
- Jennifer Bender and Taie Wang. Multi-factor portfolio construction for passively managed factor portfolios. In *Risk-Based and Factor Investing*, pages 435–447. Elsevier, 2015.
- Mostapha Benhenda. Finrl-deepseek: Llm-infused risk-sensitive reinforcement learning for trading agents. *arXiv preprint arXiv:2502.07393*, 2025.
- Gagan Bhatia, Hasan Cavusoglu, Muhammad Abdul-Mageed, et al. Fintral: A family of gpt-4 level multimodal financial large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 13064–13087, 2024.
- Fischer Black and Robert Litterman. Global portfolio optimization. *Financial analysts journal*, 48(5):28–43, 1992.
- David Blitz and Pim Van Vliet. Global tactical cross-asset allocation: Applying value and momentum across asset classes. Technical report, 2008.
- Radu Ioan Bot and Ernő Robert Csetnek. An inertial alternating direction method of multipliers. *arXiv preprint arXiv:1404.4582*, 2014.
- Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends® in Machine learning*, 3(1):1–122, 2011.
- Stephen Boyd, Steven Diamond, Kwangmoo Koh, Peter Nystrup, Enzo Busseti, Ronald N Kahn, and Jan Speth. Multi-period trading via convex optimization. *Foundations and Trends in Optimization*, 3(1):1–76, 2017.
- Stephen Boyd, Kasper Johansson, Ronald Kahn, Philipp Schiele, and Thomas Schmelzer. Markowitz portfolio construction at seventy. *arXiv preprint arXiv:2401.05080*, 2024.
- LEVY BRADFORD. Caution ahead: Numerical reasoning and look-ahead bias in ai models. *Journal of Accounting Research*, 64(3):1139–1188, 2026.
- Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.
- Hilal Anwar Butt, James W Kolari, and Mohsin Sadaqat. Market volatility, momentum, and reversal: a switching strategy. *Journal of Asset Management*, 25(5):460–478, 2024.
- Mehmet Caner, Agostino Capponi, Nathan Sun, and Jonathan Y Tan. Designing agentic ai-based screening for portfolio investment. *arXiv preprint arXiv:2603.23300*, 2026.

-
- Lang Cao. Chain-of-alpha: unleashing the power of large language models for alpha mining in quantitative trading. *arXiv preprint arXiv:2508.06312*, 2025.
- Yupeng Cao, Zhi Chen, Qingyun Pei, Nathan Jinseok Lee, KP Subbalakshmi, and Papa Momar Ndiaye. Ecc analyzer: Extract trading signal from earnings conference calls using large language model for stock performance prediction. *arXiv preprint arXiv:2404.18470*, 2024.
- Yupeng Cao, Zhi Chen, Prashant Kumar, Qingyun Pei, Yangyang Yu, Haohang Li, Fabrizio Dimino, Lorenzo Ausiello, KP Subbalakshmi, and Papa Momar Ndiaye. Risklabs: Predicting financial risk using large language model based on multimodal and multi-sources data. In *2025 IEEE International Conference on Data Mining Workshops (ICDMW)*, pages 901–909. IEEE, 2025.
- Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, et al. Extracting training data from large language models. In *30th USENIX security symposium (USENIX Security 21)*, pages 2633–2650, 2021.
- Adil Rengim Cetingoz and Olivier Guéant. Asset and factor risk budgeting: a balanced approach. *Quantitative Finance*, 25(2):181–195, 2025.
- Adil Rengim Cetingoz, Jean-David Fermanian, and Olivier Guéant. Risk budgeting portfolios: Existence and computation. *Mathematical Finance*, 34(3):896–924, 2024.
- Rong Chai, Vaclav Snasel, Xiaopeng Wang, Seyedali Mirjalili, and Crina Grosan. Llmde: A large language model-driven differential evolution algorithm for portfolio optimization. *arXiv preprint arXiv:2609.16846*, 2026.
- Sara Chehab, Giorgos Iacovides, Parisa Yazdanparast, and Danilo Mandic. Beyond co-movement: Locality by exposures enables a joint factor-graph framework for portfolio diversification. *arXiv preprint arXiv:2608.06618*, 2026.
- Binqi Chen, Hongjun Ding, Ning Shen, Jinsheng Huang, Taian Guo, Luchen Liu, and Ming Zhang. Alphasage: Structure-aware alpha mining via gflownets for robust exploration. *arXiv preprint arXiv:2509.25055*, 2025a.
- Jian Chen, Guohao Tang, Guofu Zhou, and Wu Zhu. Chatgpt and deepseek: Can they predict the stock market and macroeconomy? *arXiv preprint arXiv:2502.10008*, 2025b.
- Jiaxiang Chen, Mingxi Zou, Zhuo Wang, Qifan Wang, Dongning Sun, Chi Zhang, and Zenglin Xu. Finhear: Human expertise and adaptive risk-aware temporal reasoning for financial decision-making. *arXiv preprint arXiv:2506.09080*, 2025c.
- Junyu Chen, Tom Boot, Lingwei Kong, and Weining Wang. Transformer-based covar: Systemic risk in textual information. *arXiv preprint arXiv:2602.12490*, 2026a.
- Liyuan Chen, Shilong Li, Jiangpeng Yan, Shuoling Liu, Qiang Yang, and Xiu Li. Cn-buzz2portfolio: A chinese-market dataset and benchmark for llm-based macro and sector asset allocation from daily trending financial news. *arXiv preprint arXiv:2603.22305*, 2026b.
- Qizhao Chen and Hiroaki Kawashima. Adaptive alpha weighting with ppo: Enhancing prompt-based llm-generated alphas in quant trading. *International Journal of Data Science and Analytics*, 22(1):104, 2026.
- Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.
- Tianxiang Chen, Wei Chen, and Luyao Du. An empirical study of financial factor mining based on gene expression programming. In *2021 4th International Conference on Advanced Electronic Materials, Computers and Software Engineering (AEMCSE)*, pages 1113–1117. IEEE, 2021.

-
- Wei Chen, Qiushi Wang, Zefei Long, Xianyin Zhang, Zhongtian Lu, Bingxuan Li, Siyuan Wang, Jiarong Xu, Xiang Bai, Xuanjing Huang, et al. Disc-finllm: A chinese financial large language model based on multiple experts fine-tuning. *arXiv preprint arXiv:2310.15205*, 2023a.
- Yan Chen and Kotaro Hirasawa. A portfolio selection model using genetic relation algorithm and genetic network programming. *IEEE transactions on electrical and electronic engineering*, 6(5):403–413, 2011.
- Yan Chen and Xuancheng Wang. A hybrid stock trading system using genetic network programming and mean conditional value-at-risk. *European Journal of Operational Research*, 240(3):861–871, 2015.
- Yan Chen, Shingo Mabui, and Kotaro Hirasawa. A model of portfolio optimization using time adapting genetic network programming. *Computers & operations research*, 37(10):1697–1707, 2010.
- Yanxu Chen, Zijun Yao, Yantao Liu, Amy Xin, Jin Ye, Jianing Yu, Lei Hou, and Juanzi Li. Stockbench: Can llm agents trade stocks profitably in real-world markets? *arXiv preprint arXiv:2510.02209*, 2025d.
- Zihan Chen, Lei Nico Zheng, Cheng Lu, Jialu Yuan, and Di Zhu. Chatgpt informed graph neural network for stock movement prediction. *arXiv preprint arXiv:2306.03763*, 2023b.
- Yuhan Cheng and Ke Tang. Gpt’s idea of stock factors. *Quantitative Finance*, 24(9):1301–1326, 2024.
- Hanyong Cho and Jang Ho Kim. Constructing a portfolio optimization benchmark framework for evaluating large language models. *arXiv preprint arXiv:2603.09301*, 2026.
- Jaehyung Choi. Entropic value-at-risk portfolio optimization for tempered stable Levy processes. *arXiv preprint arXiv:2608.18022*, 2026.
- Himanshu Choudhary, Arishi Orra, Kartik Sahoo, and Manoj Thakur. Risk-adjusted deep reinforcement learning for portfolio optimization: A multi-reward approach. *International Journal of Computational Intelligence Systems*, 18(1):126, 2025.
- Benjamin Coriat and Eric Benhamou. Harlf: Hierarchical reinforcement learning and lightweight llm-driven sentiment integration for financial portfolio optimization. *arXiv preprint arXiv:2507.18560*, 2025.
- Peter Cotton. Schur complementary allocation: A unification of hierarchical risk parity and minimum variance portfolios. *arXiv preprint arXiv:2411.05807*, 2024.
- Can Cui, Wei Wang, Meihui Zhang, Gang Chen, Zhaojing Luo, and Beng Chin Ooi. Alphaevolve: A learning framework to discover novel alphas in quantitative investment. In *Proceedings of the 2021 International conference on management of data*, pages 2208–2216, 2021.
- Tianxiang Cui, Nanjiang Du, Xiaoying Yang, and Shusheng Ding. Multi-period portfolio optimization using a deep reinforcement learning hyper-heuristic approach. *Technological Forecasting and Social Change*, 198: 122944, 2024.
- George B Dantzig and Gerd Infanger. Multi-stage stochastic linear programs for portfolio optimization. *Annals of Operations Research*, 45(1):59–76, 1993.
- Adam Darmanin and Vince Vella. Language model guided reinforcement learning in quantitative trading. In *2025 3rd International Conference on Foundation and Large Language Models (FLLM)*, pages 405–412. IEEE, 2025.
- S. R. Das, Tanmay Sen, Raghu Nandan Sengupta, and A. Gupta. Deep reinforcement learning for reliability based bi-objective portfolio optimization. *arXiv preprint arXiv:2607.06610*, 2026.
- Victor DeMiguel, Lorenzo Garlappi, Francisco J Nogales, and Raman Uppal. A generalized approach to portfolio optimization: Improving performance by constraining portfolio norms. *Management science*, 55(5):798–812, 2009.
- Yu-Chen Den, Kuan-Yu Chen, Kendro Vincent, and Tien-Hao Chang. Maple: Efficient and diverse multi-alpha generation for portfolio construction. *arXiv preprint arXiv:2607.24131*, 2026.

-
- Zheyue Deng and Jiashu Wang. Alphaquant: An end-to-end tool-orchestrated agentic reinforcement learning framework for stock trading. *arXiv preprint arXiv:2510.14264*, 2025.
- Hongjun Ding, Binqi Chen, Jinsheng Huang, Taian Guo, Zhengyang Mao, Guoyi Shao, Lutong Zou, Luchen Liu, and Ming Zhang. Alphaeval: A comprehensive and efficient evaluation framework for formula alpha mining. *arXiv preprint arXiv:2508.13174*, 2025.
- Giovanni Dispoto, Marcello Restelli, and Carmine Ventre. Eliciting esg preferences for reinforcement learning-based portfolio optimization. *arXiv preprint arXiv:2609.02677*, 2026.
- Martin Dudler, Bruno Gmür, and Semyon Malamud. Momentum and risk adjustment. *The Journal of Alternative Investments*, 18(2):91, 2015.
- Adam N Elmachtoub and Paul Grigas. Smart “predict, then optimize”. *Management Science*, 68(1):9–26, 2022.
- Fang Fang, Xiaoyu Shen, and Qinling Wang. Importance sampling enhanced with the cos method for the portfolio risk allocation. *arXiv preprint arXiv:2608.30749*, 2026.
- George Fatouros, Kostas Metaxas, John Soldatos, and Manos Karathanassis. Marketsenseai 2.0: Enhancing stock analysis through llm agents. *arXiv preprint arXiv:2502.00415*, 2025.
- Georgios Fatouros, Konstantinos Metaxas, John Soldatos, and Dimosthenis Kyriazis. Can large language models beat wall street? unveiling the potential of ai in stock selection. *arXiv preprint arXiv:2401.03737*, 2024.
- Guanhao Feng, Liang Jiang, Junye Li, Yizhi Song, and Yuanzhi Wang. Deep tangency portfolio. *Available at SSRN 3971274*, 2023.
- Zhenyu Gao, Wenxi Jiang, and Yutong Yan. Detecting lookahead bias in llm forecasts. *arXiv preprint arXiv:2512.23847*, 2025.
- Divyanee Garg. Portfolio optimization under dynamic rebalancing via topological data analysis and news sentiments. *arXiv preprint arXiv:2607.21170*, 2026.
- Alex Garivaltis. Universal risk budgeting. *Annals of Financial Economics*, 16(03):2150014, 2021.
- Roberto Garrone. Dynamic inclusion and bounded multi-factor tilts for robust portfolio construction. *arXiv preprint arXiv:2601.05428*, 2026.
- Ignas Gasparavičius and Andrius Grigutis. Generalizing markowitz portfolio optimization by a quadratic risk measure. *arXiv preprint arXiv:2608.24449*, 2026.
- Marek Grzesiak and Raghav Aggarwal. Securefinai: Integrating llm-generated signals with reinforcement learning for high-frequency crypto trading. 2025.
- Fengchen Gu, Zhengyong Jiang, Ángel F García-Fernández, Angelos Stefanidis, Jionglong Su, and Huakang Li. Mts: A deep reinforcement learning portfolio management framework with time-awareness and short-selling. *Intelligent Data Analysis*, page 1088467X261416593, 2025.
- Jiacheng Guo, Suozhi Huang, Yunlong Gao, Zihao Li, Jian Ge, Xu Kuang, and Mengdi Wang. Aqua: Recursively self-improving quantitative trading research agents. *arXiv preprint arXiv:2608.12841*, 2026a.
- Taian Guo, Haiyang Shen, Junyu Luo, Binqi Chen, Hongjun Ding, Jinsheng Huang, Luchen Liu, Yun Ma, and Ming Zhang. Alphaprobe: Alpha mining via principled retrieval and on-graph biased evolution. *arXiv preprint arXiv:2602.11917*, 2026b.
- Yue Guo, Zian Xu, and Yi Yang. Is chatgpt a financial expert? evaluating language models on financial natural language processing. In *Findings of the association for computational linguistics: EMNLP 2023*, pages 815–821, 2023.

-
- Udit Gupta. Gpt-investar: Enhancing stock investment strategies through annual report analysis with large language models. *arXiv preprint arXiv:2309.03079*, 2023.
- Vincent Gurgul and Stefan Lessmann. Variational quantum circuit-based reinforcement learning for dynamic portfolio optimization. *arXiv preprint arXiv:2601.18811*, 2026.
- Igor Halperin and Andrey Itkin. Sciphy reinforcement learning for portfolio optimization. *arXiv preprint arXiv:2607.15195*, 2026.
- Jun Han, Shuo Zhang, Wei Li, Zhi Yang, Yifan Dong, Tu Hu, Jialuo Yuan, Xiaomin Yu, Yumo Zhu, Fangqi Lou, et al. Quantaalpha: An evolutionary framework for llm-driven alpha mining. *arXiv preprint arXiv:2602.07085*, 2026.
- Shijie Han, Jingshu Zhang, Yiqing Shen, Kaiyuan Yan, and Hongguang Li. Finsphere, a real-time stock analysis agent powered by instruction-tuned llms and domain tools. *arXiv preprint arXiv:2501.12399*, 2025.
- Anton Hantel. Small bias, large failure: Anchoring collapses a market of language-model agents. *Large Failure: Anchoring Collapses a Market of Language-Model Agents (June 30, 2026)*, 2026.
- Chujun He, Zhonghao Huang, Xiangguo Li, Ye Luo, Kewei Ma, Yuxuan Xiong, Xiaowei Zhang, and Mingyang Zhao. Hierarchical ai multi-agent fundamental investing: Evidence from china’s a-share market. *arXiv preprint arXiv:2510.21147*, 2025.
- J Vernon Henderson, Adam Storeygard, and David N Weil. Measuring economic growth from outer space. *American Economic Review*, 102(2):994–1028, 2012. doi: 10.1257/aer.102.2.994.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- Chung-Han Hsieh and Rong Gan. Certified high-dimensional wasserstein robust portfolio optimization. *arXiv preprint arXiv:2608.07032*, 2026.
- Anran Hu, Silvana M. Pesenti, and Xiaofei Shi. Dynamic portfolio optimization under cvar constraints. *Available at SSRN 7344098*, 2026.
- Xiaoyu Hu and Jinman Zhao. Fin-bias: Comprehensive evaluation for llm decision-making under human bias in finance domain. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 5678–5694, 2026.
- Allen Yikuan Huang and Zheqi Fan. Beyond prompting: An autonomous framework for systematic factor investing via agentic ai. *arXiv preprint arXiv:2603.14288*, 2026.
- Chien Yi Huang. Financial trading as a game: A deep reinforcement learning approach. *arXiv preprint arXiv:1807.02787*, 2018.
- Haotong Huang, Jie Peng, Zezhen Ding, Pingzhi Li, and Tianlong Chen. Alphaformer: End-to-end symbolic regression of alpha factors with transformers.
- Jimin Huang, Mengxi Xiao, Dong Li, Zihao Jiang, Yuzhe Yang, Yifei Zhang, Lingfei Qian, Yan Wang, Xueqing Peng, Yang Ren, et al. Open-finllms: Open multimodal large language models for financial applications. *arXiv preprint arXiv:2408.11878*, 2024.
- Jeonggyu Huh, Jaegi Jeon, Hyeng Keun Koo, and Byung Hwa Lim. Breaking the dimensional barrier: A pontryagin-guided direct policy optimization for continuous-time multi-asset portfolio choice. *arXiv preprint arXiv:2504.11116*, 2025.
- Ming-Kai Hung, Jun-Hao Chen, Yun-Cheng Tsai, and Samuel Yen-Chi Chen. Titans-qfwp: A regime-aware hybrid quantum fast weight programmer for portfolio optimization. *arXiv preprint arXiv:2608.29093*, 2026.

-
- Giorgos Iacovides, Wuyang Zhou, and Danilo Mandic. Findpo: Financial sentiment analysis for algorithmic trading through preference optimization of llms. In *Proceedings of the 6th ACM International Conference on AI in Finance*, pages 647–655, 2025.
- Martin Arnaiz Iglesias, Adil Rengim Cetingoz, and Noufel Frikha. Mirror descent algorithms for risk budgeting portfolios. *arXiv preprint arXiv:2411.12323*, 2024.
- Hinal Jajal, Michal Mucha, Charles Sweat, Chris Pulman, Charlie Flanagan, and Peter Anderson. Global merger-arbitrage forecasting with language models. *arXiv preprint arXiv:2607.09921*, 2026.
- Jaegi Jeon, Jeonggyu Huh, Hyeng Keun Koo, and Byung Hwa Lim. Scalable pontryagin-guided adjoint-to-control recovery for constrained dynamic portfolio choice. *arXiv preprint arXiv:2608.15667*, 2026.
- Amit Kumar Jha. Beyond var and cvar: Topological risk measures in financial markets. *arXiv preprint arXiv:2310.14604*, 2023.
- Renjun Jia, Zian Liu, Peng Zhu, Dawei Cheng, and Yuqi Liang. Beyond fixed patches: Enhancing gpts for financial prediction with adaptive segmentation and learnable wavelets. *arXiv preprint arXiv:2505.02880*, 2025.
- Fuwei Jiang, Kunpeng Li, Wei Ning, and Hao Xue. Scaled factor portfolio. *Pacific-Basin Finance Journal*, page 103161, 2026a.
- Shuyang Jiang, Yuhao Wang, Ya Zhang, Yanfeng Wang, and Yu Wang. Miner: Mining intrinsic mastery for data-efficient rl in large reasoning models. *arXiv preprint arXiv:2601.04731*, 2026b.
- Zhengyao Jiang, Dixin Xu, and Jinjun Liang. A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*, 2017.
- Zuoyou Jiang, Li Zhao, Rui Sun, Ruohan Sun, Zhongjian Li, Jing Li, Daxin Jiang, Zuo Bai, and Cheng Hua. Alpha-r1: Alpha screening with llm reasoning via reinforcement learning. *arXiv preprint arXiv:2512.23515*, 2025.
- Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, et al. Time-llm: Time series forecasting by reprogramming large language models. In *International conference on learning representations*, volume 2024, pages 23857–23880, 2024.
- Hibiki Kaibuchi, Yoshinori Kawasaki, and Gilles Stupfler. Garch-ugh: a bias-reduced approach for dynamic extreme value-at-risk estimation in financial time series. *Quantitative Finance*, 22(7):1277–1294, 2022.
- Zura Kakushadze. 101 formulaic alphas. *Wilmott*, 2016(84):72–81, 2016.
- Michael Kampouridis and Fernando EB Otero. Evolving trading strategies using directional changes. *Expert Systems with Applications*, 73:145–160, 2017.
- Shaofeng Kang and Zeying Tian. Optimal portfolio construction—a reinforcement learning embedded bayesian hierarchical risk parity (rl-bhrp) approach. *arXiv preprint arXiv:2508.11856*, 2025.
- Kamil Kashif and Robert Łślepaczuk. Deep reinforcement learning framework for diversified portfolio management across global equity markets. *arXiv preprint arXiv:2605.17307*, 2026.
- Zsolt Katona, Marcus O Painter, Panos N Patatoukas, and Jean Zeng. On the capital market consequences of big data: Evidence from outer space. *Journal of Financial and Quantitative Analysis*, 60(2):551–579, 2024. doi: 10.1017/s0022109023001448.
- Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems*, 30, 2017.
- James E Kelley, Jr. The cutting-plane method for solving convex programs. *Journal of the society for Industrial and Applied Mathematics*, 8(4):703–712, 1960.

-
- Natthawut Kertkeidkachorn, Rungsiman Nararatwong, Ziwei Xu, and Ryutaro Ichise. Finkg: a core financial knowledge graph for financial analysis. In *2023 IEEE 17th International Conference on Semantic Computing (ICSC)*, pages 90–93. IEEE, 2023.
- Tina Khezresmaeilzadeh, Parsa Razmara, Seyedarmin Azizi, Mohammad Erfan Sadeghi, and Erfan Baghaei Potraghloo. Vista: Vision-language inference for training-free stock time-series analysis. *arXiv preprint arXiv:2505.18570*, 2025.
- Hyeonjin Kim, Minseok Kim, Seunghyeon Jung, Sujin Pyo, Huisu Jang, and Woojin Lee. Favor: Llm-based agentic framework for factor mining via empirical validation. *arXiv preprint arXiv:2608.30192*, 2026.
- Kemal Kirtac and Guido Germano. Sentiment trading with large language models. *Finance Research Letters*, 62:105227, 2024.
- Kelvin J. L. Koa, Yunshan Ma, Yi Xu, Ritchie Ng, Huanhuan Zheng, and Tat-Seng Chua. Temporal relational reasoning of large language models for detecting stock portfolio crashes. In *arXiv preprint arXiv:2410.17266*, 2024a.
- Kelvin JL Koa, Yunshan Ma, Ritchie Ng, and Tat-Seng Chua. Learning to generate explainable stock predictions using self-reflective large language models. In *Proceedings of the ACM web conference 2024*, pages 4304–4315, 2024b.
- Kelvin JL Koa, Jan Chen, Yunshan Ma, Huanhuan Zheng, and Tat-Seng Chua. Reasoning on time-series for financial technical analysis. *arXiv preprint arXiv:2511.08616*, 2025.
- Burak Kocuk and Gérard Cornuéjols. Incorporating black-litterman views in portfolio construction when stock returns are a mixture of normals. *Omega*, 91:102008, 2020.
- Yaxuan Kong, Hoyoung Lee, Yoontae Hwang, Alejandro Lopez-Lira, Bradford Levy, Dhagash Mehta, Qingsong Wen, Chanyeol Choi, Yongjae Lee, and Stefan Zohren. Position: Evaluating llms in finance requires explicit bias consideration. In *Forty-third International Conference on Machine Learning Position Paper Track*, 2026.
- Thanos Konstantinidis, Giorgos Iacovides, Mingxue Xu, Tony G Constantinides, and Danilo Mandic. Finllama: Financial sentiment classification for algorithmic trading applications. *arXiv preprint arXiv:2403.12285*, 2024.
- Zhizhuo Kou, Holam Yu, Junyu Luo, Jingshu Peng, Xujia Li, Chengzhong Liu, Juntao Dai, Lei Chen, Sirui Han, and Yike Guo. Automate strategy finding with llm in quant investment. *seed*, 1:3, 2024.
- Olivier Ledoit and Michael Wolf. Honey, i shrunk the sample covariance matrix. *UPF economics and business working paper*, (691), 2003.
- Hoyoung Lee, Junhyuk Seo, Suhwan Park, Junhyeong Lee, Wonbin Ahn, Chanyeol Choi, Alejandro Lopez-Lira, and Yongjae Lee. Your ai, not your view: The bias of llms in investment analysis. In *Proceedings of the 6th ACM International Conference on AI in Finance*, pages 150–158, 2025a.
- Younghbin Lee, Yejin Kim, Juhyeong Kim, Suin Kim, and Yongjae Lee. Llm-enhanced black-litterman portfolio optimization. *arXiv preprint arXiv:2504.14345*, 2025b.
- Bangyu Li, Boping Gu, and Ziyang Ding. Llm-based personalized portfolio recommender: Integrating large language models and reinforcement learning for intelligent investment strategy optimization. *arXiv preprint arXiv:2512.12922*, 2025.
- Changlun Li, Peixian Ma, Qiqi Duan, Zhenyu Lin, and Peineng Wu. Nextfund: A unified performance tracking platform for agentic portfolio management. *arXiv preprint arXiv:2607.11141*, 2026a.
- Chi-Lin Li and Chung-Han Hsieh. On unified adaptive portfolio management. *arXiv preprint arXiv:2307.03391*, 2023.

-
- Dapeng Li, Feiyang Pan, Jia He, Zhiwei Xu, Dandan Tu, and Guoliang Fan. Style miner: Find significant and stable explanatory factors in time series with constrained reinforcement learning. *arXiv preprint arXiv:2303.11716*, 2023a.
- Jiayu Li. Kellyboost: Growth-optimal portfolio construction with gradient-boosted trees. *arXiv preprint arXiv:2608.23393*, 2026.
- Qi Li, Norshaliza Kamaruddin, Siti Sophiayati Yuhaniz, and Hamdan Amer Ali Al-Jaifi. Forecasting stock prices changes using long-short term memory neural network with symbolic genetic programming. *Scientific reports*, 14(1):422, 2024a.
- Xiaoyue Li, A Sinem Uysal, and John M Mulvey. Multi-period portfolio optimization using model predictive control with mean-variance and risk parity frameworks. *European Journal of Operational Research*, 299(3):1158–1176, 2022.
- Xinyi Li, Yinchuan Li, Yuancheng Zhan, and Xiao-Yang Liu. Optimistic bull or pessimistic bear: Adaptive deep reinforcement learning for stock portfolio allocation. *arXiv preprint arXiv:1907.01503*, 2019.
- Yang Li, Yangyang Yu, Haohang Li, Zhi Chen, and Khaldoun Khashanah. Tradinggpt: Multi-agent system with layered memory and distinct characters for enhanced financial trading performance. *arXiv preprint arXiv:2309.03736*, 2023b.
- Yinheng Li, Junhao Wang, and Yijie Cao. A general framework on enhancing portfolio management with reinforcement learning. In *Proceedings of the 2024 7th International Conference on Computers in Management and Business*, pages 119–125, 2024b.
- Yuan Li, Bingqiao Luo, Qian Wang, Nuo Chen, Xu Liu, and Bingsheng He. A reflective llm-based agent to guide zero-shot cryptocurrency trading.(6 2024). URL <https://arxiv.org/abs/2407.09546>, 2024c.
- Yuante Li, Xu Yang, Xiao Yang, Xisen Wang, Weiqing Liu, and Jiang Bian. R&d-agent-quant: a multi-agent framework for data-centric factors and model joint optimization. *Advances in Neural Information Processing Systems*, 38, 2026b.
- Zhiwei Li, Ran Song, Caihong Sun, Wei Xu, Zhengtao Yu, and Ji-Rong Wen. Can large language models mine interpretable financial factors more effectively? a neural-symbolic factor mining agent model. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3891–3902, 2024d.
- Chuan Liang. Look-ahead bias in financial forecasts generated by large language models. *Available at SSRN 6772819*, 2024.
- Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- Bryan Lim, Sercan Ö Arık, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International journal of forecasting*, 37(4):1748–1764, 2021.
- Qinhong Lin, Ruitao Feng, Yinglun Feng, Zhenxin Huang, Yukun Chen, Zhongliang Yang, Linna Zhou, Binjie Fei, Jiaqi Liu, and Yu Li. Factorengine: A program-level knowledge-infused factor mining framework for quantitative investment. *arXiv preprint arXiv:2603.16365*, 2026.
- Yuxuan Linghu, Zhiyuan Liu, and Qi Deng. Integrated prediction and multi-period portfolio optimization. *arXiv preprint arXiv:2512.11273*, 2025.
- Yuxuan Linghu, Zhiyuan Liu, and Qi Deng. A penalty approach for differentiation through black-box quadratic programming solvers. *arXiv preprint arXiv:2602.14154*, 2026.
- Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. *arXiv preprint arXiv:2401.02051*, 2024a.

-
- Fengyuan Liu, Huang Yi, Sichun Luo, Yuqi Wang, Yazheng Yang, Xinye Li, Zefa Hu, Junlan Feng, and Qi Liu. Cognitive alpha mining via llm-driven code-based evolution. *arXiv preprint arXiv:2511.18850*, 2025.
- Fengyuan Liu, Yuchen Fu, Yuqi Wang, and Qi Liu. Xalpha: A memory-driven ai quant researcher for hypothesis-to-code alpha discovery. *arXiv preprint arXiv:2607.08332*, 2026.
- Xiao-Yang Liu, Zhuoran Xiong, Shan Zhong, Hongyang Yang, and Anwar Walid. Practical deep reinforcement learning approach for stock trading. *arXiv preprint arXiv:1811.07522*, 2018.
- Xiao-Yang Liu, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. Finrl: A deep reinforcement learning library for automated stock trading in quantitative finance. *arXiv preprint arXiv:2011.09607*, 2020.
- Xiao-Yang Liu, Jie Zhang, Guoxuan Wang, Weiqing Tong, and Anwar Walid. Fingpt-hpc: Efficient pre-training and finetuning large language models for financial applications with high-performance computing. *arXiv preprint arXiv:2402.13533*, 2024b.
- Yong Liu, Guo Qin, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Autotimes: Autoregressive time series forecasters via large language models. *Advances in Neural Information Processing Systems*, 37: 122154–122184, 2024c.
- Sebastien Lleo and Wolfgang Runggaldier. Reinforcement learning for risk-sensitive investment management: a free energy–entropy duality approach. *arXiv preprint arXiv:2606.20903*, 2026.
- Dome Lohpetch and David Corne. Outperforming buy-and-hold with evolved technical trading rules: Daily, weekly and monthly trading. In *European Conference on the Applications of Evolutionary Computation*, pages 171–181. Springer, 2010.
- Joseph Lu, Randall R Rojas, Fiona C Yeung, and Patrick D Convery. Trendfolios: A portfolio construction framework for utilizing momentum and trend-following in a multi-asset portfolio. *arXiv preprint arXiv:2506.09330*, 2025.
- Haochen Luo, Yuan Zhang, and Chen Liu. Efs: Evolutionary factor searching for sparse portfolio optimization using large language models. *arXiv preprint arXiv:2507.17211*, 2025a.
- Wei Luo and Dihong Gong. Pre-trained large language models for financial sentiment analysis. *arXiv preprint arXiv:2401.05215*, 2024.
- Yichen Luo, Yebo Feng, Jiahua Xu, Paolo Tasca, and Yang Liu. Llm-powered multi-agent system for automated crypto portfolio management. *arXiv preprint arXiv:2501.00826*, 2025b.
- Yin Luo, Sheng Wang, and Javed Jussa. Dynamic allocation: extremes, tail dependence, and regime shifts. *arXiv preprint arXiv:2506.12587*, 2025c.
- Sébastien Maillard, Thierry Roncalli, and Jérôme Teïletche. The properties of equally weighted risk contribution portfolios. *Journal of portfolio management*, 36(4):60, 2010.
- Lamu Mantshimuli and John Weistrass Muteba Mwamba. Enhancing portfolio optimization with multi-llm sentiment aggregation: A black-litterman integration approach. *Available at SSRN 5394743*, 2025.
- Jie Mao, Changlun Li, Xiang Li, Q. Duan, Jiayue Yuan, Xiang Liu, Yuyu Luo, Jing Tang, and Xi Chu. Evoquant: Self-evolving verifier-guided strategy optimization for robust quantitative trading. *arXiv preprint arXiv:2607.12455*, 2026.
- Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952. doi: <https://doi.org/10.1111/j.1540-6261.1952.tb01525.x>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1540-6261.1952.tb01525.x>.

-
- R Thomas McCoy, Shunyu Yao, Dan Friedman, Matthew Hardy, and Thomas L Griffiths. Embers of autoregression: Understanding large language models through the problem they are trained to solve. *arXiv preprint arXiv:2309.13638*, 2023.
- James McDermott, Alexandros Agapitos, Anthony Brabazon, and Michael O'Neill. Geometric semantic genetic programming for financial data. In *European Conference on the Applications of Evolutionary Computation*, pages 215–226. Springer, 2014.
- R David McLean. Idiosyncratic risk, long-term reversal, and momentum. *Journal of Financial and Quantitative Analysis*, 45(4):883–906, 2010.
- Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PmLR, 2016.
- John Moody and Matthew Saffell. Learning to trade via direct reinforcement. *IEEE transactions on neural Networks*, 12(4):875–889, 2001.
- Ummara Mumtaz and Summaya Mumtaz. Potential of chatgpt in predicting stock market trends based on twitter sentiment analysis. *arXiv preprint arXiv:2311.06273*, 2023.
- Kei Nakagawa, Tomoki Ito, Masaya Abe, and Kiyoshi Izumi. Deep recurrent factor model: interpretable non-linear and time-varying multi-factor model. *arXiv preprint arXiv:1901.11493*, 2019.
- Yasuhiro Nakayama, Tomochika Sawaki, Issei Furuya, and Shunsuke Tamura. Text-based correlation matrix in multi-asset allocation. *arXiv preprint arXiv:2405.14247*, 2024.
- Revant Nayar, Dnyanesh Kulkarni, and El Mehdi Ainasse. Topological risk parity. *arXiv preprint arXiv:2604.16773*, 2026.
- Philip Ndikum and Serge Ndikum. Advancing investment frontiers: Industry-grade deep reinforcement learning for portfolio optimization. *arXiv preprint arXiv:2403.07916*, 2024.
- Christopher Neely, Paul Weller, and Rob Dittmar. Is technical analysis in the foreign exchange market profitable? a genetic programming approach. *Journal of financial and Quantitative Analysis*, 32(4):405–426, 1997.
- Yurii Nesterov and Arkadii Nemirovskii. *Interior-point polynomial algorithms in convex programming*. SIAM, 1994.
- Jeremy K Nguyen. Human bias in ai models? anchoring effects and mitigation strategies in large language models. *Journal of Behavioral and Experimental Finance*, 43:100971, 2024.
- Hui Niu, Siyuan Li, and Jian Li. Metatrader: An reinforcement learning approach integrating diverse policies for portfolio optimization. In *Proceedings of the 31st ACM international conference on information & knowledge management*, pages 1573–1583, 2022.
- Miquel Noguer i Alonso. Path portfolio optimization: Defect, lift, and the price of path complexity. *arXiv preprint arXiv:2608.02355*, 2026.
- Adamantios Ntakaris, Martin Magris, Juho Kannianen, Moncef Gabbouj, and Alexandros Iosifidis. Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods. *Journal of Forecasting*, 37(8):852–866, 2018.
- Daniel Cunha Oliveira, Dylan Sandfelder, André Fujita, Xiaowen Dong, and Mihai Cucuringu. Tactical asset allocation with macroeconomic regime detection. *Quantitative Finance*, pages 1–27, 2026.
- Joel Ong and Dorien Herremans. Optimising multi-timeframe momentum portfolio via multi-task learning with multi-gate mixture of experts. *Available at SSRN 4941669*.

-
- Razvan Oprisor and Roy Kwon. Multi-period portfolio optimization with investor views under regime switching. *Journal of Risk and Financial Management*, 14(1):3, 2020.
- Hua Ouyang, Niao He, Long Tran, and Alexander Gray. Stochastic alternating direction method of multipliers. In *International conference on machine learning*, pages 80–88. PMLR, 2013.
- Chaofan Pan, Lingfei Ren, Linbo Xiong, Yonghao Li, Wei Wei, and Xin Yang. Regime-adaptive continual learning for portfolio management. *arXiv preprint arXiv:2606.00143*, 2026a.
- Jianming Pan, Zeqi Ye, Xiao Yang, Xu Yang, Weiqing Liu, Lewen Wang, and Jiang Bian. Bpqp: A differentiable convex optimization framework for efficient end-to-end learning. *Advances in Neural Information Processing Systems*, 37:77468–77493, 2024.
- Jinfeng Pan and Jiahao Chen. Sbca: Cross-modal bert-driven actor-critic for multi-asset portfolio optimization. *arXiv preprint arXiv:2605.01384*, 2026.
- Yingjian Pan, Xiaowei Ding, and Kay Giesecke. Agentic empirical asset pricing: Methodological foundations. *arXiv preprint arXiv:2609.00731*, 2026b.
- Sahong Park, Suhwan Park, Hoyoung Lee, Gakyung Kwon, Wonbin Ahn, Jaewon Choi, Alejandro Lopez-Lira, Yoon Kim, Chanyeol Choi, Hyeongwoo Kong, et al. Your ai, on a dial: Controlling investment bias in llms with a single neuron. *arXiv preprint arXiv:2608.22852*, 2026.
- Manuel Parra-Diaz and Carlos Castro-Iragorri. Deep declarative risk budgeting portfolios. *arXiv preprint arXiv:2504.19980*, 2025.
- Yagna Patel. Optimizing market making using multi-agent reinforcement learning. *arXiv preprint arXiv:1812.10252*, 2018.
- Varun Narayan Kannan Pillai, Akshay Ajith, et al. Generating alpha: A hybrid ai-driven trading system integrating technical analysis, machine learning and financial sentiment for regime-adaptive equity strategies. *arXiv preprint arXiv:2601.19504*, 2026.
- Ramin Pishehvar. A three-phase foundation model for tax-aware personalized portfolio management. *arXiv preprint arXiv:2606.30997*, 2026.
- Austin Pollok and Kevin Robik. End-to-end parametric portfolio policies for cross-asset futures timing: When do ai models beat simple rules? *arXiv preprint arXiv:2607.00475*, 2026.
- Jean-Yves Potvin, Patrick Soriano, and Maxime Vallée. Generating trading rules on the stock markets with genetic programming. *Computers & Operations Research*, 31(7):1033–1047, 2004.
- Michael JD Powell. A method for nonlinear constraints in minimization problems. *Optimization*, pages 283–298, 1969.
- Lingfei Qian, Weipeng Zhou, Yan Wang, Xueqing Peng, Han Yi, Yilun Zhao, Jimin Huang, Qianqian Xie, and Jian-yun Nie. Fino1: On the transferability of reasoning-enhanced llms and reinforcement learning to finance. *arXiv preprint arXiv:2502.08127*, 2025.
- Gabriel Nixon Raj. Adaptive and regime-aware rl for portfolio optimization. *arXiv preprint arXiv:2509.14385*, 2025.
- Pedro Reis, Ana Paula Serra, and João Gama. A deep learning framework for medium-term covariance forecasting in multi-asset portfolios. *arXiv preprint arXiv:2503.01581*, 2025.
- Junji Ren, Junjie Zhao, Shengcai Liu, and Peng Yang. From linear to hierarchical: Evolving tree-structured thoughts for efficient alpha mining. *arXiv preprint arXiv:2508.16334*, 2025.
- Tao Ren, Ruihan Zhou, Jinyang Jiang, Jiafeng Liang, Qinghao Wang, and Yijie Peng. Riskminer: Discovering formulaic alphas via risk seeking monte carlo tree search. In *Proceedings of the 5th ACM International Conference on AI in Finance*, pages 752–760, 2024a.

-
- Weizhe Ren, Yichen Qin, and Yang Li. Alpha mining and enhancing via warm start genetic programming for quantitative investment. *arXiv preprint arXiv:2412.00896*, 2024b.
- Jean-Charles Richard and Thierry Roncalli. Constrained risk budgeting portfolios: Theory, algorithms, applications & puzzles. *arXiv preprint arXiv:1902.05710*, 2019.
- Katerina Rigana, Ernst C Wit, and Samantha Cook. Navigating market turbulence: Insights from causal network contagion value at risk. *arXiv preprint arXiv:2402.06032*, 2024.
- Jonathan Sadighian. Deep reinforcement learning in cryptocurrency market making. *arXiv preprint arXiv:1911.08647*, 2019.
- Nirvik Sahoo, Chyng Wen Tee, and Paul Robert Griffin. Photonic quantum computing vs. classical solvers in constrained factor portfolio optimization. *arXiv preprint arXiv:2608.14134*, 2026.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Slimane Sefiane and Mohamed Benbouziane. Portfolio selection using genetic algorithm. 2012.
- Mostafa Shabani, Dat Thanh Tran, Juho Kannianen, and Alexandros Iosifidis. Augmented bilinear network for incremental multi-stock time-series classification. *Pattern Recognition*, 141:109604, 2023.
- Ning Shen. Alpha-gfn: Mining formulaic alpha factors with generative flow networks, 2024. URL <https://github.com/nshen7/alpha-gfn>.
- Runze Shi, Shengyu Yan, Yuecheng Cai, and Chengxi Lv. Hubble: An llm-driven agentic framework for safe, diverse, and reproducible alpha factor discovery. *arXiv e-prints*, pages arXiv–2604, 2026a.
- Yu Shi, Yitong Duan, and Jian Li. Navigating the alpha jungle: An llm-powered mcts framework for formulaic alpha factor mining. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 997–1005, 2026b.
- Hong-Gi Shin, Sukhyun Jeong, Eui-Yeon Kim, Sungho Hong, Young-Jin Cho, and Yong-Hoon Choi. Synergistic formulaic alpha generation for quantitative trading based on reinforcement learning. In *2024 International Conference on Information Networking (ICOIN)*, pages 42–46. IEEE, 2024.
- Yizhan Shu and John M Mulvey. Dynamic factor allocation leveraging regime-switching signals. *arXiv preprint arXiv:2410.14841*, 2024.
- Yizhan Shu, Chenyu Yu, and John M Mulvey. Dynamic asset allocation with asset-specific regime forecasts. *Annals of Operations Research*, 346(1):285–318, 2025.
- Anmol Singhal and Navya Singhal. Analogy-driven financial chain-of-thought (ad-fcot): A prompting approach for financial sentiment analysis. In *2025 Artificial Intelligence for Business (AIB)*, pages 41–46. IEEE, 2025.
- Masoud Soleimani. Llm-generated counterfactual stress scenarios for portfolio risk simulation via hybrid prompt-rag pipeline. *arXiv preprint arXiv:2512.07867*, 2025.
- Changkyu Song, Sejong Yoon, and Vladimir Pavlovic. Fast admm algorithm for distributed optimization with adaptive penalty. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.
- Zifan Song, Kaitao Song, Guosheng Hu, Ding Qi, Junyao Gao, Xiaohua Wang, Dongsheng Li, and Cairong Zhao. Trade in minutes! rationality-driven agentic system for quantitative financial trading. *arXiv preprint arXiv:2510.04787*, 2025.
- Srikan Sood, Kassiani Papasotiriou, Marius Vaiciulis, and Tucker Balch. Deep reinforcement learning for optimal portfolio allocation: A comparative study with mean-variance optimization. *arXiv preprint arXiv:2602.17098*, 2026.

-
- Yejun Soun, Hosung Lee, Suyoung Park, and U Kang. Stable: Shift-tolerant allocation via black-litterman using conditional diffusion estimates. In *The Fourteenth International Conference on Learning Representations*.
- Eleni Straitouri, Cheol Woo Kim, and Milind Tambe. Generating robust portfolios of optimization models using large language models. *arXiv preprint arXiv:2605.27013*, 2026.
- Yaoyu Su. Alphag-opd: Reliability-gated sibling counterfactuals for on-policy distillation in symbolic alpha factor discovery. *arXiv preprint arXiv:2608.01303*, 2026.
- YiHeng Sun, Yanyan Shen, and Zhaoyang Liu. Alphacon: In-context adaptation for dynamic alpha generation.
- Chong Hui Tan and Qinxu Ding. Responsible large language models in finance: A descriptive bibliometric overview and taxonomy of responsibility. *FinTech*, 5(3):71, 2026.
- Ziyi Tang, Zechuan Chen, Jiarui Yang, Jiayao Mai, Yongsen Zheng, Keze Wang, Jinrui Chen, and Liang Lin. Alphaagent: Llm-driven alpha mining with regularized exploration to counteract alpha decay. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 2813–2822, 2025.
- Ziyi Tang, Xuexiong Yin, Weixing Chen, Zechuan Chen, Yongsen Zheng, Wenxuan Ye, Keze Wang, and Liang Lin. Alphaagentevo: Evolution-oriented alpha mining via self-evolving agentic reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Benjamin Turtel, Paul Wilczewski, Danny Franklin, and Kris Skotheim. Foresight learning for sec risk prediction. *arXiv preprint arXiv:2601.19189*, 2026.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Ajay Kumar Verma, Nunik Srikanth Putri, and Neo Paul Lesupi. Regime-based portfolio allocation using hidden markov models and reinforcement learning. *arXiv preprint arXiv:2605.27848*, 2026.
- Alina Voronina, Oleksandr Romanko, Ruiwen Cao, Roy H Kwon, and Rafael Mendoza-Arriaga. Generative ai-enhanced sector-based investment portfolio construction. *arXiv preprint arXiv:2512.24526*, 2025.
- Nanari Wada, Shunnosuke Ikeda, Yuichi Takano, and Jun-ya Gotoh. Safe screening rules for portfolio optimization with linear and cardinality constraints. *arXiv preprint arXiv:2608.01871*, 2026.
- Liyang Wang, Yu Cheng, Ao Xiang, Jingyu Zhang, and Haowei Yang. Application of natural language processing in financial risk detection. *arXiv preprint arXiv:2406.09765*, 2024a.
- Rundong Wang, Hongxin Wei, Bo An, Zhouyan Feng, and Jun Yao. Deep stock trading: A hierarchical reinforcement learning framework for portfolio optimization and order execution. *arXiv preprint arXiv:2012.12620*, 2020.
- Saizhuo Wang, Hang Yuan, Lionel M Ni, and Jian Guo. Quantagent: Seeking holy grail in trading by self-improving large language model. *arXiv preprint arXiv:2402.03755*, 2024b.
- Saizhuo Wang, Hang Yuan, Leon Zhou, Lionel Ni, Heung Yeung Shum, and Jian Guo. Alpha-gpt: Human-ai interactive alpha mining for quantitative investment. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 196–206, 2025.
- Shengkun Wang, Taoran Ji, Linhan Wang, Yanshen Sun, Shang-Ching Liu, Amit Kumar, and Chang-Tien Lu. Stocktime: A time series specialized large language model architecture for stock price prediction. *arXiv preprint arXiv:2409.08281*, 2024c.

-
- Tianqi Wang and Shubham Singh. Kan based autoencoders for factor models. *arXiv preprint arXiv:2408.02694*, 2024.
- Yanlong Wang, Jian Xu, Hongkang Zhang, Shao-Lun Huang, Danny Dongning Sun, and Xiao-Ping Zhang. Factorminer: A self-evolving agent with skills and experience memory for financial alpha discovery. *arXiv preprint arXiv:2602.14670*, 2026.
- Ye Wang and Kexin Gu. Who invests, who gets funded: Gender and racial bias in llm-generated investment advice: Y. wang and k. gu. *Journal of Business Ethics*, pages 1–37, 2026.
- Yilun Wang and Shengjie Guo. Rvrae: A dynamic factor model based on variational recurrent autoencoder for stock returns prediction. *arXiv preprint arXiv:2403.02500*, 2024.
- Yining Wang, Jinman Zhao, and Yuri Lawryshyn. Gpt-signal: Generative ai for semi-automated feature engineering in the alpha research process. In *Proceedings of the Eighth Financial Technology and Natural Language Processing and the 1st Agent AI for Scenario Planning*, pages 42–53, 2024d.
- Yueyi Wang and Qiyao Wei. Event-aware sentiment factors from llm-augmented financial tweets: A transparent framework for interpretable quant trading. *arXiv preprint arXiv:2508.07408*, 2025.
- Zhangyuhua Weng, Shengli Zhang, Taotao Wang, and Yihan Xia. Alphalogics: A market logic-driven multi-agent system for scalable and interpretable alpha factor generation. *arXiv preprint arXiv:2603.20247*, 2026.
- Peter Williams. Factor based approaches to risk parity. *Available at SSRN 2398202*, 2014.
- Kieran Wood, Stephen J. Roberts, and Stefan Zohren. Deepm: Regime-robust deep learning for systematic macro portfolio management. *arXiv preprint arXiv:2601.05975*, 2026.
- Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhajan Kambadur, David Rosenberg, and Gideon Mann. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.
- Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. Tradingagents: Multi-agents llm financial trading framework. *arXiv preprint arXiv:2412.20138*, 2024.
- Qianqian Xie, Weiguang Han, Xiao Zhang, Yanzhao Lai, Min Peng, Alejandro Lopez-Lira, and Jimin Huang. Pixiu: A large language model, instruction data and evaluation benchmark for finance. *arXiv preprint arXiv:2306.05443*, 2023.
- Qianqian Xie, Weiguang Han, Zhengyu Chen, Ruoyu Xiang, Xiao Zhang, Yueru He, Mengxi Xiao, Dong Li, Yongfu Dai, Duanyu Feng, et al. Finben: A holistic financial benchmark for large language models. *Advances in Neural Information Processing Systems*, 37:95716–95743, 2024.
- Feng Xu, Yan Yin, Xinyu Zhang, Tianyuan Liu, Shengyi Jiang, and Zongzhang Zhang. Alpha²: Discovering logical formulaic alphas using deep reinforcement learning. *arXiv preprint arXiv:2406.16505*, 2024.
- Wenyan Xu, Jiayu Chen, Dawei Xiang, Chen Li, Yonghong Hu, and Zhonghua Lu. Mining intraday risk factor collections via hierarchical reinforcement learning based on transferred options. *arXiv preprint arXiv:2501.07274*, 2025.
- Han Yang, Dong Hao, Zhuohan Wang, Qi Shi, and Xingtong Li. Alpha discovery via grammar-guided learning and search. *arXiv preprint arXiv:2601.22119*, 2026.
- Hongyang Yang, Xiao-Yang Liu, Shan Zhong, and Anwar Walid. Deep reinforcement learning for automated stock trading: An ensemble strategy. In *Proceedings of the first ACM international conference on AI in finance*, pages 1–8, 2020a.
- Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. Fingpt: Open-source financial large language models. *arXiv preprint arXiv:2306.06031*, 2023a.

-
- Hongyang Yang, Boyu Zhang, Neng Wang, Cheng Guo, Xiaoli Zhang, Likun Lin, Junlin Wang, Tianyu Zhou, Mao Guan, Runjia Zhang, et al. Finrobot: An open-source ai agent platform for financial applications using large language models. *arXiv preprint arXiv:2405.14767*, 2024a.
- Jie Yang, Yiqiu Tang, Yongjie Li, Lihua Zhang, and Haoran Zhang. Dynamic hedging strategies in derivatives markets with llm-driven sentiment and news analytics. In *arXiv preprint arXiv:2504.04295*, 2025.
- Shibo Yang, Jianxuan Xin, Qiang Ye, and Hao Xia. A co-evolutionary genetic programming framework for market-adaptive formulaic alpha generation. *Available at SSRN 5614909*, 2019.
- Xiao Yang, Weiqing Liu, Dong Zhou, Jiang Bian, and Tie-Yan Liu. Qlib: An ai-oriented quantitative investment platform. *arXiv preprint arXiv:2009.11189*, 2020b.
- Xu Yang, Haotian Chen, Wenjun Feng, Haoxue Wang, Zeqi Ye, Xinjie Shen, Xiao Yang, Shizhao Sun, Weiqing Liu, and Jiang Bian. Collaborative evolving strategy for automatic data-centric development. *arXiv preprint arXiv:2407.18690*, 2024b.
- Yi Yang, Yixuan Tang, and Kar Yan Tam. Investlm: A large language model for investment using financial domain instruction tuning. *arXiv preprint arXiv:2309.13064*, 2023b.
- Kumar Yashaswi. Deep reinforcement learning for portfolio optimization using latent feature state space (lfss) module. *arXiv preprint arXiv:2102.06233*, 2021.
- Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. Reevo: Large language models as hyper-heuristics with reflective evolution. *Advances in neural information processing systems*, 37:43571–43608, 2024.
- Weicheng Ye, Youran Sun, Xingyu Ren, Shun Yao Yu, Chugang Yi, and Haizhao Yang. Agonalpha: Autonomous alpha discovery via prompt economy and scalable agentic search. *arXiv preprint arXiv:2608.11250*, 2026.
- Jingyang Yi, Jian Yang, Y. J. Jin, Yuqi Li, and Jian Li. Alphaschema: Exploring the space of trading semantics for llm-based alpha mining. *arXiv preprint arXiv:2607.26642*, 2026.
- Hang Yu, Zifan Zheng, Jeff Z Pan, Tongliang Liu, Zhiyong Wang, and Fengxiang He. Alphamemo: Structured search-process memory for self-evolving alpha mining agents. *arXiv preprint arXiv:2606.20625*, 2026a.
- Haofei Yu, Fenghai Li, and Jiaxuan You. Livetradebench: Seeking real-world alpha with large language models. *arXiv preprint arXiv:2511.03628*, 2025a.
- Jiayang Yu and Kuo-Chu Chang. Neural network predictive modeling on dynamic portfolio management—a simulation-based portfolio optimization approach. *Journal of Risk and Financial Management*, 13(11): 285, 2020.
- Shuo Yu, Hongyan Xue, Xiang Ao, Feiyang Pan, Jia He, Dandan Tu, and Qing He. Generating synergistic formulaic alpha collections via reinforcement learning. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*, pages 5476–5486, 2023.
- Xinwei Yu, Yiyang Fu, Mingcheng Fan, Enqi Li, Yilin Gao, and Shugong Xu. Towards autonomous formulaic alpha discovery: An evolutionary computation perspective. *arXiv preprint arXiv:2608.01789*, 2026b.
- Yangyang Yu, Zhiyuan Yao, Haohang Li, Zhiyang Deng, Yuechen Jiang, Yupeng Cao, Zhi Chen, Jordan W Suchow, Zhenyu Cui, Rong Liu, et al. Fincon: A synthesized llm multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. *Advances in Neural Information Processing Systems*, 37:137010–137045, 2024.
- Yangyang Yu, Haohang Li, Zhi Chen, Yuechen Jiang, Yang Li, Jordan W Suchow, Denghui Zhang, and Khaldoun Khashanah. Finnem: A performance-enhanced llm trading agent with layered memory and character design. *IEEE Transactions on Big Data*, 2025b.

-
- Yishuo Yuan, Jiayi Sheng, Sirui Zeng, Jiaqi Wang, and Jiaheng Liu. Alphacrafter: A full-stack multi-agent framework for cross-sectional quantitative trading. *arXiv preprint arXiv:2605.05580*, 2026.
- Kamer Ali Yuksel. Alphaloss: Evolving portfolio optimization loss functions. *Available at SSRN 5263279*, 2025.
- Kamer Ali Yuksel and Hassan Sawaf. Llm-driven qualitative portfolio evolution and correlation-aware cluster tournaments. *Available at SSRN 5835462*, 2025.
- Boyu Zhang, Hongyang Yang, and Xiao-Yang Liu. Instruct-fingpt: Financial sentiment analysis by instruction tuning of general-purpose large language models. *arXiv preprint arXiv:2306.12659*, 2023.
- Chong Zhang, Xinyi Liu, Zhongmou Zhang, Mingyu Jin, Lingyao Li, Zhenting Wang, Wenyue Hua, Dong Shu, Suiyuan Zhu, Xiaobo Jin, et al. When ai meets finance (stockagent): Large language model-based stock trading in simulated real-world environments. *arXiv preprint arXiv:2407.18957*, 2024a.
- Jinping Zhang and Keming Zhang. Portfolio selection models based on interval-valued conditional value at risk (icvar) and empirical analysis. *arXiv preprint arXiv:2201.02987*, 2022.
- Liangliang Zhang. Objective-oriented quantitative investment: A specification-driven framework for automated synthesis of trading strategy pipelines. *arXiv preprint arXiv:2608.10410*, 2026.
- Lingzhe Zhang, Tong Jia, Yunpeng Zhai, Zixuan Xie, Chiming Duan, Minghua He, Philip S. Yu, and Ying Li. From feedback loops to policy updates: Reinforcement fine-tuning for llm-based alpha factor discovery. *arXiv preprint arXiv:2605.15412*, 2026.
- Tianping Zhang, Yuanqi Li, Yifei Jin, and Jian Li. Autoalpha: an efficient hierarchical evolutionary algorithm for mining alpha factors in quantitative investment. *arXiv preprint arXiv:2002.08245*, 2020.
- Wentao Zhang, Lingxuan Zhao, Haochong Xia, Shuo Sun, Jiaze Sun, Molei Qin, Xinyi Li, Yuqing Zhao, Yilei Zhao, Xinyu Cai, et al. A multimodal foundation agent for financial trading: Tool-augmented, diversified, and generalist. In *Proceedings of the 30th acm sigkdd conference on knowledge discovery and data mining*, pages 4314–4325, 2024b.
- Zihao Zhang, Stefan Zohren, and Stephen Roberts. Deeplob: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, 67(11):3001–3012, 2019.
- Fanyu Zhao. Portfolio construction using black-litterman model and factors. *arXiv preprint arXiv:2311.04475*, 2023.
- Junjie Zhao, Chengxi Zhang, Min Qin, and Peng Yang. Quantfactor reinforce: mining steady formulaic alpha factors with variance-bounded reinforce. *IEEE Transactions on Signal Processing*, 2025a.
- Junjie Zhao, Chengxi Zhang, Chenkai Wang, and Peng Yang. Learning from expert factors: Trajectory-level reward shaping for formulaic alpha mining. *arXiv preprint arXiv:2507.20263*, 2025b.
- Tianjiao Zhao, Jingrao Lyu, Stokes Jones, Harrison Garber, Stefano Pasquali, and Dhagash Mehta. Alphaagents: Large language model based multi-agents for equity portfolio constructions. *arXiv preprint arXiv:2508.11152*, 2025c.
- Ke Zhou, Xinman Huang, Xinnan Chen, and Jianjun Gao. An end-to-end direct reinforcement learning approach for multi-factor based portfolio management. *Available at SSRN 4729683*, 2024.
- Tian Zhou, Peisong Niu, Liang Sun, Rong Jin, et al. One fits all: Power general time series analysis by pretrained lm. *Advances in neural information processing systems*, 36:43322–43355, 2023.
- Xun Yu Zhou and Duan Li. Continuous-time mean-variance portfolio selection: A stochastic lq framework. *Applied Mathematics & Optimization*, 42(1):19–33, 2000.
- Zhi-Hua Zhou and Ji Feng. Deep forest. *National science review*, 6(1):74–86, 2019.

Qilu Zhu, Zijun Lu, Jianmin Zhu, Ning Chen, Shuo Yin, and Simon Fong. Alphaseek: Trajectory-level self-iterative factor mining framework for multi-source financial data. In *International Conference on Intelligent Computing*, pages 265–276. Springer, 2026.

Zhoufan Zhu and Ke Zhu. Alphaqcm: Alpha discovery in finance with distributional reinforcement learning. In *Forty-second International Conference on Machine Learning*, 2025.